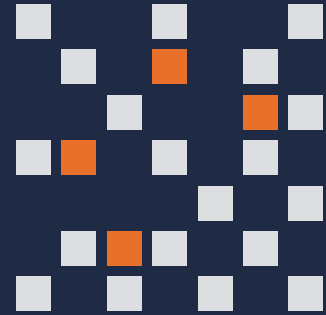


NIRNAY

An indigenous LP / MILP / QP optimisation solver, written from the mathematics up



Technical report

Algorithms, numerical safeguards, verification and benchmark results

Problem statement ID	SIH26119
Problem statement title	Indigenous GPU-Accelerated Optimization Solver (Sovereign Alternative to Xpress/CPLEX)
Organisation	Mangalore Refinery and Petrochemicals Limited (MRPL)
Theme	Smart Automation
Category	Software
Team	ZeroCloud
Software	NIRNAY 0.1.0 (Python 3.12.10, NumPy 2.4.6, Numba 0.65.0, CuPy 14.2.0)
Date	September 30, 2026

Every benchmark number in this report is read from the CSV files in `results/by deliverables/report/gen_data.py`; none is typed by hand. Rebuilding after a new sweep updates the text, tables and plots together.

Summary

Problem statement SIH26119 asks for an optimisation solver that India owns: one that reads the models refinery planners already write, solves linear, mixed-integer and quadratic programmes, uses GPUs where they help, and is not built on Xpress, CPLEX or any other existing solver library. NIRNAY is our answer. Every algorithm and every factorisation in it is implemented in the repository: an MPS/QPS reader, presolve with exact primal and dual postsolve, scaling, a Gilbert–Peierls sparse LU with eta updates and basis repair, an up-looking sparse Cholesky, a quasi-definite LDL^T , a minimum-degree ordering, a bounded dual simplex, a Mehrotra interior-point method, a restarted primal–dual hybrid gradient method (PDLP) on CPU and GPU, a branch-and-bound code with reliability branching, propagation, heuristics and Gomory cuts, and a QP interior-point method. HiGHS and Clarabel appear only in the benchmark harness, as independent references that check every answer.

This report describes each algorithm as the code implements it, with its equations and pseudocode; the numerical failures we hit and how each fix works; how answers are verified; and the benchmark results, failures included. Section 13 states plainly where NIRNAY is behind: it is slower than HiGHS, and it proves fewer MIPLIB instances optimal.

At a glance (generated from results/)

90/90	86/90	121/138	25/64
Netlib LPs solved by the dual simplex (objective within 10^{-6} of HiGHS)	Netlib LPs solved by the interior-point method	Maros–Mészáros convex QPs solved (within 10^{-6} of Clarabel)	MIPLIB 3 instances proven optimal in 60 s
3.7×	8.3×	15.8×	0
dual simplex time vs. HiGHS, shifted geometric mean (shift 1 s)	dual simplex vs. HiGHS, geometric mean of per-instance ratios	largest GPU-over-CPU speed-up of our PDLP on the large LPs run so far	dual-feasibility failures in 88 Netlib solutions checked after postsolve

Where we stand

NIRNAY is correct on the standard LP and QP test sets and substantially slower than a mature open-source solver: the dual simplex takes 3.7 times as long as HiGHS in shifted geometric mean over the 90 Netlib problems (3.9 times with a 10 s shift, and 8.3 times as the geometric mean of per-instance ratios). On MIPLIB 3 with a 60 s limit, branch-and-bound proves 25 of the 64 instances optimal, against 45 for HiGHS under the same limit. The engines are compiled with Numba inside the inner loops, but the iteration drivers are Python, and the MIP code lacks most of the presolve, cut families and heuristics that close gaps in production solvers.

Contents

Summary	i
1 Problem statement and motivation	1
1.1 The problem statement	1
1.2 What NIRNAY is	1
1.3 Design principles	1
1.4 Contributions and outline	2
2 Mathematical formulations	2
2.1 The model every engine reads	2
2.2 Duality and optimality conditions	2
2.3 Computational forms	3
3 System architecture	4
3.1 Layers	4
3.2 The path of a solve	4
3.3 Input: the MPS/QPS reader	4
3.4 Scaling	6
4 Presolve and postsolve	6
4.1 Reductions	6
4.2 Postsolve and dual recovery	7
4.3 An order-dependence bug in singleton-row duals	8
4.4 What presolve removes	8
5 Sparse linear algebra	9
5.1 Sparse LU of the simplex basis	9
5.2 Sparse Cholesky for the normal equations	10
5.3 Minimum-degree ordering and fill-in	11
5.4 Assembling the normal matrix	11
5.5 Quasi-definite LDL^T for the QP augmented system	11
6 The bounded dual simplex method	12
6.1 Basis, primal and dual values	12
6.2 One iteration	13
6.3 Phase 1, perturbation and cleanup	14

6.4	Warm starts for branch-and-bound	16
7	The interior-point method for LP	16
7.1	Newton system	16
7.2	Predictor–corrector	16
7.3	Termination and stall detection	17
8	PDLP: a first-order LP method on CPU and GPU	18
8.1	The PDHG step	18
8.2	Two backends	19
8.3	Results	20
9	Branch-and-bound for mixed-integer programmes	21
9.1	The search	21
9.2	Domain propagation	22
9.3	Primal heuristics	22
9.4	Gomory mixed-integer cuts	23
10	The interior-point method for convex QP	24
10.1	The regularised augmented system	24
11	Numerical safeguards: what went wrong and what fixed it	25
11.1	Dual simplex	26
11.2	Interior-point method	27
11.3	QP interior point: the LDL^T pivot floor	27
11.4	Branch-and-bound: heap tie-breaking	27
11.5	Presolve: dual postsolve order	28
12	Verification methodology	28
12.1	Independent references, in separate processes	28
12.2	What counts as solved	28
12.3	Optimality conditions after postsolve	29
12.4	Regression tests and file round trips	29
12.5	How speed is measured	29
13	Benchmark results	29
13.1	Netlib LP: correctness	30
13.2	Netlib LP: speed	30

13.3	Interior-point failures	31
13.4	MIPLIB 3	32
13.5	Maros–Mészáros QP	34
14	Industrial case studies	35
14.1	The models	35
14.2	Reference solutions and NIRNAY’s results	36
15	Competitor analysis	37
15.1	What the comparison shows	37
15.2	Licences and cost	38
16	Limitations and roadmap	39
16.1	Limitations	39
16.2	Roadmap, in order of expected payoff	39
16.3	Conclusion	40
	References	41
A	Full benchmark tables	44
B	Typesetting toolchain	50
C	Reproducing this report	51

List of Figures

1	System architecture	4
2	Data flow of a solve	5
3	Fill-in and ordering	11
4	Quasi-definite KKT block structure	12
5	Dual simplex iteration loop	15
6	Interior-point iteration	17
7	PDLP on the GPU	19
8	PDLP on large LPs	21
9	Branch-and-bound tree	22
10	Solved instances per suite	30
11	Distribution of relative objective errors	31
12	Performance profile on Netlib	32
13	Time against reference time	33
14	Dual simplex iterations against problem size	33
15	MIPLIB 3: gap at the time limit	34

List of Tables

1	Solver modules	5
2	Presolve reductions on Netlib	8
3	PDLP on large LPs: CPU against GPU	20
4	Dual simplex: problems changed by the fixes	25
5	Interior point: problems changed by the fixes	27
6	Summary of the benchmark sweeps	30
7	Interior-point failures on Netlib	32
8	Case-study models	35
9	HiGHS reference results for the cases	36
10	NIRNAY on the case studies	37
11	Comparator solvers on the Netlib LP set	38
12	Comparator solvers on the MIPLIB 3 set	39
13	Comparator solvers on the Maros–Mészáros set	39
14	Netlib: every problem, all three LP engines	44
15	MIPLIB 3: every instance run	46
16	Maros–Mészáros: every problem	47

List of Algorithms

1	Left-looking sparse LU with threshold pivoting and repair (<code>linalg/lu.py</code>)	10
2	Bound-flipping ratio test with Harris’s two passes (<code>dual_ratio_test</code>)	13
3	Mehrotra predictor–corrector for LP (<code>lp/ipm.py</code>)	18
4	Branch-and-bound with plunging (<code>mip/bnb.py</code> , simplified)	24

1 Problem statement and motivation

1.1 The problem statement

MRPL's problem statement SIH26119, *Indigenous GPU-Accelerated Optimization Solver (Sovereign Alternative to Xpress/CPLEX)*, starts from a practical dependency. Refinery crude selection, blending, production planning, scheduling and product distribution in India are formulated as linear and mixed-integer programmes and solved with a small number of foreign commercial solvers. Those solvers are closed: their licences cost money every year, their algorithms cannot be inspected or adapted, and their availability depends on vendors outside the country. The statement asks for a solver that is developed here, handles the same problem classes, can use GPUs, and does not build on any existing solver library.

We read four requirements out of it:

- R1 Problem classes:** linear programmes (LP), mixed-integer linear programmes (MILP) and convex quadratic programmes (QP), the classes that planning and scheduling models fall into.
- R2 Independence:** no dependence on an existing optimisation solver or sparse direct solver library. The factorisations, orderings and algorithms must be our own code.
- R3 Interoperability:** read the MPS and QPS files that modelling tools and every public benchmark library already produce.
- R4 Acceleration:** use a GPU where the algorithm can exploit one.

1.2 What NIRNAY is

NIRNAY (Sanskrit/Hindi *nirṇay*, “decision”) is a Python package whose compute kernels are compiled to machine code with Numba and, for the GPU path, to CUDA with CuPy. It has three LP engines, which suit different problem shapes:

- a **bounded dual simplex** (Section 6), which produces vertex solutions and re-solves quickly after a bound change: this is the engine branch-and-bound uses;
- a **primal–dual interior-point method** (Section 7), whose iteration count grows only slowly with problem size;
- **PDLP**, a restarted primal–dual hybrid gradient method (Section 8). It needs only products with A and A^T , so it runs on a GPU and scales to LPs too large to factorise.

On top of these sit a **branch-and-bound** MILP solver (Section 9) and a **QP interior-point** method (Section 10). Figure 1 shows how the parts fit together.

1.3 Design principles

Implement, do not wrap.

Every factorisation (LU, Cholesky, LDL^T), ordering and solver loop is in the repository. The code depends on NumPy arrays and the Numba compiler, plus CuPy for the GPU path. It depends on no optimisation or sparse-direct library. Table 1 lists the modules with their sizes.

Follow the primary literature.

Each module's docstring cites the papers its algorithm comes from, and this report follows those citations. Where the code departs from a paper, the report says so.

Verify every answer independently.

Benchmarks run every instance in its own process and re-solve it with a reference solver (HiGHS

for LP and MILP, Clarabel for QP) that reads the file with its own parser (Section 12).

Report failures.

Tables in this report list every instance, solved or not, and Section 13 analyses the failures.

1.4 Contributions and outline

Section 2 fixes the mathematical forms. Section 3 describes the system and the path of a solve, and Section 4 the presolve and postsolve. Section 5 covers the sparse linear algebra on which every engine rests. Sections 6 to 8 describe the three LP engines, Section 9 branch-and-bound and Section 10 the QP method. Section 11 collects the numerical failures we met and how each was fixed. Section 12 explains how answers are checked, Section 13 gives the benchmark results, Section 14 the industrial case studies built from Indian public data, Section 15 a hands-on comparison with other solvers, and Section 16 the limitations and next steps. The appendices list every benchmark instance and the typesetting toolchain of this report.

2 Mathematical formulations

2.1 The model every engine reads

All engines read one problem form (`nirnay/model.py`):

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^\top x + \frac{1}{2} x^\top Q x + c_0 \\ \text{s.t.} \quad & r^l \leq A x \leq r^u, \\ & \ell \leq x \leq u, \\ & x_j \in \mathbb{Z} \quad (j \in \mathcal{I}), \end{aligned} \tag{1}$$

with $A \in \mathbb{R}^{m \times n}$ sparse, $Q \in \mathbb{R}^{n \times n}$ symmetric positive semidefinite (absent for LP and MILP), and bounds that may be infinite. A row with $r_i^l = r_i^u$ is an equality, a row with both bounds finite and different is a ranged row, and a row with neither bound is free. A maximisation is stored negated with a flag `sense = -1`, so every engine minimises and the reported objective and duals come back in the sense of the file. The problem is an LP when $Q = 0$ and $\mathcal{I} = \emptyset$, a MILP when $\mathcal{I} \neq \emptyset$, and a QP when $Q \neq 0$.

A is held in compressed sparse column (CSC) form: `colptr` (length $n + 1$), `rowidx` and `vals` (length $\text{nnz}(A)$). The simplex, the LU factorisation and presolve all traverse columns. Row access, which propagation and the pivot row need, uses $A^\top$ in CSC form (that is, A in CSR form), built once and cached. The arrays are plain NumPy arrays so that Numba kernels can take them directly.

2.2 Duality and optimality conditions

For the LP case ($Q = 0$) of (1), let $y \in \mathbb{R}^m$ be the row duals and $z = c - A^\top y$ the reduced costs. A primal feasible x is optimal if and only if there is a y with

$$\begin{aligned} y_i > 0 &\Rightarrow a_i^\top x = r_i^l, & y_i < 0 &\Rightarrow a_i^\top x = r_i^u, \\ z_j > 0 &\Rightarrow x_j = \ell_j, & z_j < 0 &\Rightarrow x_j = u_j. \end{aligned} \tag{2}$$

The dual objective is

$$\phi(y) = \sum_i (r_i^l y_i^+ - r_i^u y_i^-) + \sum_j (\ell_j z_j^+ - u_j z_j^-) + c_0, \quad t^+ = \max(t, 0), \quad t^- = \max(-t, 0), \tag{3}$$

and weak duality $c^\top x + c_0 \geq \phi(y)$ holds for every primal feasible x and every y for which (3) is finite. Section 12 tests (2) on the solutions NIRNAY returns, after postsolve and in the original, unscaled variables.

For QP the reduced cost becomes $z = c + Qx - A^\top y$ and the dual objective gains the term $-\frac{1}{2}x^\top Qx$.

2.3 Computational forms

Each engine converts (1) into the form its algebra needs. Each conversion is exact, and each engine maps its solution back itself.

Simplex: logicals for every row. The dual simplex (Section 6) gives every row a logical variable $s_i = a_i^\top x$:

$$\min c^\top x \quad \text{s.t.} \quad Ax - s = 0, \quad \ell \leq x \leq u, \quad r^l \leq s \leq r^u. \quad (4)$$

There are $n + m$ variables and every constraint is an equality with right-hand side zero. The constraint matrix is $[A \ -I]$, so the all-logical basis $B = -I$ is always available and needs no factorisation. Each variable has one of five statuses: basic, at its lower bound, at its upper bound, nonbasic free at zero, or fixed.

Interior point: bounded standard form. The IPM (Section 7) keeps equality rows as they are. It gives every inequality row a slack column ($a_i^\top x - s_i = 0$, $r_i^l \leq s_i \leq r_i^u$), drops free rows, and substitutes fixed columns into the right-hand side. It then shifts every variable, and reflects the ones with only an upper bound, so that each variable is free, nonnegative, or in a box $[0, U_j]$:

$$\min c^\top x \quad \text{s.t.} \quad Bx = b, \quad x_j \geq 0 \ (j \in \mathcal{L}), \quad x_j + w_j = U_j, \ w_j \geq 0 \ (j \in \mathcal{U}), \quad (5)$$

where $\mathcal{U} \subseteq \mathcal{L}$ are the boxed variables and the rest are free. This is the bounded standard form of Lustig, Marsten and Shanno [37]. The dual has y for the rows, $z \geq 0$ on $x_{\mathcal{L}} \geq 0$ and $s \geq 0$ on $w \geq 0$, and the optimality conditions are

$$Bx = b, \quad x_{\mathcal{U}} + w = U, \quad B^\top y + z - s = c, \quad x_j z_j = 0 \ (j \in \mathcal{L}), \quad w_j s_j = 0 \ (j \in \mathcal{U}). \quad (6)$$

PDLP: the saddle point on the original rows. PDLP (Section 8) uses the original rows and bounds, with no slack columns, and solves the saddle-point problem

$$\min_{x \in X} \max_{y \in Y} \mathcal{L}(x, y) = c^\top x - y^\top Ax + p(y), \quad p(y) = (r^l)^\top y^+ - (r^u)^\top y^-, \quad (7)$$

where $X = \{\ell \leq x \leq u\}$ is the box and Y is the sign cone of the rows: $y_i \geq 0$ when $r_i^u = +\infty$, $y_i \leq 0$ when $r_i^l = -\infty$, and $y_i = 0$ on a free row.

QP: the augmented system. The QP method (Section 10) uses the same bounded standard form (5) with the objective $c^\top x + \frac{1}{2}x^\top Qx$. Q is carried through the shift and the reflection: $x_{\text{model}} = x^0 + E \sigma \circ x'$ adds Qx^0 to c and $\frac{1}{2}(x^0)^\top Qx^0$ to c_0 , and Q is conjugated by the signs σ .

MILP. A MILP node is the LP relaxation (4) with tightened bounds on the integer columns. Apart from the cutting planes added at the root, branch-and-bound therefore never changes the matrix. It changes bounds, and the dual simplex re-optimises from the parent's basis (Section 9).

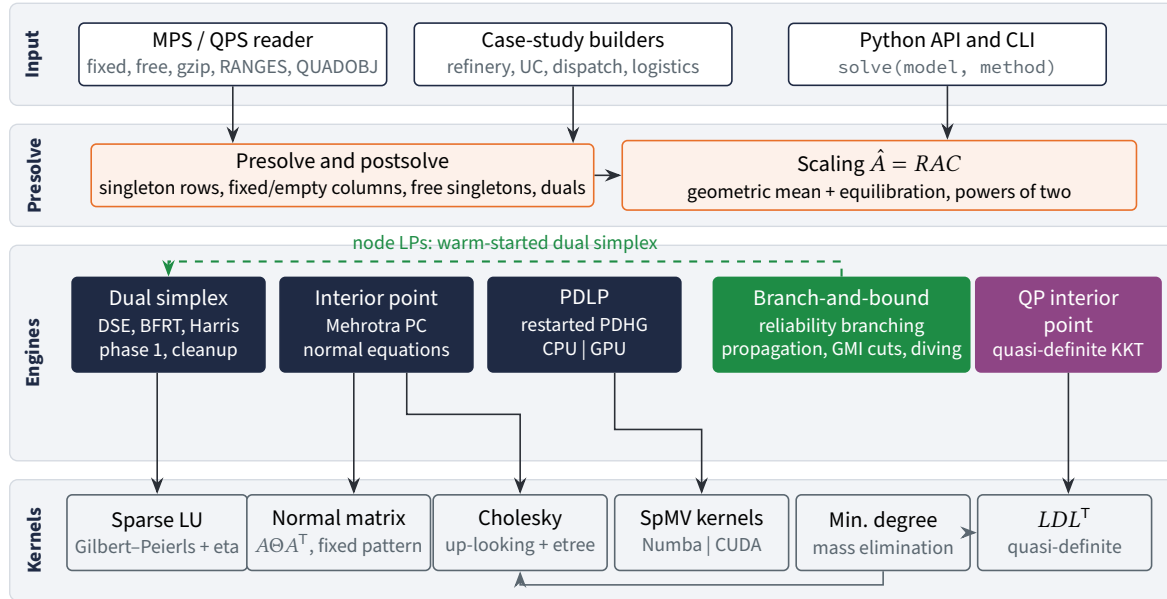


Figure 1: System architecture. Everything in the four bands is NIRNAY code. The dashed arrow shows that branch-and-bound solves its node LPs with the warm-started dual simplex. Minimum degree orders both the Cholesky and the LDL^T factorisations.

3 System architecture

3.1 Layers

Figure 1 shows the package in four layers. The input layer turns MPS/QPS files, the case-study builders or Python arrays into a `Model`. Presolve reduces the model and scaling equilibrates it. Five engines solve it. All engines share one layer of sparse kernels, and nothing below the API calls code outside the repository except NumPy, Numba and (for the GPU) CuPy. `nirnay.solve(model, method="auto")` picks branch-and-bound for a MILP, the QP interior-point method for a QP and the dual simplex for an LP. The interior-point method and PDLP are chosen by name.

3.2 The path of a solve

Figure 2 follows one LP or MILP from file to answer. Each transformation on the way in has an exact inverse on the way out, so the returned x , y and z satisfy the optimality conditions (2) of the model in the file, not only those of the scaled, reduced model the engine saw.

3.3 Input: the MPS/QPS reader

`nirnay/io/mps.py` reads fixed and free MPS, gzip-compressed files, OBJSENSE, integer MARKER blocks, RANGES, the bound types UP LO FX FR MI PL BV LI UI, an objective constant given as the RHS of the objective row, and the QP sections QUADOBJ (one triangle), QMATRIX and QSECTION (both triangles). It follows the conventions the benchmark libraries assume: an integer column in a MARKER block without bounds gets $[0, \infty)$; UP with a negative value on a column whose lower bound is still 0 sets the lower bound to $-\infty$; RANGES on an equality row extend upward for $R > 0$ and downward for $R < 0$; the objective constant is minus the RHS of the objective row. Fixed-format files whose names contain spaces (Netlib's `forplan`) are detected and parsed by column position. The earliest interior-point sweep failed to read `forplan` and `standgub` (Table 5); both read correctly now. A writer (`io/mps_write.py`)

Table 1: Solver modules of NIRNAY with their line counts (read from the source files by `gen_data.py`). The core solver is 4801 lines; the case-study builders add 1938.

Module (nirnay/...)	Lines	Role
<code>model.py</code>	210	problem form, CSC matrix, result object
<code>io/mps.py</code>	317	MPS/QPS reader (fixed and free format, gzip, RANGES, MARKER, QUADOBJ)
<code>io/mps_write.py</code>	221	MPS writer (round-trip checked)
<code>presolve/scaling.py</code>	109	geometric-mean and equilibration scaling in powers of two
<code>linalg/lu.py</code>	319	Gilbert–Peierls sparse LU, threshold pivoting, eta file, basis repair
<code>linalg/cholesky.py</code>	203	up-looking sparse Cholesky, elimination tree, pivot guard
<code>linalg/ldl.py</code>	131	quasi-definite LDL^T with dynamic regularisation
<code>linalg/ordering.py</code>	89	minimum degree with mass elimination and dense-row deferral
<code>linalg/normal.py</code>	110	normal-equations assembly on a fixed pattern
<code>linalg/sparse_kernels.py</code>	85	Numba SpMV and per-line norms
<code>lp/simplex.py</code>	502	bounded dual simplex, phase 1, perturbation, primal cleanup
<code>lp/_simplex_kernels.py</code>	165	pivot row, DSE pricing, bound-flipping ratio test, updates
<code>lp/ipm.py</code>	301	Mehrotra predictor–corrector on normal equations
<code>lp/pdip.py</code>	1009	restarted PDHG, CPU (Numba) and GPU (CuPy, CUDA graphs)
<code>mip/bnb.py</code>	475	branch-and-bound, reliability branching, heuristics, node heap
<code>mip/cuts.py</code>	147	Gomory mixed-integer cuts with safeguards
<code>mip/propagate.py</code>	127	activity-based bound propagation
<code>qp/ipm.py</code>	281	QP interior point on the regularised augmented system

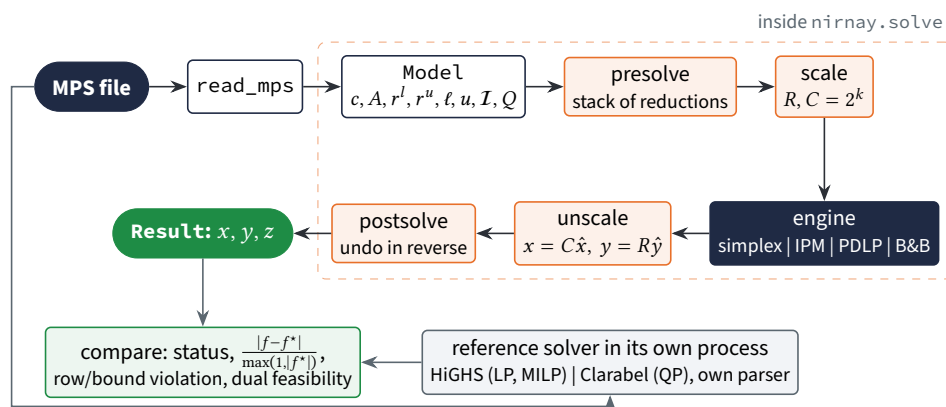


Figure 2: Data flow of a solve. The upper row reduces and scales the model. The engine solves it, and the lower row maps the solution back. The benchmark harness (bottom) re-solves the same file with a reference solver in a separate process and checks the answer.

emits free MPS. Read, write and re-read reproduce the arrays of the benchmark files, and HiGHS reads the original and the written file into the same model (Section 12).

3.4 Scaling

Industrial models mix units (tonnes, kilolitres, rupees, percentages), so the coefficients of A span many orders of magnitude and every factorisation loses digits to that spread. `presolve/scaling.py` computes diagonal R and C and solves the scaled problem with $\hat{A} = RAC$, $\hat{c} = Cc$, $\hat{r} = Rr$ and $\hat{\ell} = C^{-1}\ell$, $\hat{u} = C^{-1}u$, in two stages [19]:

1. **Geometric-mean passes.** Divide each row, then each column, by $\sqrt{\max_j |\hat{a}_{ij}| \cdot \min_j |\hat{a}_{ij}|}$. Stop after 8 passes, or earlier once a pass reduces the overall spread $\max |\hat{a}| / \min |\hat{a}|$ by less than 10%. Entries below 10^{-12} of the largest in their row or column are left out of the minimum: they carry no information in double precision, but a geometric mean would let them dominate. The QP KSIP has an entry of order 10^{-30} next to entries of order 1, and scaling by it ruined every other entry of its line.
2. **Equilibration.** A final column pass makes the largest entry of every column exactly 1.

Each factor is then rounded to the nearest power of two, $R_i \leftarrow 2^{\text{round}(\log_2 R_i)}$. Multiplying by a power of two changes only the floating-point exponent, so scaling adds no rounding error of its own, and unscaling ($x = C\hat{x}$, $y = R\hat{y}$, $z = C^{-1}\hat{z}$) returns exactly the numbers the engine computed. PDLP uses its own preconditioner (Ruiz and Pock–Chambolle, Section 8), because the convergence rate of a first-order method depends on the conditioning of A in a different way.

4 Presolve and postsolve

Presolve shrinks a model before any engine sees it. The reductions in `nirnay/presolve/presolve.py` are the classical ones of Andersen and Andersen [5] and Gondzio [28], restricted to those with an *exact primal and dual* postsolve. Postsolve therefore returns duals y and reduced costs z for the original model as well as x . `nirnay.solve` applies presolve by default to LPs and MILPs solved by the simplex, the interior-point method and branch-and-bound. It is not applied to QPs or to PDLP, which works on the original rows.

4.1 Reductions

Presolve runs over columns and rows repeatedly until a pass changes nothing (at most 20 passes). Every reduction pushes a record onto a stack; postsolve pops the stack. With $\bar{c}$ the current (possibly modified) cost vector and $[\bar{r}^l, \bar{r}^u]$ the current row bounds:

Fixed column

($\ell_j = u_j$). Substitute $x_j = \ell_j$: $\bar{r} \leftarrow \bar{r} - a_j \ell_j$, $c_0 \leftarrow c_0 + \bar{c}_j \ell_j$. Postsolve: $x_j = \ell_j$; z_j follows from the final y .

Empty column

(no nonzero in any live row). Set x_j to the bound its cost prefers: ℓ_j if $\bar{c}_j > 0$, u_j if $\bar{c}_j < 0$. If that bound is infinite the problem is unbounded. Postsolve: $z_j = \bar{c}_j$.

Empty row

Drop it, or declare infeasibility if $0 \notin [\bar{r}_i^l, \bar{r}_i^u]$. Postsolve: $y_i = 0$.

Redundant row

If the activity bounds $L_i = \sum_j \min(a_{ij}\ell_j, a_{ij}u_j)$ and $U_i = \sum_j \max(a_{ij}\ell_j, a_{ij}u_j)$ lie inside $[\bar{r}_i^l, \bar{r}_i^u]$, the row can never bind: drop it, $y_i = 0$. If $[L_i, U_i]$ misses the row interval the problem is infeasible.

Singleton row

$(a_{ij}x_j \in [\bar{r}_i^l, \bar{r}_i^u]$ with one live entry). The row becomes a bound on x_j :

$$\ell_j \leftarrow \max(\ell_j, \lambda), \quad u_j \leftarrow \min(u_j, \mu), \quad [\lambda, \mu] = \begin{cases} [\bar{r}_i^l/a_{ij}, \bar{r}_i^u/a_{ij}] & a_{ij} > 0, \\ [\bar{r}_i^u/a_{ij}, \bar{r}_i^l/a_{ij}] & a_{ij} < 0, \end{cases} \quad (8)$$

rounded inward for an integer column. The record keeps which of the two bounds the row supplied, and (see Section 4.3) the column's cost $\bar{c}_j$ and its live rows at this moment.

Free column singleton

on an equality row. If x_j appears only in row i , the row is $\sum_k a_{ik}x_k = b_i$, and the bounds of x_j cannot bind (it is free, or the row and the other columns' bounds imply its bounds), then x_j can be eliminated with the row:

$$x_j = \frac{b_i - \sum_{k \neq j} a_{ik}x_k}{a_{ij}}, \quad y_i = \frac{\bar{c}_j}{a_{ij}}, \quad \bar{c}_k \leftarrow \bar{c}_k - a_{ik} y_i \ (k \neq j), \quad c_0 \leftarrow c_0 + b_i y_i. \quad (9)$$

The dual is known at removal time, because x_j is basic (it is not at a bound), so its reduced cost $\bar{c}_j - a_{ij}y_i$ must be zero.

Integer bounds

(MILP only). Round integer bounds inward and tighten them by activity-based propagation over the live rows (Section 9.2); see Achterberg et al. [3] for the MIP presolve reductions still missing. The LP dual is not needed for a MILP, so these reductions need no dual postsolve.

4.2 Postsolve and dual recovery

Postsolve starts from the reduced solution: x on kept columns, x_j of removed columns as recorded, y on kept rows and $y_i = 0$ on removed rows. It maintains $z = c - A^T y$ over the *original* model and undoes the stack in reverse order:

- a free column singleton restores x_j and sets $y_i = \bar{c}_j/a_{ij}$ from (9);
- a singleton row computes the reduced cost of x_j *in the problem as it was when the row was removed*,

$$\bar{z}_j = \bar{c}_j^{\text{rec}} - \sum_{k \in \mathcal{R}_j^{\text{rec}}} a_{kj} y_k, \quad (10)$$

where $\bar{c}_j^{\text{rec}}$ and $\mathcal{R}_j^{\text{rec}}$ (the rows in which x_j was still live) were recorded at removal. If the bound this row created is active and $\bar{z}_j$ has the sign that bound can absorb, the reduced cost moves to the row:

$$(x_j = \lambda \text{ and } \bar{z}_j > 0) \text{ or } (x_j = \mu \text{ and } \bar{z}_j < 0) \implies y_i = \frac{\bar{z}_j}{a_{ij}}, \quad z_j \leftarrow z_j - a_{ij}y_i. \quad (11)$$

Otherwise $y_i = 0$ and the bound's multiplier stays in z_j .

- empty, redundant and fixed items need nothing: $y_i = 0$, and z_j is already correct.

Each assignment of y_i subtracts $a_{ij}y_i$ from the running z , so at the end $z = c - A^T y$ holds exactly and (2) can be checked on the original model.

4.3 An order-dependence bug in singleton-row duals

Bug: singleton-row duals depended on the order of undoing

Symptom. With presolve switched on, some postsolved solutions had an optimal x but duals that violated the sign conditions (2) of the original model. The violations sat on columns that had lost singleton rows, and they depended on the order in which the reductions were undone.

Cause. The first version decided (11) from the running reduced cost $z_j = c_j - \sum_k a_{kj}y_k$ at the moment the record was undone. That value is wrong in two ways:

1. it uses the original c_j , while the reduced problem that removed the row had cost $\bar{c}_j$, already modified by free-column-singleton substitutions (9) made *earlier*;
2. it sums over all rows of x_j , including rows removed *before* this one. Their duals are assigned *later* in postsolve (the stack is undone in reverse), so they are still zero. Another singleton row on the same column is the typical case.

Both errors depend on the order of reductions, so the answer depended on that order too.

Fix. Record $\bar{c}_j$ and the live rows of column j (indices and coefficients) when the row is removed, and evaluate (10) from the record. Every row in $\mathcal{R}_j^{\text{rec}}$ is either kept (its dual comes from the engine) or removed *after* this row, and so undone *before* it. Its dual is therefore final when (10) is evaluated. By induction over the stack, each undo step turns an optimal primal–dual pair of the smaller problem into one of the larger problem.

The simplex was changed at the same time to recompute $y = B^{-T}c_B$ exactly for the true costs when it returns an optimal basis (`lp/simplex.py, _result`). Its iteration loop updates reduced costs incrementally and refreshes y only at refactorisation, and the dual loop may shift costs (Section 11.1), so the y held at the end is not exactly the dual of the final basis for the original costs. Postsolve needs exact duals.

Check. `tools/dual_check.py` solves every Netlib LP with presolve and the dual simplex (60 s limit) and tests primal feasibility and the sign conditions (2) on the original model, relative to $1 + \|c\|_\infty$. On the run used for this report, 88 of 90 problems passed with 0 failures. The largest relative reduced-cost violation was $2.0 \cdot 10^{-12}$ and the largest row-dual sign violation $2.5 \cdot 10^{-13}$. The remaining 2 problems, `dfl001` (`time_limit`), `pilot87` (`time_limit`), hit the 60 s limit of the check and were not tested.

4.4 What presolve removes

Running presolve alone on the 90 Netlib problems removes 10.5% of all rows, 4.8% of all columns and 7.6% of all nonzeros (median 7.7% of rows per problem, up to 60% on `ship12s`). 16 problems are not reduced at all. Table 2 counts the reductions by rule. The whole census takes 43.9 s, with `dfl001` the slowest at 3.65 s, because presolve is written in plain Python loops.

Table 2: Presolve reductions on the Netlib LPs, by rule: how many rows or columns each rule removed in total, and on how many problems it fired at least once.

Rule	Removed	Problems
singleton row $\rightarrow$ bound	3875	67
fixed column	5021	61
empty row	736	35
empty column	1082	9
redundant row	45	12
free column singleton	2292	45

These are only the simplest reductions. The comparator study (Section 15) shows that doubleton equations, forcing rows, dominated columns and parallel rows account for most of what HiGHS's presolve removes. Those are next on the roadmap (Section 16).

Effect on the dual simplex. The simplex sweep with presolve is `netlib_simplex_v3.csv`. It solves 90 of its 90 problems (failures: none). On the 90 problems solved both with and without presolve, presolve reduces the dual iterations by a factor of 1.12 (geometric mean of the ratios). Timed against HiGHS inside each sweep, which cancels differences in machine load between the two runs, the shifted-mean ratio is 3.7 with presolve and 3.7 without, and the geometric mean of per-instance ratios is 8.3 with presolve and 6.3 without. (The raw times are not comparable: HiGHS itself ran 4.2 times slower, in geometric mean, during the sweep with presolve, because other sweeps shared the machine.) Presolve therefore roughly breaks even in shifted geometric mean, which weights the longer solves, and it costs time on the many small problems, where the per-instance ratio rises. Its passes are pure Python loops, which on small problems cost about as much as the iterations they save, or more. Compiling presolve is part of the first roadmap item, and the payoff should come from the stronger reductions still to be written.

5 Sparse linear algebra

Every engine except PDLP spends most of its time solving linear systems: the simplex with the basis matrix B , the LP interior-point method with the normal matrix $M = B\Theta B^T$, and the QP method with the augmented matrix K . NIRNAY implements the three factorisations it needs and the fill-reducing ordering they share. All numeric kernels are Numba-compiled loops over plain CSC arrays.

5.1 Sparse LU of the simplex basis

`linalg/lu.py` factorises $BQ = \tilde{L}U$, where Q is a column order, U is upper triangular in pivot order, and $\tilde{L}$ is unit lower triangular up to a row permutation (column k of $\tilde{L}$ has its unit entry at the pivot row p_k).

Column order. Logical (slack) columns come first. They are signed unit vectors and factor at no cost, and a simplex basis consists mostly of them. Structural columns follow in order of increasing length, a cheap stand-in for a fill-reducing order that works well on such mostly-logical bases.

Left-looking elimination. Column k is computed by the sparse triangular solve of Gilbert and Peierls [26]:

$$\tilde{L}_{:,1:k-1} x = B_{:,q_k}, \quad (12)$$

where only columns of $\tilde{L}$ already computed take part. The rows that become nonzero in x are exactly the nodes reachable from the nonzeros of $B_{:,q_k}$ in the graph of $\tilde{L}$. An iterative depth-first search finds them in topological order, so the numeric solve touches only those rows. The work is proportional to the floating-point operations, not to m . Entries of x in already-pivoted rows form column k of U ; the others are the candidates for the pivot.

Threshold pivoting. Among unpivoted rows r with $|x_r| \geq \tau \max_{r'} |x_{r'}|$ ($\tau = 0.1$), the pivot is the row with the fewest nonzeros in B , with ties broken by magnitude. This is Markowitz's idea [38] applied row-wise: it gives up a little stability for much less fill.

Algorithm 1: Left-looking sparse LU with threshold pivoting and repair (`linalg/lu.py`)

```

input : basis  $B$  (CSC), column order  $q$ , row counts  $\rho_r$ , threshold  $\tau$ , tiny  $\epsilon$ 
output :  $\tilde{L}$ ,  $U$ , pivot rows  $p$ , list of unpivoted steps
1 for  $k = 1, \dots, m$  do
2    $\mathcal{X} \leftarrow$  reach of  $\text{struct}(B_{:,q_k})$  in the graph of  $\tilde{L}$ , in topological order (DFS);
3   scatter  $x \leftarrow B_{:,q_k}$ ; foreach  $r \in \mathcal{X}$  with  $r$  pivoted at step  $s$  do
4      $x_{\mathcal{L}_s} \leftarrow x_{\mathcal{L}_s} - \tilde{L}_{\mathcal{L}_s,s} x_r$ 
5    $U_{:,k} \leftarrow \{x_r : r \text{ pivoted}\}; \quad a_{\max} \leftarrow \max\{|x_r| : r \text{ unpivoted}\};$ 
6   if  $a_{\max} > \epsilon$  then
7      $p_k \leftarrow \arg \min\{\rho_r : r \text{ unpivoted}, |x_r| \geq \tau a_{\max}\}; \quad \triangleright \text{shortest row}$ 
8      $U_{kk} \leftarrow x_{p_k}; \quad \tilde{L}_{:,k} \leftarrow x_{\text{unpivoted}}/x_{p_k};$ 
9   else
10    record step  $k$  as singular;  $\triangleright$  caller swaps in a logical
11  clear  $x$  on  $\mathcal{X}$ ;

```

Basis repair. If no candidate exceeds 10^{-11} , column q_k depends on the columns before it. Rather than reject the basis, the factorisation leaves step k unpivoted and reports a pair (basis position, unpivoted row). The simplex then swaps that basic variable for the row's logical, which is always independent of the rest, and refactorises. Near-singular bases arise from rounding and from degenerate pivots on models such as the `pilot` family, and industrial simplex codes keep going the same way.

Updates. After a pivot that replaces basis position r by a column with $\alpha_q = B^{-1}a_q$, the new inverse is $B_{\text{new}}^{-1} = E^{-1}B^{-1}$ with the elementary eta matrix

$$E = I + (\alpha_q - e_r)e_r^\top, \quad E^{-1}v = v - \frac{v_r}{\alpha_{q,r}}(\alpha_q - e_r), \quad (13)$$

the product form of the inverse [15]. The eta file stores α_q sparsely, dropping entries below 10^{-14} . FTRAN ($Bx = b$) applies the LU solve and then the etas in order; BTRAN ($B^\top y = d$) applies the etas in reverse and then the transposed LU solve. The basis is refactorised after 100 updates, or earlier when the eta file holds more than $8(\text{nnz}(\tilde{L}) + \text{nnz}(U) + m)$ entries, to bound both the work per solve and the growth of rounding error.

5.2 Sparse Cholesky for the normal equations

The LP interior-point method solves $M \Delta y = r$ with $M = B\Theta B^\top + \delta I$ symmetric positive definite and the same sparsity pattern at every iteration. `linalg/cholesky.py` splits the work in the classic way [17]:

1. **Analyse** once. Compute a fill-reducing permutation P (Section 5.3), the upper triangle of $PMP^\top$ with a map back to M 's values, the elimination tree ($\text{parent}(i) = \min\{k > i : l_{ki} \neq 0\}$, computed with path compression), and the column counts of L from the row subtrees. L 's pattern is then allocated exactly.
2. **Factor** every iteration, by the up-looking algorithm. Row k of L solves

$$L_{1:k-1,1:k-1} l_k = m_{1:k-1,k}, \quad L_{kk} = \sqrt{m_{kk} - l_k^\top l_k}, \quad (14)$$

and the nonzeros of l_k are the nodes on the paths from the nonzeros of $m_{1:k-1,k}$ up the elimination tree to k (the row subtree).

3. **Solve** $LL^\top \hat{x} = Pb$ by forward and back substitution.

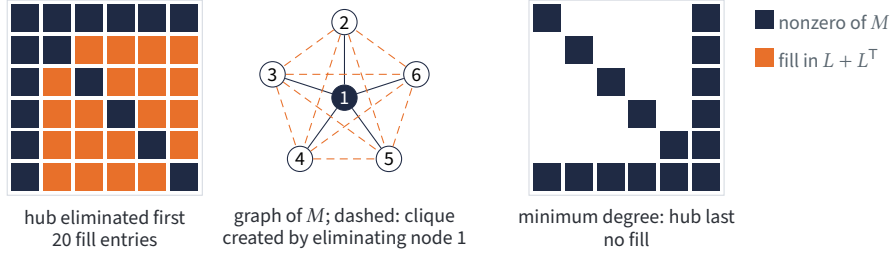


Figure 3: Fill-in and ordering on an arrowhead matrix. Eliminating the hub first (left) makes its five neighbours a clique (middle), and L becomes dense. Minimum degree eliminates the leaves first and the hub last (right), and L has no fill at all. The same effect, on a large scale, decides whether the normal matrix of an LP can be factorised at all.

Pivot guard. As the interior-point iterates converge, M becomes nearly singular: rows may be dependent, and Θ spans many orders of magnitude. If the pivot $m_{kk} - l_k^T l_k \leq 10^{-30}$, it is replaced by 10^{128} . The corresponding component of the solution then becomes (almost) zero instead of overflowing, which is the standard remedy [50, §11.1]. The number of replaced pivots is reported with each factorisation.

5.3 Minimum-degree ordering and fill-in

Eliminating a node of the graph of a symmetric matrix joins all its neighbours into a clique, and the new edges are exactly the fill-in of L . `linalg/ordering.py` implements the minimum-degree heuristic [24, 48]: it repeatedly eliminates a node of smallest current degree, on an explicit elimination graph with degree buckets. Two refinements matter on optimisation matrices:

- **mass elimination:** neighbours whose closed neighbourhood equals the pivot's would each be chosen next anyway, so they are eliminated together. This removes most of the cost on block-structured matrices;
- **dense-row deferral:** nodes with degree above $\max(16, 10\sqrt{n})$ are taken out of the graph and ordered last. Normal matrices of LPs often have a few almost-dense rows, and ordering them last both bounds the work of the ordering and keeps the dense part of L in the final block.

Figure 3 shows why the order matters on the smallest example, an arrowhead matrix.

5.4 Assembling the normal matrix

$M = B\Theta B^T + \text{diag}(\delta)$ changes its values at every interior-point iteration but not its pattern. `linalg/normal.py` therefore computes once the full pattern of BB^T (with sorted row indices) and a *slot map*. For each row k of B , each column j in that row, and each entry i of column j , the map gives the position of M_{ik} in the value array. Each iteration then assembles M in one pass,

$$M_{\text{slot}(q)} += \theta_j b_{kj} b_{ij}, \quad q = 1, 2, \dots, \sum_j \text{nnz}(b_j)^2, \quad (15)$$

with no searching and no allocation, followed by the diagonal term δ_k .

5.5 Quasi-definite LDL^T for the QP augmented system

The QP method solves systems with the regularised augmented matrix

$$K = \begin{bmatrix} -H & B^T \\ B & R \end{bmatrix}, \quad H = Q + D + \rho I \succ 0, \quad R = \delta I \succ 0, \quad (16)$$

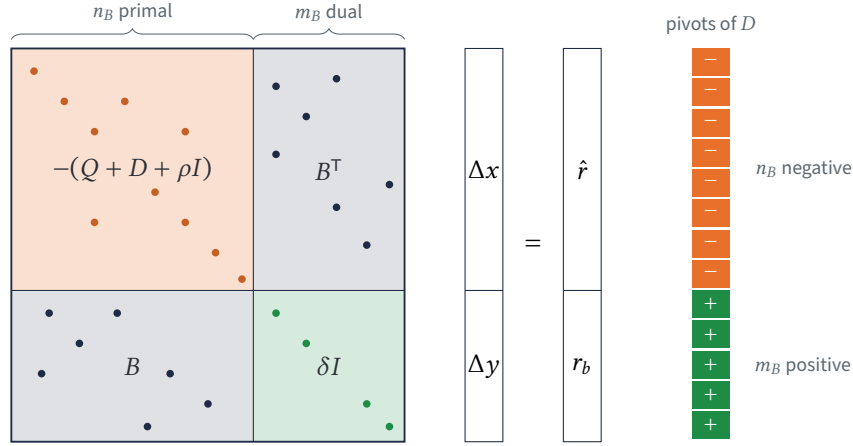


Figure 4: Block structure of the regularised augmented (KKT) system solved at every QP interior-point iteration. The (1, 1) block is negative definite, because it contains the barrier term $D = ZX^{-1} + SW^{-1}$ and the regularisation ρI , and the (2, 2) block is positive definite. Any symmetric permutation therefore factorises as LDL^T with the pivot signs shown on the right, and the ordering can be chosen for sparsity alone.

which is symmetric *quasi-definite*. Vanderbei [49] showed that such a matrix has a factorisation $PKP^T = LDL^T$ with D diagonal, n negative and m positive pivots, for *every* symmetric permutation P . The ordering can therefore be chosen for sparsity alone, exactly as for Cholesky, and no pivoting for stability is needed during the numeric factorisation. `linalg/ldl.py` reuses the Cholesky analysis (minimum degree, elimination tree, column counts) and the up-looking LDL^T kernel of Davis [16]. The numeric step computes, for row k ,

$$L_{kj} = \frac{y_j}{D_{jj}}, \quad D_{kk} = k_{kk} - \sum_{j < k} L_{kj} y_j, \quad (17)$$

where y solves the sparse triangular system for row k .

Pivot floor. The signs of the pivots are known in advance: $\sigma_k = -1$ for a primal row of K and $+1$ for a dual row. A pivot with the wrong sign, or smaller than $\delta_{\min}$ in magnitude, is replaced by $\sigma_k \delta_{\min}$ (dynamic regularisation, with $\delta_{\min} = 10^{-11}$ in the QP method). The factorisation then always exists and has the correct inertia. The perturbation is removed by iterative refinement against the unregularised matrix (Section 10). Figure 4 shows the block structure.

6 The bounded dual simplex method

The dual simplex is NIRNAY’s default LP engine and the node solver of branch-and-bound. Its design follows Koberstein’s thesis [32]: dual steepest-edge pricing [20], the bound-flipping ratio test [39] with Harris’s two-pass tolerance [30], Fourer’s auxiliary problem for phase 1 [21], cost perturbation and shifting against degeneracy, and a primal simplex to clean up at the end (`lp/simplex.py`, with the per-iteration loops in `lp/_simplex_kernels.py`).

6.1 Basis, primal and dual values

In the computational form (4) the constraint matrix is $\bar{A} = [A \ -I]$ over $n + m$ variables. A basis is an index set B of size m with $B = \bar{A}_{:,B}$ nonsingular; every nonbasic variable sits at a bound (or at zero if free). Then

$$x_B = -B^{-1} \bar{A}_{:,N} x_N, \quad y = B^{-T} c_B, \quad d = c - \bar{A}^T y \quad (d_B = 0). \quad (18)$$

Algorithm 2: Bound-flipping ratio test with Harris's two passes (`dual_ratio_test`)

input : pivot row α_r , reduced costs d , statuses, bounds, direction s , infeasibility δ , tolerances ϵ_d, ϵ_p
output : entering column q (or none), list F of bound flips

- 1 $\epsilon \leftarrow \max(\epsilon_p, 10^{-9} \max_j |\alpha_{rj}|)$; $\triangleright$ relative pivot tolerance
- 2 $C \leftarrow$ candidates with $s\alpha_{rj}$ of the right sign and $|\alpha_{rj}| > \epsilon$;
- 3 **if** $C = \emptyset$ **then return** none (the dual is unbounded along this row);
- 4 sort C by $t_j = |d_j|/|\alpha_{rj}|$; slope $\leftarrow \delta$; $F \leftarrow \emptyset$;
- 5 **for** $j \in C$ in order, except the last **do**
- 6 **if** j is free or $u_j - \ell_j = \infty$ or slope $- |\alpha_{rj}|(u_j - \ell_j) \leq 0$ **then break**;
- 7 $F \leftarrow F \cup \{j\}$; slope $\leftarrow$ slope $- |\alpha_{rj}|(u_j - \ell_j)$;
- 8 $\mathcal{R} \leftarrow$ the candidates not in F ;
- 9 $\Theta \leftarrow \min_{j \in \mathcal{R}} (|d_j| + \epsilon_d)/|\alpha_{rj}|$; $\triangleright$ Harris pass 1
- 10 $q \leftarrow \arg \max\{|\alpha_{rj}| : j \in \mathcal{R}, t_j \leq \Theta\}$; $\triangleright$ Harris pass 2
- 11 **return** q, F ;

The basis is *dual feasible* when every nonbasic reduced cost has the sign its bound allows ($d_j \geq 0$ at a lower bound, $d_j \leq 0$ at an upper bound, $d_j = 0$ if free), and *primal feasible* when $\ell_B \leq x_B \leq u_B$. The dual simplex keeps dual feasibility and removes primal infeasibility one row at a time. Each iteration does not decrease the dual objective (3).

6.2 One iteration

Pricing: dual steepest edge. Let δ_r be the primal infeasibility of basic variable r ($\ell_r - x_r$ below, $x_r - u_r$ above; only violations beyond the primal tolerance 10^{-7} count). The leaving row maximises

$$r = \arg \max_i \frac{\delta_i^2}{w_i}, \quad w_i \approx \|e_i^\top B^{-1}\|_2^2, \quad (19)$$

which measures the infeasibility per unit length of the dual edge [20]. The weights start at 1 for the slack basis.

Pivot row. One BTRAN gives $\rho_r = B^{-\top} e_r$. The pivot row $\alpha_r = \rho_r^\top \bar{A}$ is then formed row-wise: the kernel walks only the rows of A (columns of $A^\top$) where ρ_r is nonzero, so a sparse ρ_r costs little. Basic entries are set to zero.

Ratio test. If x_r leaves towards its lower bound, set $s = -1$; towards its upper bound, $s = +1$. A nonbasic j is a candidate when $s\alpha_{rj}$ has the sign that lets d_j move towards zero: $s\alpha_{rj} > \epsilon$ at a lower bound, $s\alpha_{rj} < -\epsilon$ at an upper bound, $|\alpha_{rj}| > \epsilon$ if free. The pivot tolerance is *relative*, $\epsilon = \max(10^{-7}, 10^{-9} \max_j |\alpha_{rj}|)$, so a badly scaled row cannot supply a pivot that is tiny next to its neighbours. Each candidate has a breakpoint $t_j = |d_j|/|\alpha_{rj}|$ (negative d_j of the wrong sign is clipped to zero) and a relaxed breakpoint $\tilde{t}_j = (|d_j| + \epsilon_d)/|\alpha_{rj}|$ with dual tolerance $\epsilon_d = 10^{-7}$. Algorithm 2 passes the breakpoints in increasing order, as Maros's bound-flipping ratio test does [39]. The slope of the dual objective starts at δ_r and falls by $|\alpha_{rj}|(u_j - \ell_j)$ each time a boxed candidate is passed and flipped to its other bound. The pass stops before the slope would turn negative. Among the remaining candidates, Harris's two passes [30] choose the entering q : pass 1 takes the bound $\Theta = \min \tilde{t}_j$, and pass 2 takes, among the candidates with $t_j \leq \Theta$, the one with the largest $|\alpha_{rj}|$. Stepping to any such candidate leaves every reduced cost within ϵ_d of feasibility, and the large pivot keeps the basis well conditioned.

Updates. FTRAN gives the entering column $\alpha_q = B^{-1} \bar{a}_q$. Its entry $\alpha_{q,r}$ must agree with $\alpha_{r,q}$ from the row: if $|\alpha_{q,r} - \alpha_{r,q}| > 10^{-6}(1 + |\alpha_{q,r}|)$ or $|\alpha_{q,r}| < 10^{-9}$, the factorisation has lost accuracy. The basis is

then refactorised and the iteration repeated; if the factorisation was already fresh, row r is skipped for now. Otherwise the flips move the basic variables once, $x_B \leftarrow x_B - B^{-1} \sum_{j \in F} \bar{a}_j \Delta x_j$, and

$$\begin{aligned} \theta_d &= d_q / \alpha_{rq}, & d_j &\leftarrow d_j - \theta_d \alpha_{rj} \ (j \in \mathcal{N}), & d_{\text{leave}} &= -\theta_d, \\ \theta_p &= (x_{B_r} - \text{target}) / \alpha_{q,r}, & x_B &\leftarrow x_B - \theta_p \alpha_q, & x_q &\leftarrow x_q + \theta_p, \end{aligned} \quad (20)$$

where the leaving variable goes to the bound it violated. The dual steepest-edge weights are updated exactly as in Forrest and Goldfarb [20], with one more FTRAN $\tau = B^{-1} \rho_r$ and $\beta_r = \|\rho_r\|^2$:

$$w_i \leftarrow \max\left(w_i + \left(\frac{\alpha_{iq}}{\alpha_{rq}}\right)^2 \beta_r - 2 \frac{\alpha_{iq}}{\alpha_{rq}} \tau_i, 10^{-4} \left(1 + \left(\frac{\alpha_{iq}}{\alpha_{rq}}\right)^2\right)\right) \ (i \neq r), \quad w_r \leftarrow \max\left(\frac{\beta_r}{\alpha_{rq}^2}, 10^{-8}\right). \quad (21)$$

The lower bounds on the weights keep rounding from driving a weight to zero or below. Finally q replaces the leaving variable in the basis, and an eta vector is appended to the factorisation (Section 5.1). Figure 5 shows the whole loop.

6.3 Phase 1, perturbation and cleanup

Dual phase 1 by auxiliary boxes. The all-logical start basis is rarely dual feasible. Following Fourer [21] (Koberstein [32, §4.1.2]), phase 1 solves the same problem with every bound replaced by a box:

$$\text{free} \rightarrow [-1, 1], \quad \text{lower only} \rightarrow [0, 1], \quad \text{upper only} \rightarrow [-1, 0], \quad \text{boxed or fixed} \rightarrow [0, 0]. \quad (22)$$

Every variable of the auxiliary problem is boxed, so any basis becomes dual feasible once each nonbasic variable is placed at the bound its reduced cost prefers. The dual simplex therefore starts at once. An optimal basis of the auxiliary problem is dual feasible for the real problem whenever the real problem is dual feasible at all. The code does not test the auxiliary objective. Phase 2 starts from the phase-1 basis, and any reduced cost left with the wrong sign is repaired by the flips and cost shifts described below. If the real problem is dual infeasible (unbounded), the shifted costs are removed at the end and the primal cleanup finds the unbounded ray.

Cost perturbation. Degenerate LPs make many ratio-test breakpoints tie at zero, and the dual simplex can then cycle or stall. Both phases therefore run on perturbed costs [32, §6.2.2.3]:

$$\tilde{c}_j = c_j + \sigma_j 5 \cdot 10^{-7} (1 + |c_j|)(1 + \xi_j), \quad \xi_j \sim U[0, 1], \quad (23)$$

with $\sigma_j = -1$ for a variable at its upper bound and $+1$ otherwise. The direction keeps the start dual feasible, and the random magnitude breaks the ties. Basic, fixed and free nonbasic variables are not perturbed. The random factors ξ come from a generator with a fixed seed and are drawn once per instance, so runs are reproducible and the warm-started re-solves of branch-and-bound all see the same perturbation.

Cost shifting. If the entering candidate's reduced cost has the wrong sign (by less than the tolerance, which the relaxed test allows), its cost is shifted so that $d_q = 0$ before the step. The dual objective then never moves backwards [32, §6.2.2.2]. After each refactorisation the same fix is applied to all nonbasics: a boxed variable with a wrong-sign reduced cost is flipped to its other bound, and any other variable has its cost shifted.

Cleanup by primal simplex. When phase 2 is optimal, the perturbation and shifts are removed: the true costs are restored and y, d recomputed. Any dual infeasibility that remains is removed by a primal simplex on the same factorisation. It prices by the largest dual infeasibility, uses a Harris two-pass ratio test over the basic variables, and flips the entering variable instead of pivoting when it reaches its own other bound first. The cleanup runs with the dual tolerance tightened to 10^{-9} (Section 11.1). If it loses primal feasibility by more than $10\epsilon_p$, the dual simplex runs once more. On return the duals are recomputed exactly as $y = B^{-T} c_B$ for the true costs.

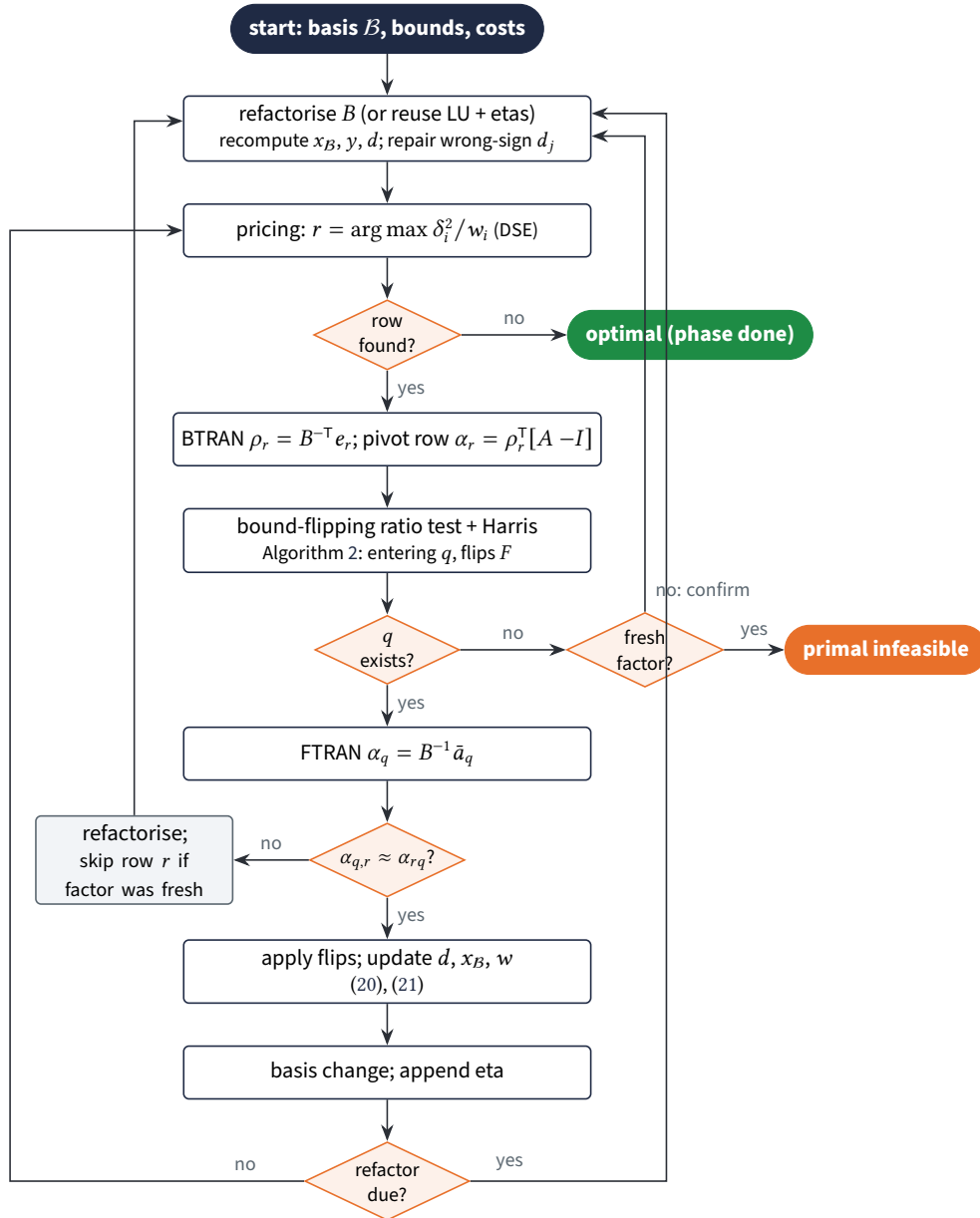


Figure 5: The dual simplex iteration loop of `lp/simplex.py`. Diamonds are tests. Every path that could declare a result on a possibly inaccurate factorisation (a missing ratio-test candidate, a pivot that the row and the column see differently) first refactorises and repeats the iteration.

6.4 Warm starts for branch-and-bound

A `SimplexLP` object keeps its factorisation, basis, weights and phase-1 status between calls. `set_structural_bounds` changes only the variables whose bounds differ, and re-replaces nonbasic ones at their new bounds. `get_basis` and `load_basis` save and restore a basis as one status byte per variable. If the loaded basis has the same basic set as the current one, the LU and eta file remain valid and only x_B , y and d are recomputed; otherwise the basis is refactorised. After a branching change the parent's optimal basis stays dual feasible, so a child node is typically re-optimised in a few dual pivots.

7 The interior-point method for LP

`lp/ipm.py` implements Mehrotra's predictor-corrector method [41] on the bounded standard form (5), with the normal equations solved by the sparse Cholesky of Section 5.2. The practical details (free variables, starting point, the normal equations and their numerical difficulties) follow Wright [50, ch. 10–11]. The upper-bound handling follows Lustig, Marsten and Shanno [37].

7.1 Newton system

Write $\mathcal{L}$ for the variables with a lower bound (after shifting, $x_j \geq 0$) and $\mathcal{U}$ for the boxed ones, and let X, Z, W, S be the diagonal matrices of x, z, w, s . The residuals of (6) are

$$r_b = b - Bx, \quad r_u = U - x - w, \quad r_c = c - B^T y - z + s, \quad \mu = \frac{x_{\mathcal{L}}^T z_{\mathcal{L}} + w^T s}{|\mathcal{L}| + |\mathcal{U}|}. \quad (24)$$

A Newton step for the perturbed complementarity conditions $XZe = \sigma\mu e$, $WSe = \sigma\mu e$ solves

$$\begin{aligned} B\Delta x &= r_b, & \Delta x + \Delta w &= r_u, & B^T \Delta y + \Delta z - \Delta s &= r_c, \\ Z\Delta x + X\Delta z &= r_{xz}, & S\Delta w + W\Delta s &= r_{ws}. \end{aligned} \quad (25)$$

Eliminating $\Delta z, \Delta w, \Delta s$ gives $-\Theta^{-1}\Delta x + B^T \Delta y = \hat{r}$ with

$$\Theta^{-1} = X^{-1}Z + W^{-1}S + \Gamma, \quad \hat{r} = r_c - X^{-1}r_{xz} + W^{-1}(r_{ws} - S r_u), \quad (26)$$

where Γ is a small primal regularisation: 10^{-8} on free variables, which have no barrier term and would otherwise make Θ infinite, and 10^{-10} elsewhere. Eliminating Δx leaves the normal equations

$$(B\Theta B^T + \delta I) \Delta y = r_b + B\Theta \hat{r}, \quad \delta = 10^{-10}, \quad (27)$$

and back-substitution recovers

$$\Delta x = \Theta(B^T \Delta y - \hat{r}), \quad \Delta z = X^{-1}(r_{xz} - Z\Delta x), \quad \Delta w = r_u - \Delta x, \quad \Delta s = W^{-1}(r_{ws} - S\Delta w). \quad (28)$$

The dual regularisation δ keeps M positive definite when rows of B are dependent. Each solve of (27) is followed by up to two steps of iterative refinement against the assembled matrix, $\Delta y \leftarrow \Delta y + M^{-1}(\text{rhs} - M\Delta y)$.

7.2 Predictor-corrector

Each iteration factorises M once and solves with it twice:

1. **Predictor** (affine scaling): $r_{xz} = -XZe$, $r_{ws} = -WSe$. The largest steps $\alpha_p, \alpha_d \leq 1$ that keep x, w and z, s nonnegative give the affine complementarity μ_{aff} , and the centring parameter is Mehrotra's $\sigma = \min(1, (\mu_{\text{aff}}/\mu)^3)$.

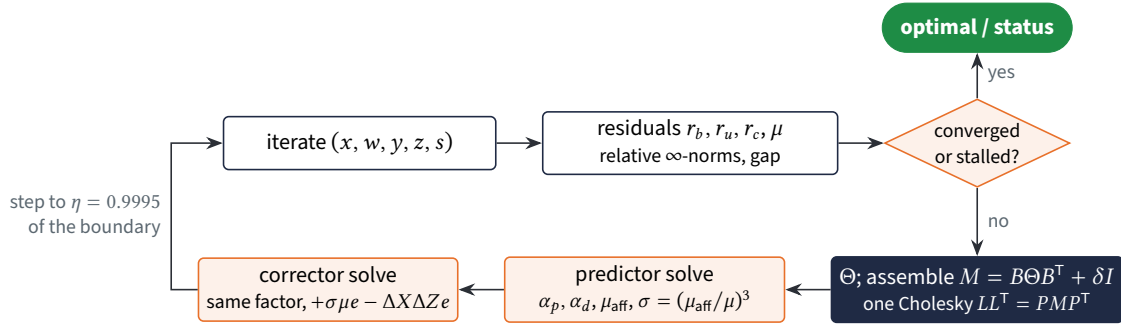


Figure 6: One iteration of the LP interior-point method: one factorisation of the normal matrix and two solves with it (predictor and corrector). The same loop, with the augmented system of Figure 4 in place of M , is the QP method of Section 10.

2. **Corrector:** $r_{xz} = \sigma \mu e - XZe - \Delta X_{\text{aff}} \Delta Z_{\text{aff}} e$, and likewise for (w, s) . This adds centring and the second-order term the predictor neglected.
3. **Step:** separate primal and dual step lengths, each the fraction $\eta = 0.9995$ of the distance to the boundary, capped at 1.

Starting point. Mehrotra’s heuristic, adapted to bounds: with $M_0 = BB^T + \delta I$ (factorised once), $x = B^T M_0^{-1} b$ and $y = M_0^{-1} Bc$. Then z and s are the positive and negative parts of $c - B^T y$, and $w = U - x$. Each group is shifted to positivity by 1.5 times its most negative entry, and all four are then shifted once more by $\frac{1}{2} x^T z / \sum z$ and $\frac{1}{2} x^T z / \sum x$, which balances the complementarity products.

7.3 Termination and stall detection

Convergence is measured by relative residuals in the ∞ -norm, the way HiGHS/IPX and most interior-point codes report them:

$$p = \frac{\|r_b\|_\infty}{1 + \|b\|_\infty} + \frac{\|r_u\|_\infty}{1 + \|U\|_\infty}, \quad d = \frac{\|r_c\|_\infty}{1 + \|c\|_\infty}, \quad g = \frac{|c^T x - (b^T y - U^T s)|}{1 + |c^T x|}, \quad (29)$$

and the method stops as optimal when $p, d, g < 10^{-8}$. Near the solution the normal equations become so ill-conditioned that progress can stop before that point. Continuing then only drives μ to zero and z/x to overflow. The method therefore tracks the best error $e^* = \min \max(p, d, g)$ so far and counts iterations that fail to halve it:

- after 4 such iterations, or once $\mu < 10^{-14}(1 + |c^T x|)$, the point is accepted as optimal if p, d, g are all below the loose tolerance 10^{-6} , the accuracy floor of the normal equations;
- after 30 such iterations it stops with `numerical_error`;
- a non-finite objective also stops with `numerical_error`.

Infeasibility is detected heuristically, from iterates that run off to infinity while the other side converges: after 20 iterations, $\mu < 10^{-12}$, $p > 10^{-6}$ and $\|y\|_\infty > 10^{12}$ mean primal infeasible, and $d > 10^{-6}$ with $\|x\|_\infty > 10^{14}$ means unbounded. A homogeneous self-dual formulation would give certificates instead, and is on the roadmap.

Algorithm 3: Mehrotra predictor–corrector for LP (`lp/ipm.py`)

input : bounded standard form (B, b, c, U) , tolerance $\epsilon = 10^{-8}$, loose tolerance 10^{-6}

- 1 analyse the pattern of BB^T once: ordering, elimination tree, slot map;
- 2 starting point (Mehrotra’s heuristic); $e^* \leftarrow \infty$, $k_{\text{stall}} \leftarrow 0$;
- 3 **for** $k = 1, \dots, 200$ **do**
- 4 compute r_b, r_u, r_c, μ and p, d, g (29);
- 5 **if** $\max(p, d, g) < \epsilon$ **then return** optimal;
- 6 **if** $\max(p, d, g) < e^*/2$ **then** $e^* \leftarrow \max(p, d, g)$, $k_{\text{stall}} \leftarrow 0$ **else** $k_{\text{stall}} \leftarrow k_{\text{stall}} + 1$;
- 7 **if** $k_{\text{stall}} \geq 4$ **or** μ tiny **then**
- 8 **if** $\max(p, d, g) < 10^{-6}$ **then return** optimal;
- 9 **if** $k_{\text{stall}} \geq 30$ **then return** numerical error;
- 10 infeasibility and unboundedness heuristics;
- 11 $\Theta \leftarrow (X^{-1}Z + W^{-1}S + \Gamma)^{-1}$; assemble M (15); factor $LL^T = PMP^T$;
- 12 predictor: solve (27) with $r_{xz} = -XZe$, $r_{ws} = -WSe$; get $\alpha_p, \alpha_d, \mu_{\text{aff}}$;
- 13 $\sigma \leftarrow \min(1, (\mu_{\text{aff}}/\mu)^3)$;
- 14 corrector: solve with $r_{xz} = \sigma\mu e - XZe - \Delta X_{\text{aff}}\Delta Z_{\text{aff}}e$ (and w, s);
- 15 step with $\alpha_p, \alpha_d \leftarrow \min(1, 0.9995 \alpha_{\text{max}})$;

8 PDLP: a first-order LP method on CPU and GPU

The simplex and interior-point methods need sparse factorisations, and those are hard to run efficiently on a GPU. PDLP [6] needs only products with A and A^T and vector operations, which GPUs do well. `lp/pdip.py` implements the restarted, preconditioned primal–dual hybrid gradient method (PDHG [13]) with the enhancements of PDLP and of its GPU descendants `cuPDLP.jl` [35] and `cuPDLP-C` [36]. The method is written once over an array module (NumPy or CuPy); the inner loop has a Numba backend for the CPU and a CUDA backend for the GPU.

8.1 The PDHG step

For the saddle-point form (7), one step with primal step $\tau = \eta/\omega$ and dual step $\sigma = \eta\omega$ (step size η , primal weight ω) is

$$x' = \text{proj}_X(x - \tau(c - A^T y)), \quad (30)$$

$$y' = \arg \max_{y \in Y} -y^T A(2x' - x) + p(y) - \frac{1}{2\sigma} \|y - y_k\|^2. \quad (31)$$

The dual step has a closed form. With $t = y - \sigma A(2x' - x)$, Moreau’s identity gives

$$y' = t + \sigma \text{proj}_{[r^l, r^u]}(-t/\sigma) = t - \text{clip}(t, -\sigma r^u, -\sigma r^l), \quad (32)$$

so an equality row gets $y - \sigma(Ax'_{\text{extr}} - b)$, a $\geq$ row gets $\max(0, t + \sigma r^l)$, and a free row gets 0. No slack columns are introduced: ranged rows, bounds and free rows are all handled by the two projections.

Adaptive steps. PDLP’s step-size rule (its Algorithm 3) accepts a step when $\eta \leq \bar{\eta}$, where

$$\bar{\eta} = \frac{\omega \|x' - x\|^2 + \|y' - y\|^2 / \omega}{2 |(x' - x)^T A^T (y' - y)|}, \quad \eta_{\text{next}} = \min\left((1 - (k+1)^{-0.3})\bar{\eta}, (1 + (k+1)^{-0.6})\eta\right). \quad (33)$$

A rejected step is not committed and is retried with η_{next} . The first step is $\eta_0 = 1/\|A\|_{\text{max}}$. Accepted iterates enter a running average weighted by their step size.

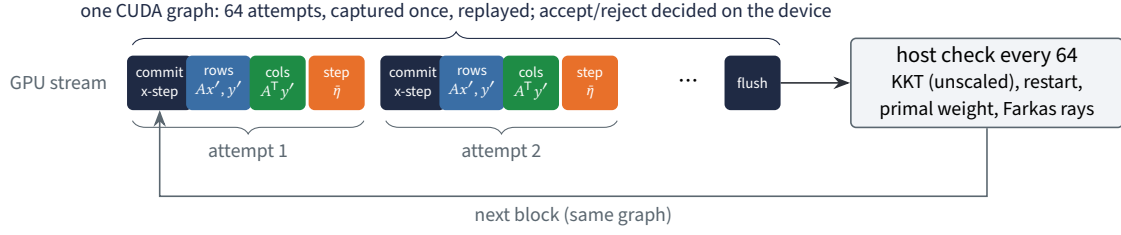


Figure 7: PDLP on the GPU. Each attempt is four fused kernels; the step-size decision never leaves the device. A block of 64 attempts is one replay of a captured CUDA graph, and the host is involved only at the check between blocks.

Restarts and the primal weight. Every 64 attempts the method evaluates the primal-weighted KKT error $\sqrt{\omega \|r_p\|^2 + \|r_d\|^2/\omega + \text{gap}^2}$ at the current iterate and at the average, and restarts from the better of the two, following cuPDLP’s criteria [35]. It restarts when that error has fallen below 0.2 times its value at the last restart; or below 0.8 times that value while rising since the previous check; or when the iterations since the last restart exceed 0.36 of all iterations. Restarts are what give PDHG linear convergence on LPs [7]. At a restart the primal weight is smoothed towards the ratio of dual to primal movement,

$$\omega \leftarrow \exp(\theta \log(\|\Delta y\|/\|\Delta x\|) + (1 - \theta) \log \omega), \quad \theta = 0.5. \quad (34)$$

Preconditioning. Ten Ruiz iterations [46] divide each row and column by the square root of its largest entry. One Pock–Chambolle pass [45] ($\alpha = 1$) then divides by the square roots of the row and column ℓ_1 norms. Finally the bounds and the objective are rescaled by $\beta_b = 1 + \|\text{bounds}\|_2$ and $\beta_c = 1 + \|c\|_2$, as in cuPDLP-C [36], so that ω starts near 1. The initial weight is $\omega_0 = \|c\|/\|b\|$ on the scaled problem.

Termination and infeasibility. The tests are PDLP’s relative ones, evaluated on the *unscaled* problem: $\|r_p\|_2 \leq \epsilon(1 + \|b\|_2)$, $\|r_d\|_2 \leq \epsilon(1 + \|c\|_2)$ and $|c^T x - \phi| \leq \epsilon(1 + |c^T x| + |\phi|)$, where r_d is the part of $c - A^T y$ that the variable bounds cannot absorb. The default is $\epsilon = 10^{-4}$. Every fourth check (256 attempts), the difference between the current iterate and the one at the previous check is projected onto the recession cones of the bounds and tested as a Farkas certificate [8]. A dual ray with positive objective and relative infeasibility below 10^{-8} proves primal infeasibility; a primal ray with $c^T \Delta x < 0$ proves unboundedness.

8.2 Two backends

CPU. One Numba call runs a whole block of 64 attempts, in three passes over the data per attempt: (1) the primal step (30); (2) a row pass that computes $(Ax')_i$ and immediately takes the dual step (32) for row i ; and (3) a column pass that computes $(A^T y')_j$ together with the two reductions the step rule (33) needs. The average is updated lazily inside the next attempt’s first two passes. The passes are thread-parallel when $\text{nnz}(A) \geq 30\,000$. Below that, on the development laptop, the fork/join cost of the thread pool outweighed the gain.

GPU. Each attempt is four CUDA kernels (Figure 7). A *commit* kernel adds the previous accepted step to the average and takes the primal step. A *rows* kernel computes Ax' with a group of G lanes per row, reducing with warp shuffles, and fuses the dual step. A *columns* kernel computes $A^T y'$ and fuses $\|\Delta x\|^2$ and $\Delta x^T A^T \Delta y$. A one-thread *step-size* kernel applies (33). G is the largest power of two not above the mean row (or column) length, capped at 32. The accept/reject decision stays on the device: a rejected step is simply not committed by the next commit kernel. A block of 64 attempts therefore needs no host round trip, and it is captured once as a CUDA graph and replayed. The host reads one small array of scalars per check.

8.3 Results

Netlib. PDLP at its default tolerance 10^{-4} reports optimal on 83 of the 90 Netlib problems (120 s limit). 4 hit the time limit (greenbea, greenbeb, pilot, pilot87) and 3 the iteration limit (bnl1, perold, pilot4). These are the badly scaled problems that every PDLP implementation in our comparator study also fails (Section 15). A relative KKT tolerance of 10^{-4} does not bound the objective error by 10^{-4} : 56 of the problems are within 10^{-4} of the HiGHS objective, 76 within 10^{-3} , and only 14 within the 10^{-6} used to score the other engines. The worst objective errors among the “optimal” runs are forplan ($5.1 \cdot 10^{-2}$), ganges ($1.8 \cdot 10^{-2}$), bnl2 ($5.6 \cdot 10^{-3}$), finnis ($3.9 \cdot 10^{-3}$), scrs8 ($3.3 \cdot 10^{-3}$), lotfi ($1.2 \cdot 10^{-3}$). The median run takes 7488 iterations (the longest 359168). The shifted geometric mean time is 0.75 s against 0.107 s for HiGHS. Netlib problems are small and hard, and PDLP is the wrong tool for them. It is there to show correctness, not speed.

Large LPs, CPU against GPU. The regime PDLP is built for is large, structured LPs. Table 3 and Figure 8 show the 10 large instances run so far (the sweep over 11 files is still running), from the Mittelmann benchmark sets [42], all at relative tolerance 10^{-4} , on an NVIDIA GeForce RTX 3050 Laptop GPU against the CPU backend on the laptop’s six cores (12 threads). On the 7 instances both backends finish, the GPU-over-CPU speed-up ranges from 0.3 to 15.8 (geometric mean 3.3; a value below 1 means the GPU was slower). On 2 more (rail01, rail02) only the GPU finishes within the 300 s limit, and 1 finished on neither. HiGHS in the same table runs its interior-point method without crossover at its default 10^{-8} tolerances, which gives the reference objective. That is a far more accurate answer, so the comparison with the PDLP times is not like for like. HiGHS’s interior point at the same 10^{-4} tolerance (last column) is the closer comparison. Where both finish, the ratio of the high-accuracy HiGHS time to the GPU time ranges from 1.0 to 264.

Table 3: PDLP on large LPs (LP relaxations for the MIPLIB files): wall time of NIRNAY PDLP on the CPU (Numba, 12 threads) and on the GPU (CuPy), the relative objective error of the GPU run against the HiGHS reference, and HiGHS’s interior point at its default tolerance (reference) and at the PDLP tolerance. †: stopped at the time limit; “>” is then a lower bound on the speed-up. ‡: HiGHS did not report Optimal.

Instance	m	n	nnz	NIRNAY PDLP			rel. err	HiGHS IPM (s)	
				CPU (s)	GPU (s)	$\times$		10^{-8}	10^{-4}
datt256	11077	262144	1503732	5.29	0.98	5.4	$2.8 \cdot 10^{-8}$	258.8	113.7
ex10	69608	17680	1162000	0.99	0.55	1.8	$7.2 \cdot 10^{-7}$	28.8	21.3
neos-5052403-cygnnet	38268	32868	4898304	67.5	15.3	4.4	$5.9 \cdot 10^{-6}$	49.4	108.5
qap15	6330	22275	94950	8.91	0.56	15.8	$2.3 \cdot 10^{-5}$	37.0	27.0
rail01	46843	117527	392086	300.1†	52.3	>5.7	$2.3 \cdot 10^{-4}$	85.8	67.1
rail02	95791	270869	756228	300.1†	103.6	>2.9	$1.5 \cdot 10^{-4}$	254.3	240.8
rmine15	358395	42438	879732	135.8	18.9	7.2	–	300.3‡	300.2
s100	14733	364417	1777917	300.6†	300.2†	–	$5.1 \cdot 10^{-5}$	300.4‡	295.5
s250r10	10962	273142	1318607	48.3	167.5	0.3	$6.6 \cdot 10^{-4}$	159.8	272.5
savsched1	295989	328575	1770507	8.59	3.15	2.7	$9.2 \cdot 10^{-5}$	78.0‡	54.4

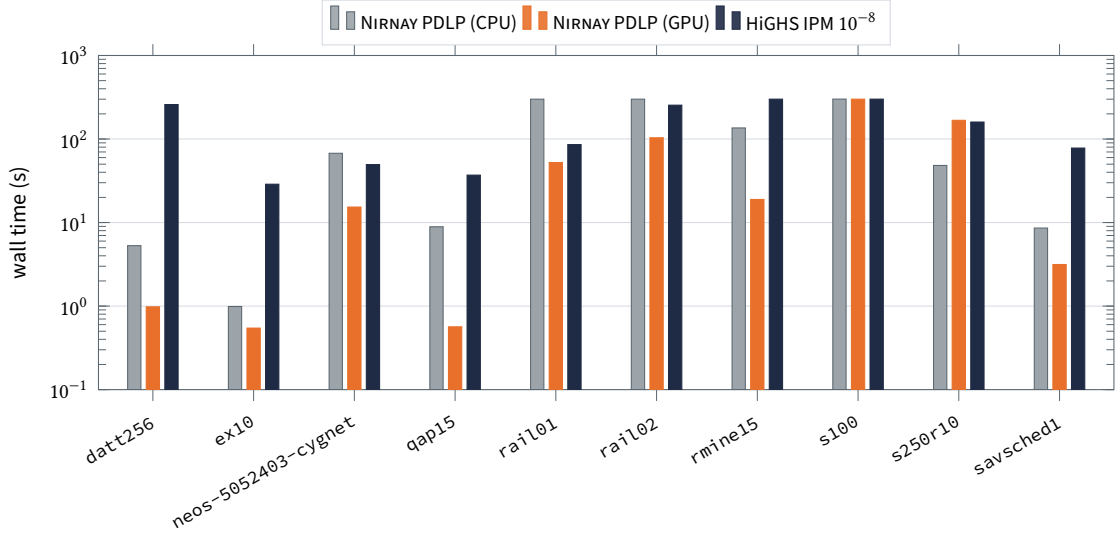


Figure 8: Wall time on the large LPs (log scale), from `results/large_pdlp_gpu.csv`. The PDLP runs stop at relative tolerance 10^{-4} ; the HiGS reference solves to 10^{-8} .

9 Branch-and-bound for mixed-integer programmes

Every node of the search is the LP relaxation (4) with some integer bounds tightened. A branching change leaves the parent’s optimal basis dual feasible, so the dual simplex re-optimises a child in a few pivots. This is why MILP codes are built on the dual simplex, and `mip/bnb.py` is built on the warm-started SimplexLP of Section 6. Its components follow Achterberg’s thesis [1] and the branching study of Achterberg, Koch and Martin [2].

9.1 The search

Node selection: best bound with plunging. Open nodes wait in a binary heap keyed by

$$(z_{\text{parent}}, -\text{depth}, \text{counter}), \quad (35)$$

so the node with the smallest bound comes first, and among equal bounds the *deepest* (Section 11.4). After a node is branched on, the child that the fractional part favours (x_j rounded up if $f_j \geq 0.5$, else down) is solved at once, and the other goes to the heap. The search keeps diving into the solved child while its bound stays within 10% of the distance between the best open bound and the cutoff:

$$z_{\text{child}} \leq z_{\text{open}} + 0.1 |z_{\text{cutoff}} - z_{\text{open}}|. \quad (36)$$

Before an incumbent exists the right-hand side is infinite, so the search dives depth-first, which finds incumbents early. Once one exists it returns to best-first search, which proves the bound. A child that fails (36) is queued with its own LP bound and basis.

Pruning and bound rounding. A node is pruned when its LP is infeasible or its rounded bound reaches the cutoff

$$z_{\text{cutoff}} = \begin{cases} z_{\text{inc}} - 1 + 10^{-6} & \text{if the objective is integral on integer points,} \\ z_{\text{inc}} - \max(10^{-9}, 10^{-9}|z_{\text{inc}}|) & \text{otherwise.} \end{cases} \quad (37)$$

The objective is integral when every continuous column has zero cost and all costs and c_0 are integers. Node bounds are then rounded up, $\lceil z - 10^{-6} \rceil$, which often closes the last unit of gap without enumeration. The search stops when the relative gap $(z_{\text{inc}} - z) / \max(1, |z_{\text{inc}}|)$ is at most 10^{-4} , the default of HiGS and SCIP.

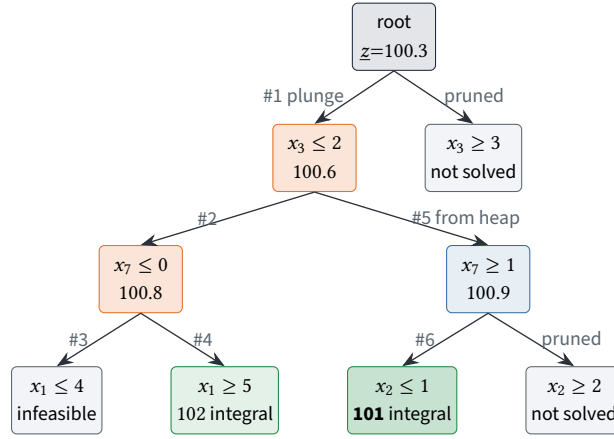


Figure 9: An illustrative branch-and-bound tree (made-up numbers, integral objective). Orange: the plunge, which dives into the favoured child after each branching and finds the incumbent 102 at node #4. The cutoff is then $102 - 1 + 10^{-6}$. The open nodes $x_7 \geq 1$ and $x_3 \geq 3$ both have the rounded key $\lceil 100.6 \rceil = \lceil 100.3 \rceil = 101$; the depth tie-break of (35) pops the deeper one (#5), whose child #6 is the optimum 101. The new cutoff $100 + 10^{-6}$ prunes the two remaining nodes without solving their LPs, and the root bound $\lceil 100.3 \rceil = 101$ closes the gap.

Branching: pseudocosts with reliability. The candidate score is the product rule

$$\text{score}_j = \max(\psi_j^- f_j, 10^{-6}) \cdot \max(\psi_j^+ (1 - f_j), 10^{-6}), \quad (38)$$

where f_j is the fractional part of x_j and $\psi_j^\pm$ are the pseudocosts: the average gain in the LP objective per unit of change, recorded from every solved child. Unobserved directions use the average over all columns. A candidate is *unreliable* while it has fewer than 4 observations in either direction. Up to 8 of the best-scored unreliable candidates are then strong-branched, both children solved by the dual simplex for at most 60 iterations from the current basis, which also records pseudocosts [2]. If both children are infeasible the node is pruned. If one is, the candidate gets a top score, because branching on it fixes a variable. An integral child LP found during strong branching is offered as an incumbent.

9.2 Domain propagation

At every node the bounds are propagated over the rows [1, 47] (mip/propagate.py). For a row $r^l \leq \sum_j a_j x_j \leq r^u$, the minimum and maximum activities of the row without x_j imply

$$a_j > 0 : \quad x_j \leq \frac{r^u - \text{act}_{-j}}{a_j}, \quad x_j \geq \frac{r^l - \text{act}_{-j}}{a_j}, \quad (39)$$

with the inequalities reversed for $a_j < 0$. Infinite contributions are counted, so a row with exactly one infinite contributor can still bound that one variable. Integer bounds are rounded inward ($\lfloor u + 10^{-6} \rfloor$, $\lceil \ell - 10^{-6} \rceil$). A continuous bound is accepted only if it improves by more than $10^{-3} \max(1, |\text{bound}|)$, so round-off cannot creep into the bounds. An empty domain, or an activity range that misses the row, proves the node infeasible. Up to 5 passes run per node. Bounds found for continuous columns guide the search but are not sent to the LP.

9.3 Primal heuristics

- **Simple rounding** at every node: round all integer columns of the LP solution and accept the point if it is feasible.

- **Fix-and-solve** at the root and every 100 nodes: fix every integer to its rounded LP value, propagate, and re-solve the LP for the continuous columns (at most 5 000 dual iterations).
- **Fractional diving** [11] at the root when no incumbent exists, and every 50 nodes while none does: repeatedly fix the least fractional integer to its nearest value, propagate and re-solve, until the LP is integral, infeasible or worse than the cutoff. The LP budget is $\max(1000, 2 \times \text{the LP iterations spent so far})$.

9.4 Gomory mixed-integer cuts

At the root, up to 20 rounds of Gomory mixed-integer (GMI) cuts [9, 27] tighten the relaxation (`mip/cuts.py`). Take an optimal tableau row whose basic variable is an integer column with fractional value. Write each nonbasic variable as its distance from the bound it sits at, $t_k \geq 0$, so that $x_B + \sum_k a_k t_k = b$ with $f_0 = \text{frac}(b) \in (0, 1)$. Every integer-feasible point then satisfies

$$\sum_{\substack{k \text{ int} \\ f_k \leq f_0}} \frac{f_k}{f_0} t_k + \sum_{\substack{k \text{ int} \\ f_k > f_0}} \frac{1 - f_k}{1 - f_0} t_k + \sum_{\substack{k \text{ cont} \\ a_k > 0}} \frac{a_k}{f_0} t_k + \sum_{\substack{k \text{ cont} \\ a_k < 0}} \frac{-a_k}{1 - f_0} t_k \geq 1, \quad f_k = \text{frac}(a_k). \quad (40)$$

The LP point has $t = 0$ and violates the cut by exactly 1. A nonbasic variable counts as integer when its column is integer and its bound is integral. Logical variables are expanded into the structural columns of their row, and the cut is mapped back to the unscaled x space.

Numerical safeguards. Following the practice of Cook, Dash, Fukasawa and Goycoolea [14], a cut is generated only from a row with $0.01 < f_0 < 0.99$. It is skipped when a free nonbasic variable appears in the row or a needed bound is infinite. Coefficients below 10^{-9} of the largest are removed by relaxing against the variable's bound, which keeps the cut valid. The cut is rejected if its coefficient ratio exceeds 10^6 or its efficacy (violation divided by $\|g\|_2$) is below 10^{-5} , and the survivors are scaled so that the largest coefficient is 1. A regression test checks that every cut generated on `p0033` keeps the known optimal integer point feasible.

The cut loop. Each round adds up to $\max(20, \min(200, m/2))$ cuts, most fractional rows first, and re-solves the LP. The loop stops when the LP is integral, no cut is found, three rounds together move the bound by less than 0.1%, or the cuts have grown the model beyond $4m + 1000$ rows. Cuts that are slack at the final root point are then dropped, because they would only slow down the node LPs.

Algorithm 4: Branch-and-bound with plunging (mip/bnb.py, simplified)

```

input :MILP, gap tolerance  $10^{-4}$ , time limit
1 propagate at the root; solve the root LP; GMI cut rounds; rounding, fix-and-solve, diving;
2  $\text{current} \leftarrow \text{root}$  (LP solved);  $\text{heap} \leftarrow \emptyset$ ;
3 while time remains do
4   if  $\text{current} = \emptyset$  then
5     pop nodes whose bound  $\geq z_{\text{cutoff}}$ ; if heap empty then break;
6     pop node with the smallest ( $\underline{z}$ ,  $-\text{depth}$ ); propagate; warm-start the dual simplex from its stored
       basis;
7     record the pseudocost of the branching that created it;
8   if LP infeasible or  $\lfloor \underline{z} \rfloor \geq z_{\text{cutoff}}$  then continue;
9   if LP solution integral then update incumbent; continue;
10  simple rounding; periodic diving and fix-and-solve;
11  choose  $j$  by reliability branching (38);
12  create children  $x_j \leq \lfloor x_j \rfloor$  and  $x_j \geq \lceil x_j \rceil$  with the parent's basis;
13  push the less favoured child; solve the favoured one;
14  if its bound satisfies (36) then  $\text{current} \leftarrow$  that child else push it;  $\text{current} \leftarrow \emptyset$ ;
15  if relative gap  $\leq 10^{-4}$  then break;

```

10 The interior-point method for convex QP

With a quadratic objective the normal equations $B(Q + D)^{-1}B^T$ lose their sparsity, because $(Q + D)^{-1}$ is dense in general. qp/ipm.py therefore solves the *augmented* system at every Newton step, factorised by the quasi-definite LDL^T of Section 5.5. The outer iteration is Mehrotra's predictor-corrector, as for LP [41, 50], in the form Gertz and Wright give for QP [25]. The regularisation follows Friedlander and Orban [22] and Altman and Gondzio [4].

10.1 The regularised augmented system

With the bounded standard form (5) and $D = X^{-1}Z + W^{-1}S$ (zero on free variables), the Newton system reduces to

$$\begin{bmatrix} -(Q + D + \rho I) & B^T \\ B & \delta I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} \hat{r} \\ r_b \end{bmatrix}, \quad \hat{r} = r_c - X^{-1}r_{xz} + W^{-1}(r_{ws} - Sr_u), \quad (41)$$

where now $r_c = c + Qx - B^T y - z + s$. The primal regularisation is $\rho = 10^{-10}$ (10^{-8} on free variables) and the dual regularisation $\delta = 10^{-10}$. The matrix is quasi-definite (Figure 4), so it has an LDL^T factorisation for the minimum-degree order. The pattern of K is fixed: the triplets of Q , the diagonal, B and B^T are sorted once, and each iteration only sums new values into the fixed slots.

Iterative refinement against the unregularised matrix. The regularisation perturbs the system being solved. Each solve is therefore followed by up to three refinement steps in which the residual is computed with the *true* matrix ($\rho = \delta = 0$, only the free-variable term kept) and corrected with the regularised factor:

$$v \leftarrow v + K_{\text{reg}}^{-1}(\text{rhs} - K_{\text{true}} v), \quad (42)$$

stopping when $\|\text{rhs} - K_{\text{true}} v\|_{\infty} \leq 10^{-13}(1 + \|\text{rhs}\|_{\infty})$. This removes the effect of ρ and δ from the direction, while the factorisation keeps the stability they bring [22].

Retry with a regularisation boost. A direction is accepted only if it is finite *and* the refined residual of the augmented system is below 10^{-6} relative to the right-hand side. A factorisation that lost too much to pivot flooring can return a direction that is finite but wrong: on QRECIPE it had entries of order 10^{46} . If the check fails, the factorisation is repeated with ρ and δ multiplied by 10^2 , up to five attempts in all. Only if every attempt fails does the method stop with `numerical_error`.

Starting point. The starting point is Mehrotra’s heuristic as for LP, with one change. Every component of x, z, w, s is floored at $\max(1, \|b\|_\infty/10^{12})$. When the heuristic lands on a point with tiny complementarity but large residuals (on YAO, $\mu \approx 2 \cdot 10^{-6}$ against a dual infeasibility of 0.5), the iterates hug the boundary and crawl, and the floor keeps the start well inside the cone.

Coupled step length. In LP the primal and dual steps are independent. In QP they are coupled through $Q\Delta x$ in the dual residual, so with $Q \neq 0$ the method takes one common step $\alpha = \min(\alpha_p, \alpha_d)$ by default, again at the fraction 0.9995 of the distance to the boundary.

Termination. The residual measures, the loose-tolerance acceptance on stalling and the infeasibility heuristics are those of the LP method (Section 7), with the primal objective $c^\top x + \frac{1}{2}x^\top Qx$ and the dual objective $b^\top y - U^\top s - \frac{1}{2}x^\top Qx$ in the gap. The gap is measured relative to $1 + \min(|f|, |f + c_0|)$, that is, also against the objective the user sees, constant included. With a large constant and an optimum near zero (HS268, GOULDQP3), a gap relative to f alone was far too loose, and points with objective errors near 10^{-5} were accepted. Reduced costs are mapped back as $z = c + Qx - A^\top y$ on the original model.

These three changes came from the failure analysis of the first Maros–Mészáros sweep (Section 13). With them, and with the corrected reference of Section 12, the second sweep solves 121 of 138 problems against 111 of 138 in the first. The two effects cannot be separated: part of the gain comes from the reference.

11 Numerical safeguards: what went wrong and what fixed it

Most of the effort in a simplex or interior-point code goes into its failure modes, not into the textbook iteration. This section collects the failures we met on the benchmark sets and explains the mechanism of each fix. The first sweeps (`netlib_simplex_v1.csv`, `netlib_ipm_v1.csv`) record the state before the fixes. Tables 4 and 5 list every problem whose result changed between versions.

Table 4: Dual simplex, v1 against v2: every Netlib problem not solved by both versions. v1 solved 84 of 90; the fixes of Section 11.1 solved 6 more. On the problems both versions finish, v2 is 1.2 times faster in shifted geometric mean, measured relative to HiGHS in each sweep.

Problem	v1			v2		
	status	rel. err	time (s)	status	rel. err	time (s)
cycle	time_limit	–	300.0	optimal	$1.7 \cdot 10^{-16}$	0.59
perold	infeasible	–	0.18	optimal	$1.4 \cdot 10^{-15}$	0.73
pilot	optimal	$3.8 \cdot 10^{-5}$	73.6	optimal	$1.7 \cdot 10^{-11}$	9.44
pilot87	time_limit	–	300.0	optimal	$6.1 \cdot 10^{-8}$	28.6
pilotnov	infeasible	–	0.84	optimal	$2.0 \cdot 10^{-16}$	0.65
tuff	time_limit	–	300.0	optimal	$3.3 \cdot 10^{-16}$	0.03

11.1 Dual simplex

False “infeasible” from the ratio test (`perold`, `pilotnov`). The v1 sweep declared two feasible problems infeasible within a second. A dual simplex concludes primal infeasibility when the ratio test finds no entering candidate: the dual is unbounded along the pivot row, which is a Farkas certificate. That conclusion is only as good as the reduced costs and the pivot row it rests on. The v1 ratio test did not use the textbook Harris bound. A Harris pass must bound the step by the *smallest relaxed* ratio $\min_j (|d_j| + \epsilon_d) / |\alpha_{rj}|$ over the candidates that remain after the bound flips. Only then does a step to any candidate with $t_j \leq \Theta$ leave every reduced cost within ϵ_d of its feasible sign. A step past that bound makes some d_j infeasible by more than the tolerance. Later ratio tests then exclude that j or treat it wrongly, and an empty candidate set, and so a false certificate, follows. The fixed kernel (Algorithm 2) computes the bound exactly as above. It also makes the pivot tolerance *relative* to the largest $|\alpha_{rj}|$ in the row, because the `pilot` models have rows whose entries span many orders of magnitude, where an absolute 10^{-7} admits pivots that are noise. Two further guards keep a certificate honest. First, before reporting infeasibility the code refactorises, recomputes x_B , y , d from scratch, repairs the dual feasibility, and repeats the iteration; only a second empty ratio test on a fresh factorisation counts. Second, the pivot seen from the row (α_{rq}) and from the column ($\alpha_{q,r}$) must agree to 10^{-6} relative, or the factorisation is renewed.

The source of v1 is not in the repository: the first commit already contains the fixed kernel. This account is therefore of the fixed code and of the failure recorded in the v1 sweep.

Stalling and cycling in phase 1 (`cycle`, `pilot87`, `tuff`). Three problems ran into the 300 s limit in v1. Degenerate problems give many zero reduced costs, so many ratio-test breakpoints tie at zero, and the dual simplex can pivot through a sequence of degenerate bases without improving the dual objective, possibly returning to a basis already visited. v1 perturbed the costs in phase 2 only, while phase 1 on Fourer’s auxiliary problem (22) is at least as degenerate: every bound there is 0 or ± 1 . The fix applies the random perturbation (23) in phase 1 as well, which breaks the ties. `cycle` went from a time-out to under a second (Table 4).

Wrong objective on `pilot`: the dual tolerance of the cleanup. v1 returned `optimal` on `pilot` with a relative objective error far above 10^{-6} (Table 4). The cause is the size of the variable ranges. After the perturbation is removed, reduced costs with the wrong sign but below the dual tolerance $\epsilon_d = 10^{-7}$ are accepted as optimal. The objective error they leave is bounded by

$$|f(x) - f^*| \lesssim \sum_{j \in \mathcal{N}} |d_j^{\text{infeas}}| (u_j - \ell_j), \quad (43)$$

and on `pilot` a reduced cost of 10^{-8} on a variable with a scaled range of 10^3 already shifts the objective by 10^{-5} . The fix polishes with the primal simplex to $\epsilon_d = 10^{-9}$ in the cleanup phase only (phase 1 and phase 2 keep 10^{-7} , which is enough to steer the search). v2 solves `pilot` to the reference objective (Table 4).

Singular bases. Basis repair (Section 5.1) replaces a dependent basic column by a logical instead of failing; the number of repairs is reported with each result. Refactorisation every 100 updates, or when the eta file grows beyond eight times the LU size, bounds the growth of rounding error in the product form.

Exact duals at return. The iteration updates d incrementally and y only at refactorisation, and cost shifts change c during the loop. The returned y is therefore recomputed as $B^{-T} c_B$ for the true costs. This matters for postsolve (Section 4.3) and for anyone who uses the duals as prices, as refinery planners do.

Table 5: Interior point, v1 against v2: every Netlib problem not solved by both versions. v1 solved 80 of 90; v2 solves 6 more. On problems both versions finish, v2 is 8.8 times faster in shifted geometric mean, measured relative to HiGHS in each sweep.

Problem	v1			v2		
	status	rel. err	time (s)	status	rel. err	time (s)
degen3	iteration_limit	$7.4 \cdot 10^{-9}$	4.39	optimal	$3.9 \cdot 10^{-8}$	1.06
dfl001	iteration_limit	$2.5 \cdot 10^{-6}$	207.7	numerical_error	$2.6 \cdot 10^{-6}$	202.6
forplan	error	–	–	optimal	$1.2 \cdot 10^{-10}$	0.03
greenbea	iteration_limit	$8.2 \cdot 10^{-3}$	3.11	numerical_error	8.6	0.44
greenbeb	iteration_limit	$1.6 \cdot 10^{-3}$	3.15	optimal	$7.2 \cdot 10^{-7}$	1.03
grow15	numerical_error	–	1.11	optimal	$2.5 \cdot 10^{-15}$	0.02
grow22	numerical_error	–	1.14	optimal	$4.6 \cdot 10^{-14}$	0.03
pilot	iteration_limit	$1.6 \cdot 10^{-5}$	10.7	numerical_error	$7.5 \cdot 10^{-5}$	4.46
pilot87	iteration_limit	$3.0 \cdot 10^{-5}$	30.4	numerical_error	$3.3 \cdot 10^{-4}$	19.7
standgub	error	–	–	optimal	$1.9 \cdot 10^{-11}$	0.01

11.2 Interior-point method

Stall detection. In v1 several problems (degen3, greenbeb, pilot, pilot87, dfl001) ran to the iteration limit, some of them with objectives already correct to 10^{-8} . Near the solution Θ spans many orders of magnitude and the normal equations lose the digits needed to reach 10^{-8} . Further iterations only drive μ towards zero and z_j/x_j towards overflow. The v2 rule of Section 7 detects this. When the error has not halved in four iterations (or μ has collapsed), a point that meets 10^{-6} is accepted, and after 30 such iterations the method gives up with `numerical_error` instead of wandering. Residuals are measured as relative ∞ -norms (29), the convention of HiGHS/IPX, so the tolerance means the same on every problem. The remaining failures (dfl001, greenbea, pilot, pilot87) are the problems where even 10^{-6} is out of reach of the normal equations without presolve and crossover (Section 13).

Cholesky pivot guard. Dependent rows make M singular at the limit. A pivot $\leq 10^{-30}$ is replaced by 10^{128} (Section 5.2), which zeroes that component of Δy instead of producing `inf`. The dual regularisation $\delta = 10^{-10}$ and the two refinement steps keep the resulting error small. The numerical errors on grow15 and grow22 in v1 no longer occur.

11.3 QP interior point: the LDL^T pivot floor

The augmented matrix is quasi-definite only in exact arithmetic with $\rho, \delta > 0$. When D spans many orders of magnitude, cancellation can still produce a pivot of the wrong sign or of magnitude 10^{-20} , and dividing by it destroys the factorisation. The kernel replaces any pivot with $\sigma_k D_{kk} < \delta_{\min}$ by $\sigma_k \delta_{\min}$ (Section 5.5), so the inertia is always right. Without the floor a single bad pivot produces nan in the direction and ends the solve. Refinement against the true matrix recovers the accuracy (42). The floor alone is not enough, though. Flooring many pivots changes the matrix so much that the direction can be finite and still wrong (QRECIPE). The method therefore also checks the refined residual of every direction, and if the check fails it boosts the regularisation by 10^2 and refactorises (Section 10).

11.4 Branch-and-bound: heap tie-breaking

When the objective is integral, node bounds are rounded up, so many open nodes share exactly the same key. A heap keyed by $(z, \text{counter})$ breaks those ties by insertion order: it takes the *oldest*, hence shallowest, node among equals. The search then proceeds breadth-first across a level of equal bounds,

the open list grows, and incumbents that a deeper node would give arrive late. Adding $-\text{depth}$ as the second key (35) makes ties go to the deepest node. This changes nothing about which bound is processed next, so the best-bound guarantee is unchanged. It only makes the search finish subtrees it has already started.

11.5 Presolve: dual postsolve order

See Section 4.3: singleton-row duals depended on the order in which reductions were undone, and recording the column’s cost and live rows at removal time fixed it.

What these failures have in common

The worst of these bugs produced confident wrong answers (“infeasible”, “optimal” with the wrong objective, duals with the wrong sign) rather than crashes or error messages. Each was found because the benchmark harness checks every result against an independent solver in a separate process, and because the dual check tests the optimality conditions on the original model. Section 12 describes these checks.

12 Verification methodology

A solver is useful only if its “optimal” can be trusted. NIRNAY is checked at four levels.

12.1 Independent references, in separate processes

The harnesses `bench/run_lp.py`, `run_mip.py` and `run_qp.py` run every instance in a fresh process with a wall-clock limit, so one pathological instance cannot stall or corrupt a sweep. Each process first solves a tiny instance, so that loading the Numba-compiled code is not charged to the first real instance, and then times only the solve call (file reading excluded). The same file is then solved by a reference solver:

- **LP and MILP:** HiGHS 1.15.1 [31] with the same time limit, reading its own copy of the file;
- **QP:** Clarabel 0.11.1 [29], an interior-point conic solver. HiGHS’s active-set QP solver times out on the larger Maros–Mészáros problems. The model Clarabel sees is read by HiGHS’s MPS parser, not ours, so a reader bug in NIRNAY cannot hide by affecting both sides. The failure analysis of the first QP sweep found two defects in this reference, and the harness was corrected for the second sweep. First, at its default tolerances Clarabel’s objective is about 10^{-6} off on several problems where NIRNAY matches the published optimum, so the reference now runs at 10^{-11} . Second, a comparison of the two parsers (`tools/qps_compare.py`) showed that HiGHS 1.15.1 misreads the RHS section of DPKL01 and drops coefficients below 10^{-9} in HUESTIS, HUES-MOD and KSIP. On files where the two parsers disagree, the reference now solves NIRNAY’s reading and says so in its status.

Neither reference is imported by the `nirnay` package. `bench/comparators.py`, which drives the other solvers of Section 15, carries the same rule in its header.

12.2 What counts as solved

With f the objective reported by NIRNAY and f^* the reference objective,

$$\text{err} = \frac{|f - f^*|}{\max(1, |f^*|)}. \quad (44)$$

- **LP:** status optimal and $\text{err} < 10^{-6}$;
- **MILP:** status optimal (relative gap proved $\leq 10^{-4}$) and $\text{err} \leq 10^{-4}$;
- **QP:** status optimal, $\text{err} < 10^{-6}$, and the reference itself reports Solved or AlmostSolved. Problems whose reference failed are listed but not scored.

The harnesses also record the largest row and bound violations (and, for MILP, the integrality violation) of the returned x on the original model. A wrong answer therefore cannot pass on the objective alone. PDLP stops at a relative KKT tolerance of 10^{-4} , and its objective errors are reported against several thresholds (Section 8.3).

12.3 Optimality conditions after postsolve

`tools/dual_check.py` tests the conditions (2) directly on the original model: primal feasibility, and the sign of every y_i and z_j against the activity of its row or bound. This checks presolve, postsolve, unscaling and the simplex duals together (Section 4.3). It currently passes on 88 of 90 Netlib problems with no failure; the other 2 stop at its 60 s limit.

12.4 Regression tests and file round trips

`tests/` holds 31 test functions (many parametrised over several instances). They check each engine against published optimal values: Koch’s final Netlib results [33], the MIPLIB 3 solution values and the Maros–Mészáros repository [40]. They also check the kernels (the LDL^T on a random quasi-definite matrix, propagation on a hand-made example), and that every Gomory cut generated on `p0033` keeps the known optimal integer point feasible. Further tests cover PDLP at three tolerances, on infeasible and unbounded problems, with limits, and on the GPU. The MPS reader and writer are checked by round trips: read, write and read again gives identical arrays on the benchmark files, and HiGHS reads the original and the written file into the same model. The case-study tests re-solve every case file with HiGHS against its stored reference.

12.5 How speed is measured

Times are wall-clock seconds on the development laptop (Intel Core i5-11400H, 6 cores / 12 threads, 16 GB RAM, NVIDIA RTX 3050 Laptop GPU with 4 GB, Windows 11), taken while other work was running. Differences below a factor of two on sub-second runs are noise. Two aggregate measures are used:

- the **shifted geometric mean** $\text{sgm}_s(t) = \exp\left(\frac{1}{N} \sum_i \log(t_i + s)\right) - s$, with shift $s = 1$ s (and $s = 10$ s as a check). The shift keeps sub-millisecond instances from dominating a ratio of means. It also compresses ratios on small instances, so we additionally report the plain geometric mean and the median of per-instance time ratios;
- **performance profiles** [18]: for solver s on problem p let $r_{p,s} = t_{p,s} / \min_{s'} t_{p,s'}$ (∞ if s failed). The profile $\rho_s(\tau) = \frac{1}{N} |\{p : r_{p,s} \leq \tau\}|$ is the fraction of problems s solves within a factor τ of the fastest. $\rho_s(1)$ is how often s is fastest, and $\rho_s(\infty)$ how often it solves at all.

13 Benchmark results

Three public test sets are used, all with published optimal values: the 90 LPs of Netlib [23], from 24 to 6071 rows and up to 144848 nonzeros (maros-r7); the 64 MILPs of MIPLIB 3 [12]; and the 138 convex QPs of Maros and Mészáros [40]. The large LPs of Section 8 come from Mittelman’s benchmark

Table 6: Summary of the benchmark sweeps. “Solved” uses the criteria of Section 12. Times are shifted geometric means (shift 1 s) over the instances NIRNAY solves, for NIRNAY and for the reference on the same instances.

Suite	Engine	Reference	Solved	NIRNAY (s)	Ref. (s)	Ratio
Netlib LP	dual simplex (v3)	HiGHS	90/90	1.11	0.298	3.7
Netlib LP	interior point (v2)	HiGHS	86/90	0.23	0.070	3.3
Netlib LP	PDLP, 10^{-4}	HiGHS	83/90 ^a	0.75	0.107	7.1
MIPLIB 3	branch-and-bound (v1)	HiGHS	25/64 ^b	10.91	1.19	9.1
Maros–Mészáros	QP interior point	Clarabel	121/138	0.43	0.30	1.4

^a status optimal at relative KKT 10^{-4} ; objective errors in Section 8.3. ^b 60 s limit; HiGHS proves 45.

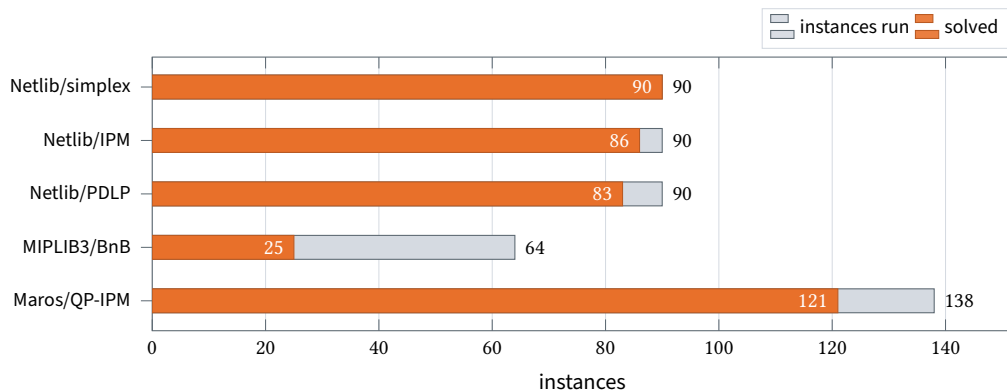


Figure 10: Solved instances per suite and engine (from `data/solved_bars.dat`, generated from `results/`). PDLP counts status optimal at its 10^{-4} tolerance.

sets [42]. Every number below is read from the result files named in the captions. Table 6 gives the overview and Figure 10 the solved counts.

Sweeps still running when this report was built

`gen_data.py` analyses the newest sweep of each suite that covers every instance. The following newer sweeps were still running and are not yet in the headline numbers: `miplib3_bnb_v2.csv` (31 of 64). Where a comparison with them is meaningful on the instances finished so far, the text gives it and says so. Rerunning `gen_data.py` and `latexmk` after they finish updates every number.

13.1 Netlib LP: correctness

The dual simplex solves all 90 of the 90 Netlib problems to the reference objective (file `netlib_simplex_v3.csv`). The median relative objective error is $3.7\text{e-}16$ and the largest $2.6\text{e-}09$, so the answers are vertex solutions exact to rounding. The interior-point method solves 86 (median error $9.0\text{e-}11$, largest $7.2\text{e-}07$). Figure 11 shows the distribution of the errors for every engine. The interior-point failures are analysed in Section 13.3.

13.2 Netlib LP: speed

Figure 12 shows the performance profiles of the dual simplex, the interior-point method and HiGHS on Netlib, and Figure 13 plots every instance’s time against the reference’s.

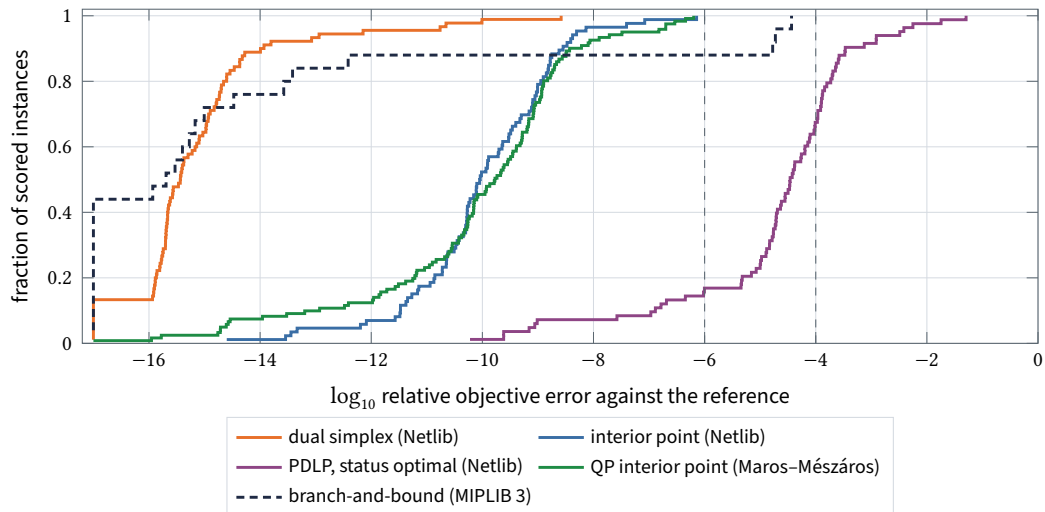


Figure 11: Cumulative distribution of the relative objective error (44) over the instances each engine reports optimal (for LP, QP and the dual simplex: the instances counted as solved; for PDLP: every status optimal). Errors of exactly zero are plotted at 10^{-17} . Dashed lines: the 10^{-6} LP/QP and 10^{-4} MILP criteria.

Speed, stated plainly

NIRNAY’s dual simplex is faster than HiGHS on 0 of the 90 Netlib problems it solves. Its shifted geometric mean time is 1.11 s against 0.298 s, a ratio of 3.7 with a 1 s shift and 3.9 with a 10 s shift. The per-instance ratios have geometric mean 8.3, median 8.8 and maximum 29. The shifted mean understates the gap because most Netlib problems solve in milliseconds, so the per-instance ratios are the fairer summary: NIRNAY is about 8.3 times slower. On load-normalised times (Figure 12), the interior-point method is faster than the simplex on 74 of the 86 problems both solve, and taking the faster of the two engines on each problem gives a shifted-mean ratio of 2.2 against HiGHS.

Where the time goes. The iteration counts are reasonable: the median Netlib solve takes 427 dual iterations, and Figure 14 shows the count growing roughly in proportion to $m + n$, as expected for a dual simplex. The cost is in each iteration. Every step calls several compiled kernels (pricing, BTRAN, pivot row, ratio test, two or three FTRANs, updates) from a Python loop. On small problems the Python overhead of each call exceeds the arithmetic. On large ones the linear algebra is less refined than in production codes: FTRAN and BTRAN work on dense vectors of length m instead of exploiting hyper-sparsity, the product-form update grows the eta file where a Forrest–Tomlin update would modify U in place. The simplex sweep analysed here runs with presolve, which is still written in plain Python; its cost and benefit are measured in Section 4. Section 16 lists the remedies in the order we expect them to pay.

13.3 Interior-point failures

Table 7 lists the failures. All end in the stall rule’s `numerical_error`, and 3 of them are within 10^{-3} of the optimum when they stop. These are the problems known to be hard for interior-point methods without presolve. The comparator study found Clarabel and GLPK’s interior point also failing on `greenbea` without presolve. The `pilot` models have coefficients spanning many orders of magnitude, and `df1001`’s normal matrix fills heavily. Production IPMs solve them with presolve, crossover to a basis, and more careful handling of free variables. Within NIRNAY, the dual simplex solves all of them.

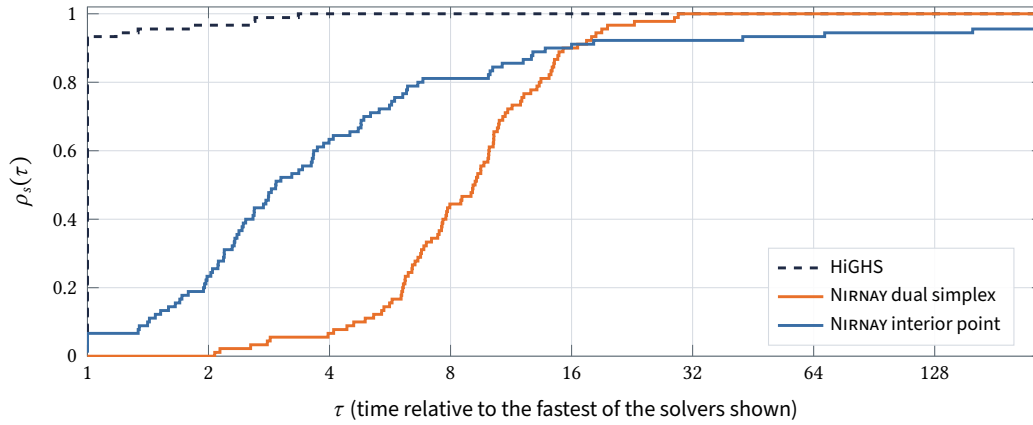


Figure 12: Dolan–Moré performance profile on the 90 Netlib problems. Times come from `netlib_simplex_v3.csv` and `netlib_ipm_v2.csv`. The two sweeps ran under different machine load (HiGHS itself was 3.4 times slower, in geometric mean, in the first than in the second), so each engine’s time is rescaled by the ratio of the common HiGHS time (the geometric mean of its two measurements) to HiGHS’s time in that engine’s sweep. HiGHS is fastest on a fraction 0.93 of the problems. The dual simplex is within a factor of 10 of the fastest on a fraction 0.61 and within 29 on all of them. The interior-point method’s curve stays below 1 because of its failures.

Table 7: Netlib problems the interior-point method does not solve (`netlib_ipm_v2.csv`).

Problem	m	n	status	rel. err	time (s)	iterations
df1001	6071	12230	numerical_error	$2.6 \cdot 10^{-6}$	202.6	85
greenbea	2392	5405	numerical_error	8.6	0.44	31
pilot	1441	3652	numerical_error	$7.5 \cdot 10^{-5}$	4.46	81
pilot87	2030	4883	numerical_error	$3.3 \cdot 10^{-4}$	19.7	101

13.4 MIPLIB 3

With a 60 s limit, branch-and-bound proves 25 of 64 instances optimal, all with the reference objective. No instance was declared optimal with a wrong objective (0). HiGHS, under the same limit, proves 45. 25 instances are proved by both, 20 only by HiGHS, and 0 only by NIRNAY. NIRNAY is faster than HiGHS on 1 of its solved instances (`air03`). On the instances it solves, its shifted geometric mean time is 10.91 s against 1.19 s for HiGHS, 9.1 times slower.

The instances not proved fall into three groups:

- **optimum found, not proved** (3): `bell3a`, `mod008`, `noswot`. The incumbent is optimal, but the bound does not close in time;
- **incumbent with a gap** (26 with an incumbent at the limit, of which 3 are within 1% and 13 within 10%), see Figure 15;
- **no incumbent at all** (12): `10teams`, `arki001`, `dano3mip`, `danoipnt`, `fast0507`, `markshare2`, `mitre`, `mkc`, `qnet1_o`, `rentacar`, `seymour`, `swath`. These are mostly large set-partitioning and network-design models, where rounding and diving fail and each LP is expensive.

The causes are the ones the comparator study (Section 15) points to. NIRNAY’s MIP presolve only tightens bounds. Its only cut family is Gomory mixed-integer; production codes add MIR, knapsack cover, flow cover, clique and zero-half cuts. It has no feasibility pump, no feasibility jump and no large-neighbourhood search. Node throughput is low: the median solved instance processes 34 nodes per second at a median of 35 dual iterations per node, and each node pays the Python overhead of the

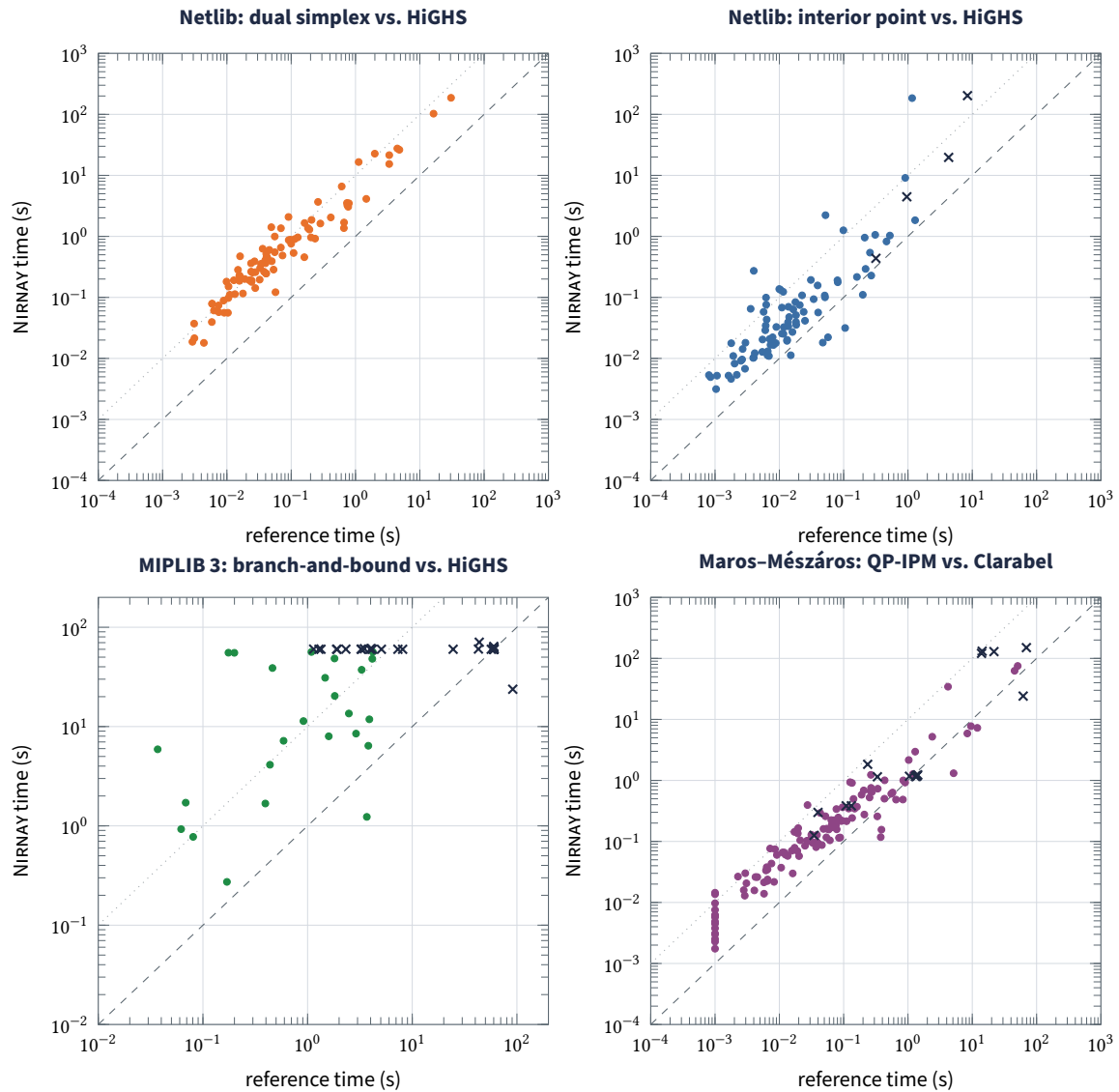


Figure 13: Time of NIRNAY against the time of the reference solver, per instance, on log-log axes. Dots: solved; crosses: not solved (plotted at the time used). Dashed: equal time; dotted: ten times slower. Points above the dashed line are instances where NIRNAY is slower. On MIPLIB the 60 s limit shows as a horizontal row of crosses.

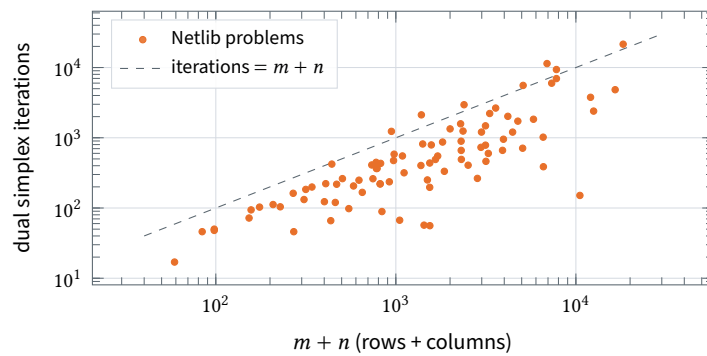


Figure 14: Dual simplex iterations (phase 1, phase 2 and cleanup together) against $m + n$ on the Netlib problems.

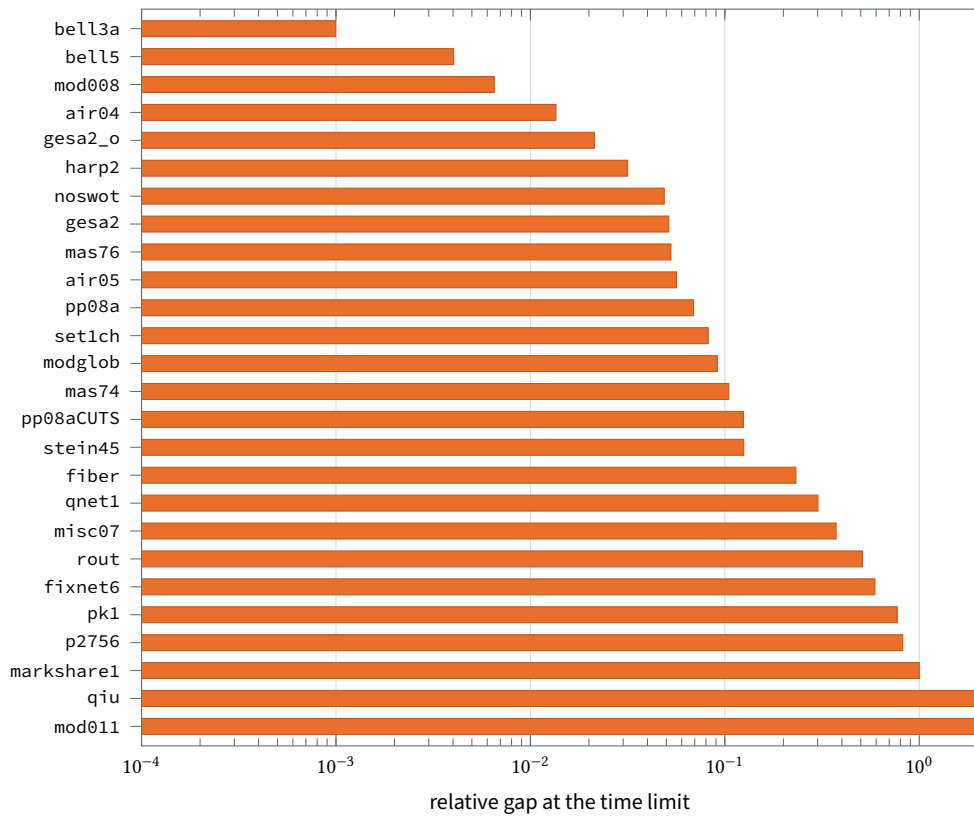


Figure 15: MIPLIB 3 instances that reached the 60 s limit with an incumbent: relative gap $|f - \underline{z}| / \max(1, |f|)$ at the limit (log scale). The target is 10^{-4} .

simplex driver. The largest search on record took 53129 nodes.

Effect of cuts and presolve. The sweep with Gomory cuts and presolve (`miplib3_bnb_v2.csv`) was still running when this report was built, with 31 of 64 instances finished. It proves 17 of them optimal. On the 31 instances both sweeps cover, it proves 3 instances that v1 (no cuts, no presolve) did not (bell5, fiber, gesa2_o), and loses 1 (gt2).

13.5 Maros–Mészáros QP

The QP interior point solves 121 of the 138 problems (`maros_qpipm_v2.csv`), with a median error of $1.7\text{e-}10$ and a median of 14 iterations (the most 78). It is faster than Clarabel on 9 of them, and slower by a factor of 1.4 in shifted geometric mean. The rest divide as follows:

- **reference failed** (1): BOYD2. Clarabel did not report a solution, so these count as not solved whatever NIRNAY returned;
- **accepted loosely** (0): none. NIRNAY reports optimal with an objective error between 10^{-6} and 10^{-3} against the reference: points whose relative residuals met the tolerance while the objective error did not (or, see below, reference error);
- **wrong answer** (0): none. Reported optimal with a large objective error against the reference;
- **numerical error** (12): HUES-MOD, HUESTIS, KSIP, LISWET1, LISWET10, LISWET11, LISWET12, LISWET7, LISWET8, LISWET9, UBH1, YAO. The stall rule gave up. Clarabel itself reports only AlmostSolved on 9 of the problems NIRNAY does not solve;
- **other** (4): CONT-300, CVXQP1_L, CVXQP2_L, CVXQP3_L. The solve stopped at the time limit, or the benchmark process returned no status; these count as not solved.

No problem is reported optimal with an objective error above 10^{-6} : every failure is an honest status (numerical error, time limit) rather than a wrong answer. On 4 files the two MPS parsers disagree, and the reference solves NIRNAY's reading (Section 12). Presolve is not yet applied to QPs. Table 16 in the appendix lists every problem.

14 Industrial case studies

Benchmarks show that a solver is correct; case studies show that it fits the problems the problem statement is about. `nirnay/cases/` builds 7 optimisation models from public data, most of it Indian. Every module carries a `CASE_INFO` record of its sources (URL and licence), of what is taken from them unchanged, and of what is assumed. `docs/CASE_STUDIES.md` documents each model in full. Each case is written to `data/cases/<case>.mps` and re-solved by HiGHS as a reference. Table 8 gives the sizes.

Table 8: Case-study models: problem class and size of the default instance, read from the MPS files in `data/-cases/` (integer columns in parentheses).

Case	Module	Class	Rows	Columns	Nonzeros
Refinery crude selection and blending	<code>refinery_planning</code>	LP	93	188	726
Twelve-month refinery production plan	<code>refinery_multiperiod</code>	MILP	1391	2579 (227)	9721
Thermal unit commitment (RTS-GMLC, 48 h)	<code>unit_commitment</code>	MILP	39395	44544 (16128)	167806
Economic dispatch with DC network (2000 buses)	<code>economic_dispatch</code>	QP	9266	5871	25669 (Q: 122)
Capacitated facility location (capb)	<code>facility_location</code>	MILP	101100	100100 (100)	400100
BS-VI petrol and diesel distribution	<code>product_distribution</code>	LP	114	1512	3024
Crude unloading and blending schedule	<code>crude_scheduling</code>	MILP	2470	1848 (112)	17748

14.1 The models

Refinery crude selection and blending (LP). The model chooses how much of each crude to run on each of MRPL's three crude distillation units (capacities from MRPL's public project filing), where each distillation cut goes, and how to blend BS-VI petrol (MS 91, MS 95), BS-VI diesel, ATF, LPG, naphtha and fuel oil to the BIS specifications (IS 2796, IS 1460, IS 1571), to maximise gross margin at PPAC's 2024–25 average prices. Crude assays come from the US Department of Energy's Strategic Petroleum Reserve (public domain). Every cut of every crude is its own stream, so the property blending is linear. For a product pool P with components i of mass m_i and density ρ_i , a volumetric specification such as the octane number is written

$$\sum_{i \in P} \frac{m_i}{\rho_i} (\text{RON}_i - \text{RON}_{\min}) \geq 0, \quad (45)$$

which is linear in the mass flows. Density, cetane index, smoke point, naphthalenes and sulphur are handled the same way.

Twelve-month production plan (MILP). Twelve monthly copies of the refinery network are linked by product inventories and storage limits. Crude is bought in integer parcels, at most three crudes

are processed per month (tank segregation), and the FCC severity is a binary choice with at most two changeovers per year. Monthly demand follows PPAC’s national consumption pattern.

Thermal unit commitment (MILP). The 73 thermal units of the RTS-GMLC system over 48 hours, with the tight and compact formulation of Morales-España, Latorre and Ramos [43] as published with PGLib-UC: on, start and stop binaries, start-up categories, ramping, minimum up and down times, spinning reserve, and a piecewise-linear production cost.

Economic dispatch with a DC network (QP). Quadratic generation cost over the 2000-bus PGLib-OPF case, with DC power flow, line limits and angle-difference limits. The HiGHS optimum matches the DC objective PGLib-OPF publishes for the case.

Capacitated facility location (MILP). OR-Library’s capb instance [10] (100 depots, 1000 customers) with the strong linking constraints $x_{ij} \leq y_i$. The HiGHS optimum equals the published optimum. It stands in for depot and terminal network design, for which no Indian dataset with costs and capacities is public.

BS-VI fuel distribution across India (LP). A transport model from 21 Indian refineries to 36 states and union territories. It meets every state’s 2024–25 MS and HSD sales (PPAC) with the least tonne-kilometres, using refinery and state locations from Wikidata.

Crude-oil unloading and blending schedule (MILP). The crude scheduling problems of Lee et al. [34] in the multi-operation sequencing formulation of Mouret, Grossmann and Pectiaux [44] (the MILP relaxation that is the first step of their method). All four instances reproduce the published optimal values.

14.2 Reference solutions and NIRNAY’s results

Table 9 gives the HiGHS reference for each case file, and Table 10 the record of NIRNAY’s runs kept in the case-study document.

Table 9: HiGHS 1.15.1 reference results for the case files (data/cases/reference.csv). Refinery objectives are negative gross margins in thousand US\$. The unit-commitment reference was solved to a 1% gap target (final gap 0.73%), as in PGLib-UC’s own reference script, so it is not a proven optimum.

Case	Class	Status	Objective	MIP gap	Time (s)
refinery_planning	LP	Optimal	−290 643.2878	–	0.01
refinery_multiperiod	MILP	Optimal	−290 457.5768	$9.1 \cdot 10^{-7}$	76.0
unit_commitment	MILP	Optimal	1 235 464.148	$7.3 \cdot 10^{-3}$	255.9
economic_dispatch	QP	Optimal	943 042.2073	–	1.90
facility_location	MILP	Optimal	13 656 379.58	0	592.3
product_distribution	LP	Optimal	45 369 609.95	–	0.01
crude_scheduling	MILP	Optimal	101.174 566 3	0	6.78

5 of the 7 recorded runs reach the HiGHS optimum: both LP cases with the simplex and the interior point, and the economic dispatch QP. The MILP cases do not yet solve. The twelve-month refinery plan stops at the time limit with a poor incumbent, the crude schedule returns nothing within the wall limit, and unit commitment and facility location, the two largest models in Table 8, were not attempted with branch-and-bound. This is the same gap as on MIPLIB (Section 13.4), and closing it is the first item of the roadmap.

Table 10: NIRNAY on the case studies (from the “NIRNAY on the cases” record in docs/CASE_STUDIES.md; one process per run, 120 s limit, times including Numba compilation on first call).

Case	Method	Status	Objective	Rel. diff.	Time (s)
refinery_planning	ipm	optimal	-290 643.2877	$4.6 \cdot 10^{-10}$	1.60
refinery_planning	simplex	optimal	-290 643.2878	$2.1 \cdot 10^{-14}$	1.70
product_distribution	ipm	optimal	45 369 609.95	$5.9 \cdot 10^{-12}$	1.80
product_distribution	simplex	optimal	45 369 609.95	$1.6 \cdot 10^{-16}$	2.70
economic_dispatch	qp-ipm	optimal	943 042.2073	$6.4 \cdot 10^{-11}$	2.60
refinery_multiperiod	bnb	time_limit	–	–	124.0
crude_scheduling	bnb	no result	–	–	–

15 Competitor analysis

Before deciding what to build next, we installed and ran the established solvers on the development machine and read their logs (docs/COMPETITOR_ANALYSIS.md, driver bench/comparators.py). The field covers HiGHS (dual simplex, IPX interior point, two PDLP variants, MIP, QP), SCIP with SoPlex, Google OR-Tools (GLOP, PDLP, CLP, CBC, SCIP, CP-SAT), CBC, GLPK, Clarabel, OSQP and NVIDIA cuOpt on the GPU. CPLEX, Gurobi and Xpress are commercial and were not installed. Each (instance, solver) pair ran in its own process at default settings on small sets of 12 Netlib LPs, 10 MIPLIB 3 instances and 10 Maros–Mészáros QPs. Tables 11 to 13 add NIRNAY’s own results on the same instances, taken from its sweeps.

15.1 What the comparison shows

- **Every winner is a dual simplex first.** HiGHS, SoPlex inside SCIP, GLOP and cuOpt’s concurrent mode all rely on a dual simplex for small and medium LPs and for node LPs. The interior point is the alternative for large LPs, and PDLP a third engine for very large LPs at low accuracy. cuOpt races all three and takes the first answer, and on the small Netlib LPs its own concurrent mode always picked the CPU dual simplex. NIRNAY has the same three engines; running them concurrently is on the roadmap.
- **PDLP failures are algorithmic.** Every PDLP implementation tested failed on the same badly scaled Netlib problems (pilot4, greenbea), as ours does. On large, structured LPs the picture reverses: GPU PDLP beats every factorisation-based method on the laptop (Section 8.3).
- **Presolve matters more than anything else.** A census of HiGHS’s presolve on Netlib and MIPLIB 3 shows that singleton rows, doubleton equations, the aggregator (free-column substitution), forcing rows, and fixed and dominated columns do most of the work. It is also what makes an interior point robust: Clarabel and GLPK’s interior point fail on greenbea without it, and HiGHS’s IPX succeeds with it.
- **MIP gaps close at the root.** Gomory mixed-integer and (c-)MIR cuts are the common core of SCIP, CBC and cuOpt, with knapsack cover, zero-half, clique and implied-bound cuts next. First incumbents come from cheap heuristics run before or right after the first LP (HiGHS’s feasibility jump, SCIP’s trivial, locks and shifting heuristics).
- **References make mistakes too.** The study found a wrong “optimal” from one solver’s MIP on p0201, a “Solved” status at a 10^{-3} objective error from another on greenbea, false infeasibility verdicts from GLPK’s interior point, and a time-limit defect in one PDLP. NIRNAY’s harness therefore scores against a reference in a separate process and records the reference’s own status (Section 12).

Table 11: Comparator solvers on the LP set (25fv47, adlittle, afiro, blend, degen2, ex10, greenbea, pilot4, qap15, sc50a, share2b, stocfor1). Solved: objective within 10^{-6} of the reference (10^{-3} for the first-order PDLP and OSQP runs, whose default tolerances are loose). Times: shifted geometric mean over the solved instances and the largest time. From results/comparators_lp.csv and NIRNAY's sweeps; sorted by solved count, then time.

Solver	Platform	Solved	sgm (s)	max (s)
highs	Windows 11 (CPU)	10/12	0.04	0.26
highs-simplex	Windows 11 (CPU)	10/10	0.04	0.26
highs-ipm	Windows 11 (CPU)	10/12	0.06	0.36
ortools-glop	Windows 11 (CPU)	10/10	0.08	0.71
glpk	Windows 11 (CPU)	10/10	0.08	0.62
ortools-clp	Windows 11 (CPU)	10/10	0.09	0.83
cbc	WSL2 (CPU)	10/10	0.13	0.72
scip	Windows 11 (CPU)	10/10	0.28	3.32
cuopt-dualsimplex	WSL2 + RTX 3050 GPU	10/10	0.30	1.10
cuopt	WSL2 + RTX 3050 GPU	10/12	0.54	2.77
NIRNAY dual simplex	Windows 11 (CPU)	10/10	0.77	16.5
cuopt-barrier	WSL2 + RTX 3050 GPU	10/12	1.63	3.05
clarabel	Windows 11 (CPU)	9/10	0.02	0.09
NIRNAY IPM	Windows 11 (CPU)	9/10	0.06	0.23
cuopt-pdlp	WSL2 + RTX 3050 GPU	9/12	2.62	32.1
glpk-interior	Windows 11 (CPU)	8/10	0.01	0.07
NIRNAY PDLP (1e-4)	Windows 11 (CPU)	8/10	0.08	0.46
highs-hipdlp	Windows 11 (CPU)	8/10	0.18	1.91
ortools-pdlp	Windows 11 (CPU)	8/12	0.19	1.93
highs-pdlp	Windows 11 (CPU)	8/10	0.25	3.86
ortools-pdlp-tight	Windows 11 (CPU)	8/10	0.44	7.00
osqp	Windows 11 (CPU)	5/10	0.005	0.009
osqp-tight	Windows 11 (CPU)	3/10	0.13	0.44

15.2 Licences and cost

All the open-source comparators are free: HiGHS (MIT), SCIP and OR-Tools, Clarabel, OSQP and cuOpt (Apache-2.0), CBC (EPL-2.0), and GLPK (GPL-3.0, whose copyleft prevents linking into a non-GPL product). CPLEX, Gurobi and Xpress are proprietary: their free editions are limited in model size, and production licences are sold per user or per deployment, with prices quoted on request. NIRNAY is intended to be released under Apache-2.0, like most of the open-source field, and because it depends on no solver library, no licence restriction flows into it.

Table 12: MIP set (bell5, egout, flugpl, lseu, misc03, mod008, p0033, p0201, p0282, stein27); solved: optimal within 10^{-4} . *: GPU.

Solver	Solved	sgm (s)	max (s)
cbc	10/10	0.47	1.20
highs	10/10	0.52	1.28
scip	10/10	0.55	2.20
ortools-scip	10/10	0.57	1.51
glpk-cuts	10/10	1.50	5.96
glpk	9/10	0.25	0.89
ortools-cpsat	9/10	0.36	2.53
ortools-cbc	9/10	0.56	2.24
cuopt*	9/10	0.76	1.19
NIRNAY branch-and-bound	8/10	6.76	38.9

Table 13: QP set (CVXQP1_S, DUALC1, HS21, HS35, PRIMALC1, QADLITTL, QAFIRO, QPCBLEND, QSC205, QSHARE2B); solved: within 10^{-6} (10^{-3} for OSQP).

Solver	Solved	sgm (s)	max (s)
clarabel	10/10	0.002	0.004
highs	10/10	0.004	0.006
NIRNAY QP-IPM	10/10	0.03	0.08
osqp-tight	8/10	0.003	0.007
scip	8/10	0.19	0.42
osqp	7/10	0.001	0.005

16 Limitations and roadmap

16.1 Limitations

- **Speed.** The dual simplex is about 8.3 times slower than HiGHS on Netlib (geometric mean of per-instance ratios; 3.7 in shifted geometric mean), and branch-and-bound is 9.1 times slower on the MIPLIB instances it solves. The iteration drivers are Python; only the kernels are compiled.
- **MILP strength.** 25 of 64 MIPLIB 3 instances are proved optimal in 60 s against 45 for HiGHS. The MILP case studies do not yet solve. MIP presolve only tightens bounds, Gomory mixed-integer is the only cut family, the heuristics are simple, and there is no parallel tree search.
- **Presolve.** Only reductions with a simple exact postsolve exist so far (Section 4). Doubleton equations, forcing rows, dominated columns, parallel rows and columns, and coefficient tightening are missing, and QPs are not presolved at all.
- **Interior points.** Infeasibility is detected heuristically, not certified. There is no crossover to a basic solution, and the QP termination test can accept a point whose objective is wrong (Section 13).
- **Linear algebra.** FTRAN and BTRAN use dense work vectors, updates are product-form, and the Cholesky is up-looking rather than supernodal, so it uses no dense BLAS kernels.
- **Scope.** No MIQP, no second-order cones, no SOS constraints, no callbacks or modelling layer beyond the Python API and MPS files.

16.2 Roadmap, in order of expected payoff

1. **Move the simplex driver into compiled code.** Moving the whole iteration into one Numba (or C) loop removes the per-call overhead that dominates small problems. We expect this to be the largest single speed-up.
2. **Complete presolve,** following the census order: doubleton equations and the aggregator, forcing rows, dominated columns, parallel rows and columns, then MIP coefficient tightening and probing.
3. **MIP:** MIR and knapsack-cover cuts with a cut pool and ageing, feasibility jump and a feasibility pump for first incumbents, RINS/RENS-style neighbourhood search, reduced-cost fixing and a root restart.
4. **Linear algebra:** hyper-sparse FTRAN/BTRAN, a Forrest–Tomlin update, and a supernodal

Cholesky/ LDL^T with dense kernels, which would also make a GPU factorisation possible.

5. **Interior points:** a homogeneous self-dual formulation for certified infeasibility, crossover to a vertex, and a stricter QP gap test.
6. **Concurrent LP:** race dual simplex, interior point and GPU PDLP and take the first answer, as cuOpt does. The three engines already exist.
7. **Refinery-scale validation:** run the case-study MILPs to proven optimality, and add MRPL's own planning models when they become available.

16.3 Conclusion

NIRNAY shows that a solver meeting the independence requirement of SIH26119 can be built: every factorisation and algorithm is our own, the answers are verified against independent solvers, and on the standard LP and QP test sets it is correct. It is not yet fast, and its MILP code is far behind the state of the art. The report documents both the engine and its numerical failures, so the work can be continued.

References

- [1] Tobias Achterberg. *Constraint Integer Programming*. PhD thesis, Technische Universität Berlin, 2007.
- [2] Tobias Achterberg, Thorsten Koch, and Alexander Martin. Branching rules revisited. *Operations Research Letters*, 33(1):42–54, 2005. doi:[10.1016/j.orl.2004.04.002](https://doi.org/10.1016/j.orl.2004.04.002).
- [3] Tobias Achterberg, Robert E. Bixby, Zonghao Gu, Edward Rothberg, and Dieter Weninger. Presolve reductions in mixed integer programming. *INFORMS Journal on Computing*, 32(2):473–506, 2020. doi:[10.1287/ijoc.2018.0857](https://doi.org/10.1287/ijoc.2018.0857).
- [4] Anna Altman and Jacek Gondzio. Regularized symmetric indefinite systems in interior point methods for linear and quadratic optimization. *Optimization Methods and Software*, 11(1–4): 275–302, 1999. doi:[10.1080/10556789908805754](https://doi.org/10.1080/10556789908805754).
- [5] Erling D. Andersen and Knud D. Andersen. Presolving in linear programming. *Mathematical Programming*, 71(2):221–245, 1995. doi:[10.1007/BF01586000](https://doi.org/10.1007/BF01586000).
- [6] David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O’Donoghue, and Warren Schudy. Practical large-scale linear programming using primal-dual hybrid gradient. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021. arXiv:2106.04756.
- [7] David Applegate, Oliver Hinder, Haihao Lu, and Miles Lubin. Faster first-order primal-dual methods for linear programming using restarts and sharpness. *Mathematical Programming*, 201: 133–184, 2023. doi:[10.1007/s10107-022-01901-9](https://doi.org/10.1007/s10107-022-01901-9).
- [8] David Applegate, Mateo Díaz, Haihao Lu, and Miles Lubin. Infeasibility detection with primal-dual hybrid gradient for large-scale linear programming. *SIAM Journal on Optimization*, 34(1):459–484, 2024. doi:[10.1137/22M1510467](https://doi.org/10.1137/22M1510467).
- [9] Egon Balas, Sebastián Ceria, Gérard Cornuéjols, and N. Natraj. Gomory cuts revisited. *Operations Research Letters*, 19(1):1–9, 1996. doi:[10.1016/0167-6377\(96\)00007-7](https://doi.org/10.1016/0167-6377(96)00007-7).
- [10] John E. Beasley. OR-Library: Distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072, 1990. doi:[10.1057/jors.1990.166](https://doi.org/10.1057/jors.1990.166).
- [11] Timo Berthold. Primal heuristics for mixed integer programs. Diploma thesis, Technische Universität Berlin (Zuse Institute Berlin), 2006.
- [12] Robert E. Bixby, Sebastián Ceria, Cassandra M. McZeal, and Martin W. P. Savelsbergh. An updated mixed integer programming library: MIPLIB 3.0. *Optima*, 58:12–15, 1998.
- [13] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011. doi:[10.1007/s10851-010-0251-1](https://doi.org/10.1007/s10851-010-0251-1).
- [14] William Cook, Sanjeeb Dash, Ricardo Fukasawa, and Marcos Goycoolea. Numerically safe Gomory mixed-integer cuts. *INFORMS Journal on Computing*, 21(4):641–649, 2009. doi:[10.1287/ijoc.1090.0324](https://doi.org/10.1287/ijoc.1090.0324).
- [15] George B. Dantzig and William Orchard-Hays. The product form for the inverse in the simplex method. *Mathematical Tables and Other Aids to Computation*, 8(46):64–67, 1954.
- [16] Timothy A. Davis. Algorithm 849: A concise sparse Cholesky factorization package. *ACM Transactions on Mathematical Software*, 31(4):587–591, 2005. doi:[10.1145/1114268.1114277](https://doi.org/10.1145/1114268.1114277).

- [17] Timothy A. Davis. *Direct Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2006. doi:[10.1137/1.9780898718881](https://doi.org/10.1137/1.9780898718881).
- [18] Elizabeth D. Dolan and Jorge J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213, 2002. doi:[10.1007/s101070100263](https://doi.org/10.1007/s101070100263).
- [19] Joseph M. Elble and Nikolaos V. Sahinidis. Scaling linear optimization problems prior to application of the simplex method. *Computational Optimization and Applications*, 52(2):345–371, 2012. doi:[10.1007/s10589-011-9420-4](https://doi.org/10.1007/s10589-011-9420-4).
- [20] John J. Forrest and Donald Goldfarb. Steepest-edge simplex algorithms for linear programming. *Mathematical Programming*, 57:341–374, 1992. doi:[10.1007/BF01581089](https://doi.org/10.1007/BF01581089).
- [21] Robert Fourer. Notes on the dual simplex method. Unpublished notes, Northwestern University, 1994.
- [22] Michael P. Friedlander and Dominique Orban. A primal-dual regularized interior-point method for convex quadratic programs. *Mathematical Programming Computation*, 4(1):71–107, 2012. doi:[10.1007/s12532-012-0035-2](https://doi.org/10.1007/s12532-012-0035-2).
- [23] David M. Gay. Electronic mail distribution of linear programming test problems. *Mathematical Programming Society COAL Newsletter*, 13:10–12, 1985.
- [24] Alan George and Joseph W. H. Liu. The evolution of the minimum degree ordering algorithm. *SIAM Review*, 31(1):1–19, 1989. doi:[10.1137/1031001](https://doi.org/10.1137/1031001).
- [25] E. Michael Gertz and Stephen J. Wright. Object-oriented software for quadratic programming. *ACM Transactions on Mathematical Software*, 29(1):58–81, 2003. doi:[10.1145/641876.641880](https://doi.org/10.1145/641876.641880).
- [26] John R. Gilbert and Tim Peierls. Sparse partial pivoting in time proportional to arithmetic operations. *SIAM Journal on Scientific and Statistical Computing*, 9(5):862–874, 1988. doi:[10.1137/0909058](https://doi.org/10.1137/0909058).
- [27] Ralph E. Gomory. An algorithm for the mixed integer problem. Technical Report RM-2597, The RAND Corporation, 1960.
- [28] Jacek Gondzio. Presolve analysis of linear programs prior to applying an interior point method. *INFORMS Journal on Computing*, 9(1):73–91, 1997. doi:[10.1287/ijoc.9.1.73](https://doi.org/10.1287/ijoc.9.1.73).
- [29] Paul J. Goulart and Yuwen Chen. Clarabel: An interior-point solver for conic programs with quadratic objectives. arXiv:2405.12762, 2024.
- [30] Paula M. J. Harris. Pivot selection methods of the Devex LP code. *Mathematical Programming*, 5:1–28, 1973. doi:[10.1007/BF01580108](https://doi.org/10.1007/BF01580108).
- [31] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi:[10.1007/s12532-017-0130-5](https://doi.org/10.1007/s12532-017-0130-5).
- [32] Achim Koberstein. *The Dual Simplex Method, Techniques for a Fast and Stable Implementation*. PhD thesis, Universität Paderborn, 2005.
- [33] Thorsten Koch. The final NETLIB-LP results. *Operations Research Letters*, 32(2):138–142, 2004. doi:[10.1016/S0167-6377\(03\)00094-4](https://doi.org/10.1016/S0167-6377(03)00094-4).
- [34] Heeman Lee, Jose M. Pinto, Ignacio E. Grossmann, and Sunwon Park. Mixed-integer linear programming model for refinery short-term scheduling of crude oil unloading with inventory management. *Industrial & Engineering Chemistry Research*, 35(5):1630–1641, 1996. doi:[10.1021/ie950519h](https://doi.org/10.1021/ie950519h).

- [35] Haihao Lu and Jinwen Yang. cuPDLP.jl: A GPU implementation of restarted primal-dual hybrid gradient for linear programming in Julia. arXiv:2311.12180, 2023.
- [36] Haihao Lu, Jinwen Yang, et al. cuPDLP-C: A strengthened implementation of cuPDLP for linear programming by C language. arXiv:2312.14832, 2023.
- [37] Irvin J. Lustig, Roy E. Marsten, and David F. Shanno. Interior point methods for linear programming: Computational state of the art. *ORSA Journal on Computing*, 6(1):1–14, 1994. doi:[10.1287/ijoc.6.1.1](https://doi.org/10.1287/ijoc.6.1.1).
- [38] Harry M. Markowitz. The elimination form of the inverse and its application to linear programming. *Management Science*, 3(3):255–269, 1957. doi:[10.1287/mnsc.3.3.255](https://doi.org/10.1287/mnsc.3.3.255).
- [39] István Maros. A generalized dual phase-2 simplex algorithm. *European Journal of Operational Research*, 149(1):1–16, 2003.
- [40] István Maros and Csaba Mészáros. A repository of convex quadratic programming problems. *Optimization Methods and Software*, 11(1–4):671–681, 1999. doi:[10.1080/10556789908805768](https://doi.org/10.1080/10556789908805768).
- [41] Sanjay Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2(4):575–601, 1992. doi:[10.1137/0802028](https://doi.org/10.1137/0802028).
- [42] Hans D. Mittelmann. Benchmarks for optimization software. <https://plato.asu.edu/bench.html>. Accessed September 2026.
- [43] Germán Morales-España, Jesús M. Latorre, and Andres Ramos. Tight and compact MILP formulation for the thermal unit commitment problem. *IEEE Transactions on Power Systems*, 28(4):4897–4908, 2013.
- [44] Sylvain Mouret, Ignacio E. Grossmann, and Pierre Pestiaux. A novel priority-slot based continuous-time formulation for crude-oil scheduling problems. *Industrial & Engineering Chemistry Research*, 48(18):8515–8528, 2009. doi:[10.1021/ie8019592](https://doi.org/10.1021/ie8019592).
- [45] Thomas Pock and Antonin Chambolle. Diagonal preconditioning for first order primal-dual algorithms in convex optimization. In *2011 International Conference on Computer Vision (ICCV)*, pages 1762–1769, 2011. doi:[10.1109/ICCV.2011.6126441](https://doi.org/10.1109/ICCV.2011.6126441).
- [46] Daniel Ruiz. A scaling algorithm to equilibrate both rows and columns norms in matrices. Technical Report RAL-TR-2001-034, Rutherford Appleton Laboratory, 2001.
- [47] Martin W. P. Savelsbergh. Preprocessing and probing techniques for mixed integer programming problems. *ORSA Journal on Computing*, 6(4):445–454, 1994. doi:[10.1287/ijoc.6.4.445](https://doi.org/10.1287/ijoc.6.4.445).
- [48] William F. Tinney and John W. Walker. Direct solutions of sparse network equations by optimally ordered triangular factorization. *Proceedings of the IEEE*, 55(11):1801–1809, 1967. doi:[10.1109/PROC.1967.6011](https://doi.org/10.1109/PROC.1967.6011).
- [49] Robert J. Vanderbei. Symmetric quasidefinite matrices. *SIAM Journal on Optimization*, 5(1):100–113, 1995. doi:[10.1137/0805005](https://doi.org/10.1137/0805005).
- [50] Stephen J. Wright. *Primal-Dual Interior-Point Methods*. SIAM, Philadelphia, 1997. doi:[10.1137/1.9781611971453](https://doi.org/10.1137/1.9781611971453).

A Full benchmark tables

[†] marks a result that does not count as solved under the criteria of Section 12. Times are in seconds, and “rel. err” is the relative objective error (44).

Table 14: Netlib: every problem, with the dual simplex (netlib_simplex_v3.csv), the interior point (netlib_ipm_v2.csv) and PDLP at 10^{-4} (netlib_pdlp.csv; for PDLP [†] marks an objective error of 10^{-3} or more).

Problem	m	n	nnz	HiGHS time	dual simplex		interior point		PDLP	
					time	rel. err	time	rel. err	time	rel. err
25fv47	821	1571	10400	0.74	3.53	$2.0 \cdot 10^{-15}$	0.23	$5.0 \cdot 10^{-11}$	0.46	$6.4 \cdot 10^{-5}$
80bau3b	2262	9799	21002	0.61	6.58	$2.4 \cdot 10^{-16}$	0.54	$2.8 \cdot 10^{-9}$	1.62	$1.3 \cdot 10^{-4}$
adlittle	56	97	383	0.007	0.06	$1.2 \cdot 10^{-15}$	0.02	$3.5 \cdot 10^{-11}$	0.01	$3.9 \cdot 10^{-5}$
afiro	27	32	83	0.003	0.02	$1.2 \cdot 10^{-16}$	0.005	$2.4 \cdot 10^{-11}$	0.001	$9.8 \cdot 10^{-5}$
agg	488	163	2410	0.02	0.23	$1.0 \cdot 10^{-14}$	0.08	$1.4 \cdot 10^{-11}$	0.16	$2.3 \cdot 10^{-4}$
agg2	516	302	4284	0.03	0.26	$3.7 \cdot 10^{-16}$	0.14	$3.2 \cdot 10^{-10}$	0.12	$1.9 \cdot 10^{-4}$
agg3	516	302	4300	0.02	0.26	$7.2 \cdot 10^{-16}$	0.13	$3.5 \cdot 10^{-10}$	0.12	$3.8 \cdot 10^{-5}$
bandm	305	472	2494	0.04	0.52	$4.3 \cdot 10^{-15}$	0.05	$4.0 \cdot 10^{-10}$	0.16	$1.9 \cdot 10^{-5}$
beaconfd	173	262	3375	0.01	0.11	$2.2 \cdot 10^{-16}$	0.02	$9.6 \cdot 10^{-11}$	0.05	$1.6 \cdot 10^{-7}$
blend	74	83	491	0.009	0.06	$1.6 \cdot 10^{-15}$	0.01	$7.0 \cdot 10^{-11}$	0.007	$2.3 \cdot 10^{-5}$
bnl1	643	1175	5121	0.11	0.90	$1.8 \cdot 10^{-15}$	0.16	$1.7 \cdot 10^{-9}$	29.9 [†]	$2.3 \cdot 10^{-2}$
bnl2	2324	3489	13999	0.26	3.67	$2.0 \cdot 10^{-15}$	1.26	$2.3 \cdot 10^{-11}$	13.9 [†]	$5.6 \cdot 10^{-3}$
boeing1	351	384	3485	0.05	0.39	$3.4 \cdot 10^{-16}$	0.08	$7.2 \cdot 10^{-12}$	0.14	$5.4 \cdot 10^{-5}$
boeing2	166	143	1196	0.01	0.10	$1.8 \cdot 10^{-16}$	0.04	$3.0 \cdot 10^{-10}$	0.02	$7.0 \cdot 10^{-6}$
bore3d	233	315	1429	0.01	0.15	0	0.07	$8.2 \cdot 10^{-13}$	3.94	$2.1 \cdot 10^{-7}$
brandy	220	249	2148	0.02	0.19	$6.0 \cdot 10^{-16}$	0.03	$7.7 \cdot 10^{-10}$	0.19	$3.4 \cdot 10^{-5}$
capri	271	353	1767	0.02	0.36	$1.2 \cdot 10^{-15}$	0.10	$1.4 \cdot 10^{-9}$	1.19	$1.2 \cdot 10^{-4}$
cycle	1903	2857	20720	1.47	4.12	$1.7 \cdot 10^{-16}$	0.83	$2.9 \cdot 10^{-14}$	14.9	$4.5 \cdot 10^{-6}$
czprob	929	3523	10669	0.16	1.66	$2.1 \cdot 10^{-16}$	0.11	$5.4 \cdot 10^{-12}$	0.28	$2.5 \cdot 10^{-4}$
d2q06c	2171	5167	32417	3.35	21.5	$1.7 \cdot 10^{-15}$	1.84	$1.2 \cdot 10^{-10}$	74.3	$1.3 \cdot 10^{-4}$
d6cube	415	6184	37704	0.66	1.69	0	0.95	$4.7 \cdot 10^{-10}$	0.31	$7.5 \cdot 10^{-4}$
degen2	444	534	3978	0.07	0.49	$1.6 \cdot 10^{-16}$	0.11	$7.8 \cdot 10^{-11}$	0.04	$2.4 \cdot 10^{-5}$
degen3	1503	1818	24646	0.79	3.49	$8.1 \cdot 10^{-16}$	1.06	$3.9 \cdot 10^{-8}$	0.74	$1.7 \cdot 10^{-5}$
df1001	6071	12230	35632	30.6	186.7	$1.2 \cdot 10^{-14}$	202.6 [†]	$2.6 \cdot 10^{-6}$	13.5	$2.0 \cdot 10^{-5}$
e226	223	282	2578	0.03	0.20	$8.5 \cdot 10^{-14}$	0.02	$1.2 \cdot 10^{-10}$	0.21	$1.9 \cdot 10^{-5}$
etamacro	400	688	2409	0.05	0.60	$9.8 \cdot 10^{-11}$	0.03	$6.9 \cdot 10^{-10}$	0.04 [†]	$1.2 \cdot 10^{-3}$
ffff800	524	854	6227	0.04	0.40	$2.1 \cdot 10^{-16}$	0.12	$1.1 \cdot 10^{-9}$	8.52	$7.2 \cdot 10^{-5}$
finnis	497	614	2310	0.03	0.39	$6.7 \cdot 10^{-16}$	0.06	$8.4 \cdot 10^{-8}$	0.20 [†]	$3.9 \cdot 10^{-3}$
fit1d	24	1026	13404	0.06	0.12	$2.0 \cdot 10^{-16}$	0.03	$6.0 \cdot 10^{-11}$	0.44	$2.6 \cdot 10^{-8}$
fit1p	627	1677	9868	0.19	1.29	$8.0 \cdot 10^{-16}$	2.22	$3.3 \cdot 10^{-11}$	0.27	$9.6 \cdot 10^{-7}$
fit2d	25	10500	129018	0.66	1.37	$4.3 \cdot 10^{-16}$	0.29	$1.7 \cdot 10^{-9}$	30.0	$8.8 \cdot 10^{-10}$
fit2p	3000	13525	50284	4.47	27.6	$1.5 \cdot 10^{-15}$	184.5	$1.3 \cdot 10^{-10}$	21.0	$4.5 \cdot 10^{-7}$
forplan	161	421	4563	0.03	0.14	$3.4 \cdot 10^{-15}$	0.03	$1.2 \cdot 10^{-10}$	1.07 [†]	$5.1 \cdot 10^{-2}$
ganges	1309	1681	6912	0.07	1.36	$4.0 \cdot 10^{-16}$	0.07	$6.4 \cdot 10^{-13}$	0.26 [†]	$1.8 \cdot 10^{-2}$
gfrd-pnc	616	1092	2377	0.04	0.63	$2.7 \cdot 10^{-16}$	0.01	$8.8 \cdot 10^{-10}$	2.79	$1.3 \cdot 10^{-5}$
greenbea	2392	5405	30877	1.12	16.5	$2.9 \cdot 10^{-15}$	0.44 [†]	8.6	120.6 [†]	$9.4 \cdot 10^{-1}$
greenbeb	2392	5405	30877	1.99	22.7	$2.2 \cdot 10^{-16}$	1.03	$7.2 \cdot 10^{-7}$	122.0 [†]	$1.0 \cdot 10^{-3}$
grow15	300	645	5620	0.20	0.96	$1.4 \cdot 10^{-16}$	0.02	$2.5 \cdot 10^{-15}$	0.63	$6.0 \cdot 10^{-11}$
grow22	440	946	8252	0.41	2.05	0	0.03	$4.6 \cdot 10^{-14}$	0.55	$2.4 \cdot 10^{-10}$
grow7	140	301	2612	0.05	0.29	$1.6 \cdot 10^{-16}$	0.01	$3.6 \cdot 10^{-14}$	0.09	$2.4 \cdot 10^{-10}$

Problem	m	n	nnz	HiGHS	time	rel. err	time	rel. err	time	rel. err
israel	174	142	2269	0.02	0.12	$3.9 \cdot 10^{-16}$	0.03	$1.3 \cdot 10^{-11}$	0.05	$7.7 \cdot 10^{-5}$
kb2	43	41	286	0.004	0.02	$1.2 \cdot 10^{-13}$	0.005	$5.0 \cdot 10^{-11}$	0.06	$6.9 \cdot 10^{-10}$
lotfi	153	308	1078	0.01	0.19	$4.2 \cdot 10^{-16}$	0.009	$3.4 \cdot 10^{-12}$	0.99 [†]	$1.2 \cdot 10^{-3}$
maros	846	1443	9614	0.21	1.86	$2.5 \cdot 10^{-15}$	0.10	$1.5 \cdot 10^{-9}$	6.04	$1.1 \cdot 10^{-5}$
maros-r7	3136	9408	144848	3.34	15.4	$9.3 \cdot 10^{-16}$	9.10	$1.9 \cdot 10^{-10}$	1.30	$4.2 \cdot 10^{-5}$
modszk1	687	1620	3168	0.06	1.00	$7.1 \cdot 10^{-13}$	0.04	$2.3 \cdot 10^{-10}$	5.91	$9.7 \cdot 10^{-10}$
nesm	662	2923	13288	0.75	3.24	$1.1 \cdot 10^{-15}$	0.11	$1.8 \cdot 10^{-9}$	0.44	$6.4 \cdot 10^{-5}$
perold	625	1376	6018	0.28	1.63	$1.6 \cdot 10^{-14}$	0.18	$1.2 \cdot 10^{-9}$	51.2 [†]	$1.0 \cdot 10^{-4}$
pilot	1441	3652	43167	4.81	26.3	$1.7 \cdot 10^{-11}$	4.46 [†]	$7.5 \cdot 10^{-5}$	126.1 [†]	$9.9 \cdot 10^{-4}$
pilot4	410	1000	5141	0.13	0.96	$5.3 \cdot 10^{-15}$	0.19	$9.4 \cdot 10^{-10}$	32.8 [†]	$6.8 \cdot 10^{-3}$
pilot87	2030	4883	73152	16.3	102.6	$2.6 \cdot 10^{-9}$	19.7 [†]	$3.3 \cdot 10^{-4}$	126.9 [†]	$1.2 \cdot 10^{-2}$
pilotnov	975	2172	13057	0.77	3.05	$2.0 \cdot 10^{-16}$	0.22	$1.9 \cdot 10^{-11}$	2.61	$1.1 \cdot 10^{-7}$
recipe	91	180	663	0.006	0.08	$1.1 \cdot 10^{-15}$	0.005	$2.4 \cdot 10^{-9}$	0.008	$9.9 \cdot 10^{-7}$
sc105	105	103	280	0.006	0.06	$1.4 \cdot 10^{-16}$	0.005	$1.6 \cdot 10^{-9}$	0.02	$1.5 \cdot 10^{-5}$
sc205	205	203	551	0.010	0.18	0	0.008	$7.8 \cdot 10^{-11}$	0.09	$1.3 \cdot 10^{-4}$
sc50a	50	48	130	0.003	0.02	$2.2 \cdot 10^{-16}$	0.005	$5.3 \cdot 10^{-11}$	0.002	$3.4 \cdot 10^{-4}$
sc50b	50	48	118	0.003	0.04	$2.0 \cdot 10^{-16}$	0.003	$5.3 \cdot 10^{-11}$	0.003	$2.8 \cdot 10^{-5}$
scagr25	471	500	1554	0.04	0.45	$5.1 \cdot 10^{-16}$	0.02	$9.9 \cdot 10^{-10}$	0.25	$2.3 \cdot 10^{-4}$
scagr7	129	140	420	0.01	0.06	$2.2 \cdot 10^{-15}$	0.010	$8.9 \cdot 10^{-11}$	0.02	$3.3 \cdot 10^{-4}$
scfxm1	330	457	2589	0.04	0.36	$2.0 \cdot 10^{-16}$	0.02	$4.5 \cdot 10^{-12}$	0.11	$2.0 \cdot 10^{-4}$
scfxm2	660	914	5183	0.10	0.76	$2.0 \cdot 10^{-16}$	0.05	$8.0 \cdot 10^{-10}$	0.40	$1.7 \cdot 10^{-5}$
scfxm3	990	1371	7777	0.18	1.37	$2.7 \cdot 10^{-16}$	0.06	$2.0 \cdot 10^{-10}$	0.69	$1.0 \cdot 10^{-5}$
scorpion	388	358	1426	0.01	0.28	$2.4 \cdot 10^{-16}$	0.01	$7.9 \cdot 10^{-11}$	0.28	$1.6 \cdot 10^{-5}$
scrs8	490	1169	3182	0.05	1.41	$3.8 \cdot 10^{-16}$	0.02	$9.8 \cdot 10^{-10}$	0.24 [†]	$3.3 \cdot 10^{-3}$
scsd1	77	760	2388	0.01	0.11	$2.0 \cdot 10^{-16}$	0.007	$5.2 \cdot 10^{-11}$	0.006	$2.0 \cdot 10^{-5}$
scsd6	147	1350	4316	0.04	0.25	$2.2 \cdot 10^{-11}$	0.01	$3.3 \cdot 10^{-9}$	0.04	$1.3 \cdot 10^{-5}$
scsd8	397	2750	8584	0.24	0.92	$1.3 \cdot 10^{-16}$	0.02	$5.4 \cdot 10^{-11}$	0.32	$3.2 \cdot 10^{-5}$
sctap1	300	480	1692	0.04	0.25	0	0.01	$3.1 \cdot 10^{-11}$	0.02	$9.6 \cdot 10^{-5}$
sctap2	1090	1880	6714	0.10	0.88	0	0.03	$5.3 \cdot 10^{-11}$	0.03	$3.5 \cdot 10^{-5}$
sctap3	1480	2480	8874	0.07	0.66	0	0.04	$3.4 \cdot 10^{-12}$	0.03	$1.9 \cdot 10^{-5}$
seba	515	1028	4352	0.02	0.47	$1.2 \cdot 10^{-16}$	0.27	$2.2 \cdot 10^{-11}$	0.10	$4.8 \cdot 10^{-6}$
share1b	117	225	1151	0.009	0.09	0	0.01	$6.8 \cdot 10^{-12}$	0.12	$1.5 \cdot 10^{-5}$
share2b	96	79	694	0.006	0.04	$2.7 \cdot 10^{-16}$	0.01	$2.7 \cdot 10^{-12}$	0.18	$7.9 \cdot 10^{-5}$
shell	536	1775	3556	0.03	0.36	0	0.02	$4.2 \cdot 10^{-9}$	0.17	$4.6 \cdot 10^{-6}$
ship04l	402	2118	6332	0.02	0.26	$1.3 \cdot 10^{-16}$	0.03	$5.0 \cdot 10^{-9}$	0.11	$1.4 \cdot 10^{-4}$
ship04s	402	1458	4352	0.02	0.19	$1.3 \cdot 10^{-16}$	0.02	$4.5 \cdot 10^{-11}$	0.12	$1.1 \cdot 10^{-4}$
ship08l	778	4283	12802	0.06	0.55	$3.7 \cdot 10^{-16}$	0.04	$8.1 \cdot 10^{-10}$	0.26	$1.6 \cdot 10^{-4}$
ship08s	778	2387	7114	0.03	0.30	$1.1 \cdot 10^{-15}$	0.03	$3.0 \cdot 10^{-10}$	0.15	$9.3 \cdot 10^{-5}$
ship12l	1151	5427	16170	0.09	0.87	$1.3 \cdot 10^{-15}$	0.06	$2.3 \cdot 10^{-11}$	0.91	$2.8 \cdot 10^{-5}$
ship12s	1151	2763	8178	0.11	0.53	$2.5 \cdot 10^{-15}$	0.04	$4.2 \cdot 10^{-12}$	0.35	$4.1 \cdot 10^{-5}$
sierra	1227	2036	7302	0.04	0.45	$1.2 \cdot 10^{-16}$	0.06	$2.3 \cdot 10^{-10}$	0.26	$2.6 \cdot 10^{-4}$
stair	356	467	3856	0.04	0.26	$4.3 \cdot 10^{-15}$	0.07	$7.3 \cdot 10^{-9}$	0.67	$5.5 \cdot 10^{-5}$
standata	359	1075	3031	0.02	0.19	0	0.01	$5.7 \cdot 10^{-11}$	0.03	$1.1 \cdot 10^{-4}$
standgub	361	1184	3139	0.02	0.23	0	0.01	$1.9 \cdot 10^{-11}$	0.02	$1.1 \cdot 10^{-4}$
standmps	467	1075	3679	0.02	0.20	0	0.02	$3.9 \cdot 10^{-9}$	0.07	$1.3 \cdot 10^{-4}$
stocfor1	117	111	447	0.007	0.07	$1.8 \cdot 10^{-16}$	0.005	$3.3 \cdot 10^{-12}$	0.05	$3.4 \cdot 10^{-5}$
stocfor2	2157	2031	8343	0.09	2.08	$1.9 \cdot 10^{-15}$	0.09	$5.0 \cdot 10^{-10}$	1.54	$8.5 \cdot 10^{-6}$
tuff	333	587	4520	0.02	0.18	$1.7 \cdot 10^{-16}$	0.07	$1.1 \cdot 10^{-11}$	1.81	$1.3 \cdot 10^{-7}$
vtplibase	198	203	908	0.006	0.08	$2.2 \cdot 10^{-16}$	0.02	$9.2 \cdot 10^{-11}$	0.40	$1.0 \cdot 10^{-5}$

Problem	m	n	nnz	HiGHS	time	rel. err	time	rel. err	time	rel. err
wood1p	244	2594	70215	0.16	0.46	$4.8 \cdot 10^{-15}$	0.19	$3.8 \cdot 10^{-11}$	1.01	$2.0 \cdot 10^{-4}$

Table 15: MIPLIB 3: every instance run so far (miplib3_bnb_v1.csv, 60 s limit). “ref” is HiGHS’s outcome under the same limit (opt: proved optimal; TL: time limit).

Instance	m	n	int	status	gap	rel. err	nodes	time	HiGHS	ref
10teams	230	2025	1800	time_limit [†]	–	–	288	60.0	1.89	opt
air03	124	10757	10757	optimal	0	0	3	1.23	3.68	opt
air04	823	8904	8904	time_limit [†]	$1.4 \cdot 10^{-2}$	$3.5 \cdot 10^{-3}$	54	61.7	60.1	TL
air05	426	7195	7195	time_limit [†]	$5.6 \cdot 10^{-2}$	$4.3 \cdot 10^{-2}$	69	60.4	42.9	opt
arki001	1048	1388	538	time_limit [†]	–	–	3196	60.0	60.0	TL
bell3a	123	133	71	time_limit [†]	$1.0 \cdot 10^{-3}$	$1.6 \cdot 10^{-14}$	29291	60.0	1.28	opt
bell5	91	104	58	time_limit [†]	$4.0 \cdot 10^{-3}$	$3.1 \cdot 10^{-3}$	24970	60.1	1.34	opt
blend2	274	353	264	optimal	0	$1.2 \cdot 10^{-16}$	4985	48.1	4.15	opt
cap6000	2176	6000	6000	optimal	0	$1.9 \cdot 10^{-5}$	23	11.8	3.89	opt
dano3mip	3202	13873	552	time_limit [†]	–	–	0	60.1	60.1	TL
danoint	664	521	56	time_limit [†]	–	–	109	60.7	60.0	TL
dcmulti	290	548	75	optimal	0	$3.7 \cdot 10^{-5}$	111	8.49	2.90	opt
dsbmip	1182	1886	192	optimal	0	$3.4 \cdot 10^{-15}$	300	56.4	1.09	opt
egout	98	141	55	optimal	0	$4.0 \cdot 10^{-16}$	23	0.93	0.06	opt
enigma	21	100	100	optimal	0	0	1250	7.19	0.59	opt
fast0507	507	63009	63009	time_limit [†]	–	–	0	61.2	60.0	TL
fiber	363	1298	1254	time_limit [†]	$2.3 \cdot 10^{-1}$	$2.5 \cdot 10^{-1}$	674	60.1	3.25	opt
fixnet6	478	878	378	time_limit [†]	$5.9 \cdot 10^{-1}$	$8.3 \cdot 10^{-1}$	2235	60.0	7.30	opt
flugpl	18	18	11	optimal	0	0	295	0.27	0.17	opt
gen	780	870	150	optimal	0	0	201	5.90	0.04	opt
gesa2	1392	1224	408	time_limit [†]	$5.2 \cdot 10^{-2}$	$5.2 \cdot 10^{-2}$	2121	60.0	1.92	opt
gesa2_o	1248	1224	720	time_limit [†]	$2.1 \cdot 10^{-2}$	$2.1 \cdot 10^{-2}$	3800	60.1	3.30	opt
gesa3	1368	1152	384	optimal	0	$2.7 \cdot 10^{-14}$	1282	37.3	3.28	opt
gesa3_o	1224	1152	672	optimal	0	$5.3 \cdot 10^{-16}$	294	13.5	2.48	opt
gt2	29	188	188	optimal	0	0	846	1.71	0.07	opt
harp2	112	2993	2993	time_limit [†]	$3.2 \cdot 10^{-2}$	$2.8 \cdot 10^{-2}$	2238	60.1	24.6	opt
khh05250	101	1350	24	optimal	0	0	85	1.68	0.39	opt
l152lav	97	1989	1989	optimal	0	0	569	56.3	4.20	opt
lseu	28	89	89	optimal	0	$2.0 \cdot 10^{-16}$	38816	38.9	0.46	opt
markshare1	6	62	50	time_limit [†]	1.0	1.6	53129	60.1	60.0	TL
markshare2	7	74	60	time_limit [†]	–	–	0	60.0	60.0	TL
mas74	13	151	150	time_limit [†]	$1.0 \cdot 10^{-1}$	$1.5 \cdot 10^{-2}$	42525	60.1	60.0	TL
mas76	12	151	150	time_limit [†]	$5.3 \cdot 10^{-2}$	$3.8 \cdot 10^{-2}$	13338	23082.5	60.0	TL
misc03	96	160	159	optimal	0	$6.8 \cdot 10^{-16}$	914	31.0	1.47	opt
misc06	820	1808	112	optimal	0	$1.7 \cdot 10^{-5}$	54	4.13	0.44	opt
misc07	212	260	259	time_limit [†]	$3.7 \cdot 10^{-1}$	$1.7 \cdot 10^{-1}$	1709	60.0	56.9	opt
mitre	2054	10724	10724	time_limit [†]	–	–	48	60.8	3.49	opt
mkc	3411	5325	5323	time_limit [†]	–	–	0	60.8	60.0	TL
mod008	6	319	319	time_limit [†]	$6.5 \cdot 10^{-3}$	$7.6 \cdot 10^{-15}$	14189	60.0	8.09	opt
mod010	146	2655	2655	optimal	0	0	1	6.41	3.81	opt
mod011	4480	10958	96	time_limit [†]	$6.2 \cdot 10^7$	1.0	6	64.1	60.1	TL
modglob	291	422	98	time_limit [†]	$9.2 \cdot 10^{-2}$	$9.5 \cdot 10^{-2}$	6088	60.2	3.96	opt
noswot	182	128	100	time_limit [†]	$4.9 \cdot 10^{-2}$	0	16784	60.1	60.0	TL

Instance	m	n	int	status	gap	rel. err	nodes	time	HiGHS	ref
nw04	36	87482	87482	optimal [†]	0	–	1	23.8	91.2	TL
p0033	16	33	33	optimal	0	$2.9 \cdot 10^{-16}$	445	0.77	0.08	opt
p0201	133	201	201	optimal	0	$3.8 \cdot 10^{-13}$	432	20.4	1.82	opt
p0282	241	282	282	optimal	0	0	1190	11.4	0.91	opt
p0548	176	548	548	optimal	0	0	7378	55.3	0.20	opt
p2756	755	2756	2756	time_limit [†]	$8.2 \cdot 10^{-1}$	4.6	1042	60.2	1.14	opt
pk1	45	86	55	time_limit [†]	$7.7 \cdot 10^{-1}$	$3.6 \cdot 10^{-1}$	24629	60.1	60.0	TL
pp08a	136	240	64	time_limit [†]	$6.9 \cdot 10^{-2}$	$4.6 \cdot 10^{-2}$	11286	60.0	5.07	opt
pp08aCUTS	246	240	64	time_limit [†]	$1.2 \cdot 10^{-1}$	$1.1 \cdot 10^{-1}$	14520	60.1	4.15	opt
qiu	1192	840	48	time_limit [†]	6.4	1.9	146	60.6	60.0	TL
qnet1	503	1541	1417	time_limit [†]	$3.0 \cdot 10^{-1}$	$3.3 \cdot 10^{-1}$	36	60.6	4.02	opt
qnet1_o	456	1541	1417	time_limit [†]	–	–	0	60.8	4.06	opt
rentacar	6803	9557	55	time_limit [†]	–	–	27	70.7	43.6	opt
rgn	24	180	100	optimal	0	$3.8 \cdot 10^{-14}$	5733	48.5	1.81	opt
rout	291	556	315	time_limit [†]	$5.1 \cdot 10^{-1}$	$8.4 \cdot 10^{-1}$	244	60.1	60.0	TL
set1ch	492	712	240	time_limit [†]	$8.2 \cdot 10^{-2}$	$6.2 \cdot 10^{-2}$	1610	60.2	2.32	opt
seymour	4944	1372	1372	time_limit [†]	–	–	0	62.7	60.0	TL
stein27	118	27	27	optimal	0	$9.9 \cdot 10^{-16}$	4533	7.98	1.59	opt
stein45	331	45	45	time_limit [†]	$1.2 \cdot 10^{-1}$	$6.7 \cdot 10^{-2}$	10163	60.0	60.0	TL
swath	884	6805	6724	time_limit [†]	–	–	0	60.4	60.2	TL
vpml	234	378	168	optimal	0	0	2749	55.5	0.18	opt

Table 16: Maros–Mészáros convex QPs: every problem (maros_qpipm_v2.csv). “Clarabel” is the reference’s own status; [‡]: the MPS parsers disagree on this file and the reference solves NIRNAY’s reading (Section 12).

Problem	m	n	nnz	status	Clarabel	rel. err	iters	time	ref. time
AUG2D	10000	20200	40000	optimal	Solved	$1.1 \cdot 10^{-14}$	2	0.91	0.13
AUG2DC	10000	20200	40000	optimal	Solved	$1.9 \cdot 10^{-15}$	2	0.94	0.13
AUG2DCQP	10000	20200	40000	optimal	Solved	$1.7 \cdot 10^{-10}$	16	2.16	1.03
AUG2DQP	10000	20200	40000	optimal	Solved	$2.0 \cdot 10^{-9}$	17	2.96	1.29
AUG3D	1000	3873	6546	optimal	Solved	$2.5 \cdot 10^{-15}$	2	0.14	0.02
AUG3DC	1000	3873	6546	optimal	Solved	$3.0 \cdot 10^{-14}$	2	0.17	0.02
AUG3DCQP	1000	3873	6546	optimal	Solved	$8.4 \cdot 10^{-10}$	12	0.34	0.15
AUG3DQP	1000	3873	6546	optimal	Solved	$1.3 \cdot 10^{-9}$	13	0.22	0.09
BOYD1	18	93261	558985	optimal	AlmostSolved	$1.5 \cdot 10^{-7}$	20	7.27	12.0
BOYD2	186531	93263	423784	numerical_error [†]	MaxIterations	$8.8 \cdot 10^{-1}$	39	24.1	61.8
CONT-050	2401	2597	12005	optimal	Solved	$1.2 \cdot 10^{-10}$	11	1.01	0.43
CONT-100	9801	10197	49005	optimal	Solved	$5.0 \cdot 10^{-10}$	11	5.88	8.38
CONT-101	10098	10197	49599	optimal	Solved	$5.6 \cdot 10^{-12}$	11	7.74	9.54
CONT-200	39601	40397	198005	optimal	Solved	$4.2 \cdot 10^{-12}$	13	62.7	45.5
CONT-201	40198	40397	199199	optimal	Solved	$1.1 \cdot 10^{-12}$	13	75.1	50.9
CONT-300	–	–	–	timeout [†]	Solved	–	–	150.1	69.4
CVXQP1_L	5000	10000	14998	time_limit [†]	Solved	$1.2 \cdot 10^{-1}$	6	129.2	21.7
CVXQP1_M	500	1000	1498	optimal	Solved	$1.7 \cdot 10^{-9}$	12	0.92	0.90
CVXQP1_S	50	100	148	optimal	Solved	$1.5 \cdot 10^{-12}$	9	0.03	0.006
CVXQP2_L	2500	10000	7499	time_limit [†]	Solved	$4.5 \cdot 10^{-6}$	9	120.8	13.9
CVXQP2_M	250	1000	749	optimal	Solved	$2.8 \cdot 10^{-10}$	10	0.48	0.65
CVXQP2_S	25	100	74	optimal	Solved	$2.9 \cdot 10^{-11}$	10	0.04	0.006

Problem	m	n	nnz	status	Clarabel	rel. err	iters	time	ref. time
CVXQP3_L	7500	10000	22497	time_limit [†]	Solved	$1.8 \cdot 10^{-2}$	5	131.2	14.2
CVXQP3_M	750	1000	2247	optimal	Solved	$1.4 \cdot 10^{-10}$	14	1.27	1.19
CVXQP3_S	75	100	222	optimal	Solved	$5.5 \cdot 10^{-11}$	10	0.04	0.01
DPKLO1	77	133	1575	optimal	Solved [§]	$2.9 \cdot 10^{-15}$	3	0.03	0.003
DTOC3	9998	14999	34993	optimal	Solved	$1.9 \cdot 10^{-15}$	4	0.29	0.04
DUAL1	1	85	85	optimal	Solved	$4.8 \cdot 10^{-10}$	10	0.02	0.006
DUAL2	1	96	96	optimal	Solved	$1.1 \cdot 10^{-12}$	8	0.02	0.008
DUAL3	1	111	111	optimal	Solved	$5.2 \cdot 10^{-10}$	9	0.03	0.02
DUAL4	1	75	75	optimal	Solved	$1.1 \cdot 10^{-11}$	9	0.01	0.006
DUALC1	215	9	1935	optimal	Solved	$1.9 \cdot 10^{-12}$	9	0.02	0.003
DUALC2	229	7	1603	optimal	Solved	$3.9 \cdot 10^{-13}$	9	0.02	0.003
DUALC5	278	8	2224	optimal	Solved	$6.5 \cdot 10^{-12}$	7	0.01	0.003
DUALC8	503	8	4024	optimal	Solved	$5.5 \cdot 10^{-11}$	7	0.02	0.006
EXDATA	3001	3000	7500	optimal	Solved	$4.5 \cdot 10^{-7}$	20	34.4	4.19
GENHS28	8	10	24	optimal	Solved	$1.1 \cdot 10^{-16}$	2	0.003	0.000
GOULDQP2	349	699	1047	optimal	Solved	$6.3 \cdot 10^{-10}$	13	0.07	0.02
GOULDQP3	349	699	1047	optimal	Solved	$4.1 \cdot 10^{-10}$	10	0.07	0.009
HS118	17	15	39	optimal	Solved	$3.2 \cdot 10^{-10}$	9	0.010	0.001
HS21	1	2	2	optimal	Solved	$2.7 \cdot 10^{-12}$	9	0.008	0.000
HS268	5	5	25	optimal	Solved	$2.0 \cdot 10^{-7}$	12	0.01	0.000
HS35	1	3	3	optimal	Solved	$1.2 \cdot 10^{-10}$	6	0.004	0.000
HS35MOD	1	3	3	optimal	Solved	$2.2 \cdot 10^{-9}$	11	0.006	0.000
HS51	3	5	7	optimal	Solved	$2.7 \cdot 10^{-15}$	2	0.003	0.000
HS52	3	5	7	optimal	Solved	$1.7 \cdot 10^{-16}$	2	0.003	0.000
HS53	3	5	7	optimal	Solved	$1.7 \cdot 10^{-15}$	6	0.005	0.000
HS76	3	4	10	optimal	Solved	$7.1 \cdot 10^{-11}$	6	0.006	0.000
HUES-MOD	2	10000	20000	numerical_error [†]	Solved [§]	$1.9 \cdot 10^1$	26	0.38	0.13
HUESTIS	2	10000	20000	numerical_error [†]	Solved [§]	$5.9 \cdot 10^{-1}$	27	0.38	0.11
KSIP	1001	20	19898	numerical_error [†]	Solved [§]	7.5	34	0.30	0.04
LASER	1000	1002	3000	optimal	Solved	$1.3 \cdot 10^{-9}$	16	0.14	0.02
LISWET1	10000	10002	30000	numerical_error [†]	AlmostSolved	$4.6 \cdot 10^{-2}$	38	1.21	1.35
LISWET10	10000	10002	30000	numerical_error [†]	AlmostSolved	$1.7 \cdot 10^{-3}$	40	1.15	0.33
LISWET11	10000	10002	30000	numerical_error [†]	AlmostSolved	$2.5 \cdot 10^{-1}$	39	1.23	1.42
LISWET12	10000	10002	30000	numerical_error [†]	AlmostSolved	$8.9 \cdot 10^{-1}$	37	1.17	1.06
LISWET2	10000	10002	30000	optimal	AlmostSolved	$6.4 \cdot 10^{-7}$	22	0.76	0.27
LISWET3	10000	10002	30000	optimal	Solved	$2.9 \cdot 10^{-7}$	15	0.53	0.25
LISWET4	10000	10002	30000	optimal	AlmostSolved	$3.2 \cdot 10^{-8}$	14	0.63	0.58
LISWET5	10000	10002	30000	optimal	Solved	$1.3 \cdot 10^{-8}$	18	0.69	0.21
LISWET6	10000	10002	30000	optimal	AlmostSolved	$6.1 \cdot 10^{-9}$	19	0.73	0.34
LISWET7	10000	10002	30000	numerical_error [†]	AlmostSolved	$4.0 \cdot 10^{-1}$	38	1.19	1.39
LISWET8	10000	10002	30000	numerical_error [†]	AlmostSolved	$5.4 \cdot 10^{-1}$	38	1.17	1.30
LISWET9	10000	10002	30000	numerical_error [†]	AlmostSolved	$9.5 \cdot 10^{-1}$	37	1.16	1.35
LOTSCHD	7	12	54	optimal	Solved	$9.1 \cdot 10^{-12}$	9	0.006	0.000
MOSARQP1	700	2500	3422	optimal	Solved	$7.0 \cdot 10^{-11}$	10	0.13	0.04
MOSARQP2	600	900	2930	optimal	Solved	$3.7 \cdot 10^{-9}$	9	0.10	0.02
POWELL20	10000	10000	20000	optimal	Solved	$1.8 \cdot 10^{-9}$	54	1.23	0.27
PRIMAL1	85	325	5815	optimal	Solved	$1.5 \cdot 10^{-9}$	13	0.06	0.02
PRIMAL2	96	649	8042	optimal	Solved	$1.7 \cdot 10^{-9}$	11	0.07	0.02
PRIMAL3	111	745	21547	optimal	Solved	$1.1 \cdot 10^{-9}$	12	0.17	0.07

Problem	m	n	nnz	status	Clarabel	rel. err	iters	time	ref. time
PRIMAL4	75	1489	16031	optimal	Solved	$8.5 \cdot 10^{-10}$	10	0.16	0.05
PRIMALC1	9	230	2070	optimal	Solved	$1.6 \cdot 10^{-10}$	15	0.03	0.004
PRIMALC2	7	231	1617	optimal	Solved	$8.6 \cdot 10^{-9}$	17	0.03	0.005
PRIMALC5	8	287	2296	optimal	Solved	$1.5 \cdot 10^{-11}$	14	0.02	0.007
PRIMALC8	8	520	4160	optimal	Solved	$7.0 \cdot 10^{-11}$	15	0.09	0.03
Q25FV47	820	1571	10400	optimal	Solved	$1.3 \cdot 10^{-11}$	37	5.20	2.39
QADLITTL	56	97	383	optimal	Solved	$2.4 \cdot 10^{-9}$	12	0.03	0.007
QAFIRO	27	32	83	optimal	Solved	$1.4 \cdot 10^{-12}$	12	0.03	0.002
QBANDM	305	472	2494	optimal	Solved	$3.7 \cdot 10^{-11}$	25	0.16	0.05
QBEACONF	173	262	3375	optimal	Solved	$3.6 \cdot 10^{-10}$	16	0.12	0.09
QBORE3D	233	315	1429	optimal	Solved	$2.4 \cdot 10^{-10}$	16	0.09	0.04
QBRANDY	220	249	2148	optimal	Solved	$5.2 \cdot 10^{-11}$	20	0.13	0.03
QCAPRI	271	353	1767	optimal	Solved	$7.1 \cdot 10^{-11}$	58	0.34	0.08
QE226	223	282	2578	optimal	Solved	$6.4 \cdot 10^{-10}$	16	0.10	0.06
QETAMACR	400	688	2409	optimal	Solved	$3.2 \cdot 10^{-9}$	33	0.49	0.83
QFFFFF80	524	854	6227	optimal	Solved	$5.4 \cdot 10^{-11}$	28	0.65	0.28
QFORPLAN	161	421	4563	optimal	Solved	$1.2 \cdot 10^{-9}$	24	0.19	0.08
QGFRDXPN	616	1092	2377	optimal	Solved	$6.9 \cdot 10^{-10}$	22	0.22	0.09
QGROW15	300	645	5620	optimal	Solved	$6.3 \cdot 10^{-14}$	19	0.21	0.11
QGROW22	440	946	8252	optimal	Solved	$1.2 \cdot 10^{-13}$	22	0.37	0.16
QGROW7	140	301	2612	optimal	Solved	$6.6 \cdot 10^{-11}$	17	0.12	0.05
QISRAEL	174	142	2269	optimal	AlmostSolved	$1.9 \cdot 10^{-8}$	24	0.12	0.38
QPCBLEND	74	83	491	optimal	Solved	$1.1 \cdot 10^{-9}$	15	0.04	0.008
QPCBOEI1	351	384	3485	optimal	Solved	$1.2 \cdot 10^{-9}$	27	0.22	0.07
QPCBOEI2	166	143	1196	optimal	Solved	$3.8 \cdot 10^{-11}$	25	0.10	0.03
QPCSTAIR	356	467	3856	optimal	Solved	$2.5 \cdot 10^{-10}$	30	0.25	0.08
QPILOTNO	975	2172	13057	optimal	Solved	$2.9 \cdot 10^{-10}$	34	1.31	5.12
QPTTEST	2	2	4	optimal	Solved	$3.1 \cdot 10^{-12}$	6	0.01	0.001
QRECIPE	91	180	663	optimal	Solved	$2.7 \cdot 10^{-9}$	13	0.06	0.01
QSC205	205	203	551	optimal	Solved	$8.0 \cdot 10^{-9}$	13	0.06	0.01
QSCAGR25	471	500	1554	optimal	Solved	$2.9 \cdot 10^{-11}$	20	0.10	0.04
QSCAGR7	129	140	420	optimal	Solved	$1.8 \cdot 10^{-10}$	19	0.06	0.009
QSCFXM1	330	457	2589	optimal	Solved	$7.9 \cdot 10^{-11}$	39	0.26	0.05
QSCFXM2	660	914	5183	optimal	Solved	$1.2 \cdot 10^{-9}$	44	0.58	0.19
QSCFXM3	990	1371	7777	optimal	Solved	$3.2 \cdot 10^{-9}$	46	0.67	0.26
QSCORPIO	388	358	1426	optimal	Solved	$4.2 \cdot 10^{-11}$	14	0.08	0.02
QSCRS8	490	1169	3182	optimal	Solved	$2.0 \cdot 10^{-9}$	23	0.24	0.14
QSCSD1	77	760	2388	optimal	Solved	$6.8 \cdot 10^{-10}$	10	0.10	0.03
QSCSD6	147	1350	4316	optimal	Solved	$6.7 \cdot 10^{-10}$	14	0.22	0.06
QSCSD8	397	2750	8584	optimal	Solved	$6.6 \cdot 10^{-12}$	13	0.35	0.12
QSCTAP1	300	480	1692	optimal	Solved	$1.2 \cdot 10^{-9}$	17	0.10	0.03
QSCTAP2	1090	1880	6714	optimal	Solved	$5.8 \cdot 10^{-11}$	14	0.36	0.10
QSCTAP3	1480	2480	8874	optimal	Solved	$1.1 \cdot 10^{-10}$	15	0.50	0.14
QSEBA	515	1028	4352	optimal	Solved	$4.8 \cdot 10^{-11}$	42	0.35	0.12
QSHARE1B	117	225	1151	optimal	Solved	$7.0 \cdot 10^{-11}$	21	0.09	0.05
QSHARE2B	96	79	694	optimal	Solved	$8.3 \cdot 10^{-11}$	26	0.08	0.007
QSHELL	536	1775	3556	optimal	Solved	$5.2 \cdot 10^{-10}$	38	0.16	0.39
QSHIP04L	402	2118	6332	optimal	Solved	$6.1 \cdot 10^{-12}$	19	0.08	0.03
QSHIP04S	402	1458	4352	optimal	Solved	$8.2 \cdot 10^{-10}$	17	0.07	0.01

Problem	m	n	nnz	status	Clarabel	rel. err	iters	time	ref. time
QSHIP08L	778	4283	12802	optimal	Solved	$2.1 \cdot 10^{-10}$	20	0.60	0.55
QSHIP08S	778	2387	7114	optimal	Solved	$2.2 \cdot 10^{-10}$	18	0.16	0.06
QSHIP12L	1151	5427	16170	optimal	Solved	$8.2 \cdot 10^{-10}$	19	1.00	0.84
QSHIP12S	1151	2763	8178	optimal	Solved	$8.5 \cdot 10^{-10}$	18	0.19	0.07
QSIERRA	1227	2036	7302	optimal	AlmostSolved	$4.7 \cdot 10^{-11}$	22	0.11	0.08
QSTAIR	356	467	3856	optimal	Solved	$4.1 \cdot 10^{-10}$	46	0.08	0.04
QSTANDAT	359	1075	3031	optimal	Solved	$2.1 \cdot 10^{-11}$	14	0.07	0.01
S268	5	5	25	optimal	Solved	$2.0 \cdot 10^{-7}$	12	0.004	0.000
STADAT1	3999	2001	9997	optimal	Solved	$2.5 \cdot 10^{-11}$	78	0.39	0.03
STADAT2	3999	2001	9997	optimal	Solved	$2.8 \cdot 10^{-11}$	23	0.16	0.08
STADAT3	7999	4001	19997	optimal	Solved	$9.1 \cdot 10^{-10}$	22	0.28	0.21
STCQP1	2052	4097	13338	optimal	Solved	$3.5 \cdot 10^{-10}$	11	0.26	0.33
STCQP2	2052	4097	13338	optimal	Solved	$5.2 \cdot 10^{-10}$	13	0.50	0.43
TAME	1	2	2	optimal	Solved	0	5	0.002	0.000
UBH1	12000	18009	48000	numerical_error [†]	AlmostSolved	$1.3 \cdot 10^{-1}$	99	1.84	0.24
VALUES	1	202	202	optimal	Solved	$2.4 \cdot 10^{-11}$	13	0.02	0.004
YAO	2000	2002	6000	numerical_error [†]	AlmostSolved	$8.2 \cdot 10^{-1}$	38	0.13	0.03
ZECEVIC2	2	2	4	optimal	Solved	$3.3 \cdot 10^{-13}$	6	0.002	0.000

B Typesetting toolchain

The report is built with pdfL^AT_EX from TeX Live 2023 by latexmk. Every figure is drawn in TikZ or PGFPlots from source, with no raster images, and every plot and generated table reads files that gen_data.py writes from results/*.csv. The packages were chosen after checking their CTAN pages for maintenance status and licence, and for how widely they are used on TeX StackExchange, where tikz-pgf and pgfplots are among the most-used tags.

Package	Why it was chosen
TikZ/PGF 3.1	The standard native drawing system for $\text{\LaTeX}$ (actively maintained, LPPL/GPL). Used with the libraries <code>positioning</code> , <code>arrows.meta</code> , <code>shapes.geometric</code> , <code>shapes.misc</code> , <code>fit</code> , <code>calc</code> , <code>backgrounds</code> , <code>matrix</code> and <code>decorations.pathreplacing</code> for the architecture, data-flow, flowchart, KKT and fill-in figures. https://ctan.org/pkg/pgf
PGFPlots 1.18	Plots computed by $\text{\TeX}$ from data files, in the document's fonts; log axes, <code>groupplots</code> for the scatter panel, <code>const plot</code> for profiles and ECDFs. https://ctan.org/pkg/pgfplots
pgfplotstable	Reads CSV/whitespace tables directly (tick labels read from the <code>.dat</code> files). https://ctan.org/pkg/pgfplotstable
forest	Trees from a bracket notation with automatic, compact packing; used for the branch-and-bound tree. https://ctan.org/pkg/forest
algorithm2e	Floating pseudocode with numbered lines and block rules, referenced with <code>cleveref</code> . It was preferred to <code>algorithmicx</code> , whose CTAN entry lists its maintainer as inactive. https://ctan.org/pkg/algorithm2e
booktabs, longtable, tabularx	Professional table rules; multi-page appendix tables. https://ctan.org/pkg/booktabs
siunitx	Consistent typesetting of the generated numbers (<code>\num{1.7e-16}</code>). https://ctan.org/pkg/siunitx
tcolorbox	The summary, lesson and “honest” boxes. https://ctan.org/pkg/tcolorbox
listings	Code listings. https://ctan.org/pkg/listings
hyperref, cleveref	Links and automatic reference names (“Figure”, “Algorithm”). https://ctan.org/pkg/cleveref
microtype	Character protrusion and font expansion for even text colour. https://ctan.org/pkg/microtype
natbib + plainnat	Numbered citations with DOIs. <code>biblatex/biber</code> would be the modern choice, but the TeX Live installation on the build machine does not include <code>biblatex</code> , and <code>natbib</code> builds everywhere. https://ctan.org/pkg/natbib
libertinus, Source Sans/Code Pro	Open fonts with matching mathematics (Libertinus) and a sans-serif for headings and figures. https://ctan.org/pkg/libertinus

`tikz-cd` was considered and not used: none of the diagrams is a commutative diagram.

C Reproducing this report

```
# benchmark sweeps (each writes results/<suite>.csv)
python bench/run_lp.py data/netlib --method simplex --limit 300 --out results/
  netlib_simplex_v3.csv
python bench/run_lp.py data/netlib --method ipm      --limit 300 --out results/
  netlib_ipm_v2.csv
python bench/run_mip.py data/miplib --limit 60 --out results/miplib3_bnb_v2.csv
python bench/run_qp.py data/qp/mm  --limit 120 --out results/maros_qpmp_v1.csv
python bench/run_pdlp_gpu.py data/large --tol 1e-4 --limit 300 --out results/
  large_pdlp_gpu.csv
python tools/dual_check.py data/netlib/*.mps.gz > deliverables/report/data/
  dual_check.txt

# data for the report (picks the newest version of every sweep), then the PDF
cd deliverables/report
python gen_data.py
```

```
latexmk -pdf -interaction=nonstopmode main.tex
```

gen_data.py prefers the highest `_v<k>` version of each sweep, so a new sweep appears in the report as soon as its file exists. It was last run on 2026-09-30.